2D Physics engine

By James Wathen

Contents

Analysis 4

Project outline: 4

Initial research: 4

Forces I could implement 4

Further physics concepts I would need 5

The client and what he wants from the project: 7

General overview of what my client wants to see from the project and the workings of the engine: 7

Goals: 7

Initial Planning and prototyping 9

Prototyping: 9

Documented Design 11

Basic Summary: 11

Class details in UML style: 11

Polygon UML classes: 12

Circle UML classes: 16

Miscellaneous UML classes: 17

Simplified UML diagram using classes above: 19

Flow chart for majority of program 19

Windows forms and set up of the program 22

Graphics: 22

Timer: 22

How movement occurs 22

Polygon vertex changes: 22

Angular velocity: 23

Force objects (gravity): 24

Collision Detection 25
Finding the minkowski difference 26
Does the minkowski difference contain the origin? 27
Resolving the collision 31
Circles: 31
Polygons: 31
Adding objects 36
Inputting values 36
Technical solution 37
Some examples of coding skills: 37
Abstract data structures + stack/queue operations: 37
List operations: 37
Sort, search and recursion: 37
Complex user defined algorithms: 37
Dynamic generation of OOP objects: 37
Code: 38
Main classes: 38
Program.cs 38
Start.cs 38
Entity.cs 39
Force.cs 40
Point.cs 41
Circle simulation files: 41
CircleManager.cs 41
CircleEntity.cs 49
CircleEditor.cs 51
Polygon simulation files: 56
PolyForm.cs 56
PolyManager.cs 60
Polygon.cs 71

NewPolygon.cs 75
Simplex.cs 82
Polytope.cs 83
Border.cs 84
Collision.cs 84
Designer code for forms other than CircleManager: 84
Start.Designer.cs 84
PolyForm.Designer.cs 87
NewPolygon.Designer.cs 90
CircleEditor.Designer.cs 102
Testing 112
Test overview: 112
Tests with reference to original goals: 113
GUI 113
Customisation of objects data and gravity 115
Veriety of shapes ~ usability of inputting a new shape 118
Collisions at appropriate frame rate 120
Force of gravity ~ ability to change 125
Evaluation 127
Individual goal success: 127
Did I achieve my final project? 128
What would I change next time? 129
Final thoughts 129

Analysis

Project outline:

Title: 2D Physics Engine

The idea for this project is to create a 2D physics engine in Windows forms using the language, C#. It should allow people to easily play around with how physical scenarios could play out and have code that others can add to, to make their own physics-based games and programs.

Making this program will be a large learning curve for me and I expect to be researching a lot of code for it; this is because of the complex nature of the program since I will need to learn how to: generate images on windows forms quickly, implement moving objects by keeping track of data about each one and making frame rates to produce movement, designing algorithms to locate and equate collisions, and finally doing all of this (and likely more) efficiently to not sacrifice computer performance for the simulation to run smoothly at 60fps. This project is being made for my client that wants a physics engine as described previously so that the code can be implemented (similar to a library) in my clients own games or programs. The main reason that I have chosen to design this kind of program is because of all the challenges that will come with it since learning to overcome them should have major beneficial impacts on my coding, designing, and problem-solving skills.

Initial research:

The first piece of research I did was to make sure that my project was possible in Windows Forms, and I found the following video on YouTube of a 2d physics engine in Windows Forms:
https://www.youtube.com/watch?v=O_O9jhB9bcI
This showed me that my project was feasible to make with my initial requirements and that it could be a good project to improve throughout the year beyond my original plan since it can easily be improved with new features.

Forces I could implement

Here I have made a list of different forces I could consider adding to the program:

Types of forces (no date) *The Physics Classroom*. Available at: https://www.physicsclassroom.com/class/newtlaws/Lesson-2/Types-of-Forces (Accessed: 04 September 2024).
(This source was used for a list of forces that could be included.)

- Friction
  - To implement friction, I would need to make a variable of the roughness of objects, in a perfect physics engine this could be done practically, but for simplicity I may need to make a simple

speed-reducing algorithm that is affected by the roughness of the colliding objects depending on the material.

- Tension
  - I would hope that tension will just be a given with the rules I implement to the program as when a force is pulling on a material and not moving it, tension should be generated.
- Normal force
  - This is just contact, but I would need to be sure to calculate equal and opposite forces depending on the velocity of the two objects, therefore it can stop objects by them landing on them.
  - An issue I can for see though could be the possibility of loss of Kinetic energy, if I were to implement an energy value this may be a challenge to diminish due to the amount of things it could affect, kinetic energy would have to be diminished by the elasticity of the material meaning the sides would also need one.
- Air resistance
  - I will not want to make tens of thousands of small objects to simulate drag as this could cause run time issues; instead, I could simply make a speed reducing algorithm depending on what fluid an object is in. However, if I have time, it would be fine to add another version of fluids which is fully practical.
- Spring / elasticity
  - Certain materials could have different young modulus and therefore stretch and deform differently. However, I will not make this a prime goal since it could be quite challenging, instead, I may add something similar at the end if possible.
- Gravity
  - I want this to have the most change as it should be moveable and scalable, potentially via mass.
- Magnetic/electrostatic
  - If I implement this, it could be quite difficult due to how different materials are interacted by it; but if I can, I would like to try including it.

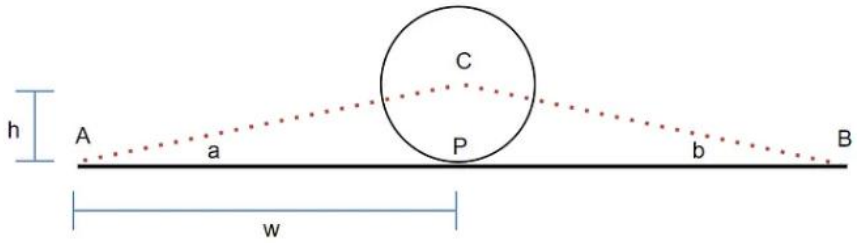
Further physics concepts I would need

https://medium.com/@www.seymour/coding-a-2d-physics-engine-from-scratch-and-using-it-to-simulate-a-pendulum-clock-964b4ac2107a - this source gives good pointers on what kinds of algorithms to use, but it does not tell me explicitly how to do a lot of them and I am not using the code given since it is in a different language to the one I am currently using.

- Units and measurements
  - I originally planned to use SI units since this would make use of known physics equations much easier, however in order to do this, functions for conversion between pixel position and metres, mass, newtons etc will be needed.
- Time in the simulation

- At first I just assumed to use a frames per second value and just stick all processing to that, but if I want to change more details about the code (maintain a framerate but change how much it runs per frame) is should include other variables such as:
- Sim multiple – the amount of times that the simulation runs between frames are rendered
- Speed which is the multiplier on the simulations speed relative to normal time, this would also be quite useful for testing as the simulation could be ran more quickly or slowly to be able to tell what is happening.
- By use of these values the time step can be derived which is the amount of times the simulation is ran per unit of time within the simulation – the higher the time step, the more accurate movements of objects and their collisions will be.

- Detecting collisions
  - Between two circles this is simple as we just find the distance between the centres of each circle, if D<the sum of the radii then there is a collision
  - Detecting for circles and a wall is simple in concept as they are colliding when distance between the circle and wall is less than or equal to the radius of the circle. However there is an issue in how to calculate this distance. We can only check for distances between points so we should use the ends of each line for this instead, and then we can use trig to find the distance to the actual line compared to the edge of the circle as well as angles of velocity vectors to the wall.



    - Good things to take note of are: if the angle to the end of a line < 90 a collision is with the line, if it is > 90 it must be with the end point
    - Also we can make use of vector dot product for finding the position of P with AC and AB

- Resulting elastic collisions
  - For vertical and horizontal walls this is easy as we just need to reverse the velocity component in that direction, but for angled walls we need to use a more complex equation: ((2 * M1) + (M2 * pow(M1, 2)) - M2) / (2 * M1 * M2 - pow(M1, 2) + 1), from [this stack overflow page] which can be explained
  - For collisions between objects this would need to first find the gradient of the line between the objects, and then equate the resultant forces for each object and then the vectors needed for those forces using the same algorithms as before
  - Thought I need to be able to simulate more than just balls and rigid lines. To do this things like angular velocity and momentum.

The client and what he wants from the project:

Client: Phil Dexter, Software Engineer

Phil wants to make use of the code of a 2d physics engine because he is a professional software developer and therefore can use the code to help him make a physics-based game.

General overview of what my client wants to see from the project and the workings of the engine:

They want a 2d rigid body simulation where objects react to each other realistically with consistent movement updates that run in a single threaded environment. They also want broad and narrow phase collision detection. The following information has been acquired via talking to my client about the different aspects of the program and how they would like them to be.

Simulation Manager

They want me to make a manager class which manages all the collisions and objects while being able to add new objects and remove them, the algorithms for which must be efficient with low time complexity.

Broad and Narrow Phase Collision Detection

I need to keep the complexity of collision detection reasonable to not brute force it and avoid making a complexity of $O(n^2)$. I should use an algorithm such as sweep and prune or grid-based approaches. Once the broad phase has been completed the narrow phase should be ran separately afterwards. Care should be taken to avoid fragmenting the heap.

Numerical Integration & Collision Response

Entities Should be responding to all forces consistently whether that is forces between collisions or external forces implemented by the user.

Object Representation

The simulation should support a wide variety of rigid-body shapes which can be added and removed and allow for implementation of new shapes to the program as the customer would like to extend the program. The client would also like me to allow for implementation of new collision and movement algorithms.

Graphical User Interface

There should be a minimal GUI for the user to use the simulation easily. I must also include documentation and testing for explanation of the algorithms and design decisions. Testing documentation should also be included to show how well the result works to the clients wishes.

Goals:

I personally want this program to contain:

1. Readable and intuitive GUI for the user

1.1. This should have a front menu that can be used to then access the simulation
1.2. Settings for the simulation are available at the start menu in an easy-to-access way, and during run time they settings are accessible intuitively via something like various drop-down lists.

2. A system where the forces and effects on the objects can be customized fully for any kind of desired physical environment by the user in real time.
    2.1. Each force can be scaled to any desired value by the user, though there will be an automatic setting for each which relate to normal physics on earth.
    2.2. This will work by using methods (or inherited classes) for each force and having a strength value for each of them which will be edited.
    2.3. Furthermore, I will make the effective range of the vector forces scale directly with the strength value
3. An environment where the user can place a large variety of sized objects which can have complex shapes.
    3.1. Each object will have an automatic size and be a circle, however the user should also be able to input more complex shapes, this will make collisions more complicated but should be worth it
    3.2. Circles would just have a centre of mass in the middle, but for other shapes, this would require a more complex algorithm, though it should not constantly be running as when a centre of mass is found it should not change for the object.
    3.3. The user should have an intuitive method for making whatever shape they want
    3.4. The best way would be a drawing editor within the program; this would work by the user clicking in places where corners will be and an algorithm to fill in the vertices and the shape itself.
    3.5. The shape size will be either by inputting a diameter or directly from the size of the shape drawn.
4. A collision system that changes velocities and kinetic energy of objects in realistic ways while maintaining acceptable performance at 60 frames per second for hundreds of objects.
    4.1. This system should imagine a shape simpler and larger for each object in the simulation and when that larger shape crosses another one, a more complex collision algorithm will take over to see if the actual shapes are touching and if they do, their new velocities and directions. - This system should make the program run smoother than constantly running a complex collision algorithm.
    4.2. Maintaining 60fps might be a challenge for performance but this just means I should improve my collision algorithm.
    4.3. The fps will be made by an elapsing timer every 1/60 of a second which will erase the current frame and draw the next frame which should have been calculated and stored at the end of the previous one.
5. A force of gravity that can be moved, resized, and changed in any way.

5.1. A more complex force to implement would be to make gravity exist for all objects to the point that you could make an object have an incredibly high mass and it will then attract others with mass.
5.2. The best way to do this will just be to place a gravity centre on the centre of mass for each object which will have a strength that is directly proportional to the mass of the object.
5.3. My only worry is that this could negatively affect performance so this would need to work efficiently.

6. Secure and readable code that can be updated in any way with ease by using good object orientated programming practices.
    6.1. I want all the forces, effects, and code to be in their own classes and methods to make improving the program easier since it will get quite complicated.
    6.2. I plan to make a class for all entities, this way they will all work seamlessly with other methods in the program.
    6.3. I will also make a class for forces. This class will then be inherited from for each of the different forces so that different instances/objects of gravity, friction, air resistance etc, can be made.
7. A database for storing material aspects in an efficient method
    7.1. I would like to implement a database into the program, the best way to do this could be to make a database of different materials.
    7.2. Each different material could have a different value for friction or density which would affect how objects with each material would affect the others around them during collisions.
    7.3. This database should be fully normalised and easily/quickly accessed by the program.

Initial Planning and prototyping

Prototyping:

My goal for a prototype is to just learn the necessary code relating to the project and make a series of balls that move around and collide with walls, but I won't make myself do any optimization, momentum transfers or alternate shapes to make sure I only use the prototype to teach myself the basics.
The prototype taught me how to:

- Use a timer in C# to repeatedly draw changing images in windows forms
- The logic behind how physics simulations work:
    - Store data about each object in the simulation in an object-oriented way so that one circle object can have its two velocity and acceleration components that will cause its position coordinates to change
    - After each elapse of the timer the new velocities and positions of the objects must be calculated and applied

    - The distances between objects must also be calculated to find out whether any two objects are currently colliding
    - If they are colliding then a collision must be resolved by use of the normal between the two objects and the current velocity of each to then find their new resultant velocities.
- Use windows forms in a more comprehensive way that allows for multiple forms to be opened and then share data and methods.

How I learnt:
I didn't have any knowledge of the kind of code needed to make a refresh rate for the simulation so I used this YouTube video to learn about how some different types of timers work in C# and how I can implement them. To gain further understanding of the code I used this Microsoft article which explains the C# code for a timer class. In order to learn drawing methods, I used other articles by Microsoft that describe how they work Pen making: Microsoft learn pen , Drawing an ellipse/ any other shape: Microsoft learn ellipse.

Wall collision:
Colliding the balls with the walls was very simple as I just needed to find the distance to one axis as each wall was vertical (distance to x=10, y=10,x=1500,y=1025 for example), then, depending on which wall the ball is colliding with, flip the velocity components (negative to positive or positive to negative).

Collisions between balls:
My first challenge with the project was figuring out a method of detecting collisions between the balls themselves; this is because I have to make an algorithm that can detect whether any part of a ball is in contact with any other part of every other ball, and then repeat this for the number of balls there are while also doing a similar thing for the walls of the simulation. My solution is far simpler than I thought as we just check if the distance between the balls was less than or equal to the sum or their radii. Distances could be found by just subtracting position coordinates

Finding the new direction of velocities: source of main equation
My next challenge was to find the line of where the two balls would reflect around which could be done by finding the gradient of the normal to the line the balls collide on by dividing the difference in y of the circle centres by the difference in x. Once I have the line that the velocities get reflected on I just need to find the gradient of the resultant velocity which is just the y component divided by the x component. I then use the matrix that I mentioned in my research, but I will instead use a direct equation form to simplify calculations, this looks like:

Where M3 is the reflected angle, M1 the gradient of the normal and M2 is the original gradient.
```
double M3 = ((2 * M1) + (M2 * pow(M1, 2)) - M2) / (2 * M1 * M2 - pow(M1, 2) + 1);
```

Documented Design

Both the prototype (with some minor improvements) and my final polygon simulation are in the final program for the two separate purposes that they serve. Due to this they are both made in slightly different ways, for example, the circle sim is made with a manager class that inherits from Form. But the polygon simulation (the final product) separates this into two, one for the form and one for the manager.

Basic Summary:

The program starts with asking users to pick between two simulations, a polygon sim which can use any shape and includes things such as momentum conversion, or the original prototype and circle-only simulation which has no physical energy changes and just calculates the movements and new directions of each circle after a collision.

After choosing the simulation you can add and remove physical and force field objects that can each affect that instance of the simulation. Once objects are added, they will be included in the algorithms for collision and force detection, if a collision is then found it will be added to a list of the class type Collision which is the gone through and resolved, this happens every 60th of a second or as many times as the user requires.

Objects can be added via either directly inputting data, or drawing where points of an object will be (only in polygon sim) and using some of the saved values from the inputted data.

Class details in UML style:

Entity
+velocity : float[] +acceleration : float[] +position : float[] + mass : float + colour : Pen
+Entity(Vx : float, Vy : float, Ax : float, Ay: float, Px : float, Py : float, Colour : Pen, mass : float) Assigns the values +updateVelocity() Changes velocity depending on acceleration and framerate +updatePosition() Changes position depending on velocity and framerate +updateAcceleration(forces : List<Force>) Finds the acceleration for the entity depending on the resultant force of multiple Force objects +RecursiveMergeSort(Array : float[]) Sorts the array via merge sort +ValueIndex(Array : float[], value : float) : int Binary search to find the index for a value in an array

+findDistance(w : char) Finds the distance to one side of the form for an object (only used for circle object)

Polygon UML classes:

PolyForm : Form
-polyManager : PolyManager -context : BufferedGraphicsContext - buffer : BufferedGraphics -points : List<Point> -DrawnPoly : List<float[]> -AddDrawing : bool -newPolygon : NewPolygon +UniGravMag : float +Side : char
+PolyForm() Initializes the component and declares the polyManager, newPolygon and the graphics context and buffer #OnPaint(e : PaintEventArgs) Clears the graphics, runs Draw() and runs buffer.render to render before the images are put on the screen +Draw(e : PaintEventArgs) It runs the update functions for each polygon in polyManager and draws them via the graphics object depending on the polygons attributes. It also draws the list of points and a circle for each force object. -Quit_Click(sender : object, e : EventArgs) Closes the form -AddPolygon(sender : object, e : EventArgs) Uses the newPolygon form data to add either a new polygon or force, it could also use the positional data from drawn points on the form to decide where the polygon or force should go -Clear_Click(sender : object, e : EventArgs) Clears list of forces and objects -PolyForm_MouseClick(sender : object, e : EventArgs) Records the position of a mouse click when the simulation is paused +CentreDepVertices() : float[] Finds the centre of the drawn points to use as the position of an added polygon from drawing points

- Composition uses as newPolygon object is only made in polyform

PolyManager

-Timer : Timer

Timer for repeating methods every set interval of time

+Objects : List<Polygon>

List of Polygons to store all shapes in the environment

+forces : List<Force>

Store all force objects to iterate through them just like Objects

+Pen : Pen[]

Array of Pen types to be used in the graphics object to draw in the polyform

+form : PolyForm

To use the data from the polyform That called upon PolyManager

+framerate : float

To adjust framerate at which objects move, time between each frame of animation.

+side : char

+PolyManager (form : Polyform)

This sets the Pen array and the timer by running setTimer() as well as any extra objects I want to directly add here. It also linkes the PolyManager to PolyForm

-setTimer()

This assigns how much the timer is repeating itself and runs Timer.Start() to begin the running of code.

-Timer_Tick()

Runs Collision() and form.Invalidate() which gets rid of all graphics objects on the original polyform and runs its draw() method.

-Collision()

Checks for every object whether it is colliding and resolves the collision if any are by finding the normal to collision via EPA() and running that in resolveCollision()

+resolveCollision(Polygon A, Polygon B, float[] normal)

Uses normal as the collision to the normal between A and B and then resolves the collision realistically depending on the polygons' values and the collision normal. It also calls changeVelocity and ChangeAngular for each polygon.

+EPA(A : Polygon, B : Polygon, polytope : Polytope) : float[]

Uses the polytope's points to find the normal to collision between the two polygons

+SupportPoint(D : Float[], Polygon A) : float[]

Returns the furthest point on polygon A in the direction of vector D

+Support(A : Polygon, B : Polygon, D : float[]) : float[]

Uses SupportPoint() to find the difference between the maximum point on A and minimum in B in direction D, this is then used as a point that must be on the minkowski difference.

+GJK(A : Polygon, B : Polygon, simplex : Simplex) : Bool

By adding supports to simplex GJK finds whether the origin lies in the minkowski difference of the two polygons, therefore finding whether they are colliding. It also uses SimplexHandler for two different cases of the simplex.

+SimplexHandler(simplex : Simplex, inout D : float[]) : bool

Depending on whether simplex has 2 or 3 points it finds whether the origin is going to be in the simplex, will not be or if we still do not know. It will repeat in GJK() until it gets a final answer

+RecursiveCheck(vertexes : float[,], support : float[,], simplex : float[,]) : bool
Checks whether the support vector is found in the vertexes array while compensating for error of 0.001
+DotProduct(x : float[], y : float[]) : float
Finds the dot product between two vectors
+ VectorXtoY(x : float[], y : float[]) : float[]
Finds the vector between points x and y in direction x to y
+MultVector(x : float[], y : float[]) : float[]
Multiplies to vectors together
+Negative(x : float[]) : float[]
Returns the negative of vector x
+GetCoords(vertexes : float[,], rowIndex : int) : float[]
Return the vertex at rowIndex in the 2d array vertexes
+Normalize(vector : float[]) : float[]
Normalizes the vector

- Composition is shown in forces and objects list of objects only existing when poly manager exists
- Aggregation since poly manager uses poly form to draw graphics

Polygon : Entity
-vertexVectors : float[,] Stores vectors from position of polygon to its vertices -angularVelocity : float -angularAcceleration : float - elasticity
+Polygon(Vx : float, Vy : float, Ax : float, Ay: float, Px : float, Py : float, Colour : Pen, vertexVectors : float[,], angularVelocity : float, angularAcceleration : float, elasticity : float, mass : float) : base(Vx, Vy, Ax, Ay, Px, Py, Colour, mass) Sets the polygons attributes and runs setPosition() +setPosition() Finds the proper position that the position coordinates should be at based on the center of mass of the polygon +updatePosition() Runs entity base but also changes angular position and adds vertex vectors to position to find the drawn points locations. +updateVelocity() Runs entity base again but updates angular velocity based on angular acceleration +centre() : float[] Finds the centre of mass based on the position of the vertices +getMomentOfInertia() : float Returns the moment of inertia for the polygon based on its area and mass +getAngularVelocity() : float +getElasticity() : float +getMass() : float

+changeVelocity(newVelocity : float[]) Changes the velocity to the inputted value +changeAngular(newAngular : float) Changes the angular velocity to the inputted value

- Inherits from entity

NewPolygon : Form
+polyform : PolyForm +PolygonAdd : bool + GravityAdd: bool +velocity : float[] +acceleration : float[] +Position : float[] +AngularVelocity : float +mass : float +elasticity : float +Colour : Pen +VertexVectors : float[,] +GravMagnitude : float +GravLocation : float[] +UniGravMag : float +Values : List<TextBox>
+NewPolygon() initializes the component and links the form to the PolyForm class +finalCheck(Add : bool) Checks for whether the inputted values into the form are valid floats or valid chars by use of EnterValidCheck(), if they are not then a messagebox shows the error message, otherwise the bool relating to the thing being added is set to true so that thing gets added in the PolyForm. +EnterValidCheck(string String) : bool Tests whether an inputted value is a valid float (other methods are miscellaneous buttons and input takers) -ColourBox_SelectedIndexChanged(sender : object, e : EventArgs) -AddPolyButton_Click(sender : object, e : EventArgs) -AddForceButton_Click(sender : object, e : EventArgs) -MassBox_TextChanged(sender : object, e : EventArgs) -VelocityXBox_TextChanged(sender : object, e : EventArgs) -VelocityYBox_TextChanged(sender : object, e : EventArgs) -AccelerationXBox_TextChanged(sender : object, e : EventArgs) -AccelerationYBox_TextChanged(sender : object, e : EventArgs) -AngularVelBox_TextChanged(sender : object, e : EventArgs) -AngularAccelBox_TextChanged(sender : object, e : EventArgs) -ElasticityBox_TextChanged(sender : object, e : EventArgs) -RadialMagnitudeBox_TextChanged(sender : object, e : EventArgs) -PositionXBox_TextChanged(sender : object, e : EventArgs)

-PositionYBox_TextChanged(sender : object, e : EventArgs) -UniformMagBox_TextChanged(sender : object, e : EventArgs) -UniformDirectionBox_TextChanged(sender : object, e : EventArgs)

Circle UML classes:

CircleManager : Form
-timer : Timer - AddBall : Button -Clear : Button +Objects : List<CircleEntity> + forces : List<Force> +Pens : List<Pen>
+CircleManager() Initializes the component and graphics objects and sets the timer -setTimer() This assigns how much the timer repeats itself and start the timer to begin the program -Timer_Tick(sender : object, e : EventArgs) Runs collision() and invalidates the form to redraw the form, it also includes WallColl() for each object to make it collide with the wall separate to Collision() +draw(sender : object, e : EventArgs) Updates the objects' values and draws each object on the form as well as drawing the border lines -KeyPressEvent(sender : object, e : EventArgs) In the case that a key is pressed this runs the correct methods for each key -AddBall_Click(sender : object, e : EventArgs) Adds a new random ball to the list of circles +Collision() Checks for collisions between every ball. If a ball does collide it runs SetVelocityComponents to resolve the collision -WallColl(i : int) Finds whether ball with index of i hits a wall, and then resolves that collision -SetVelocityComponents(i : int, n : int, M1 : double) Reflects the gradient of the velocity vector of the circle around the normal of collision found in Collision() -OutBoundsCheck() Checks whether a ball is out of bounds

- Once again uses composition as it makes the forces and circle entity objects lists

CircleEntity : Entity
+Diameter : float -totalVelocity : float
+CircleEntity()

Assigns values +findVelocity() Returns total velocity

- Inherits from entity

CircleEditor
+NewGMass : float +GravPosition : float[] +CirclcePosition : float[] +CircleVelocity : float[] +CircleAccelertation : float[] +CircleDiameter : float + CircleMass : float +CircleColour : Pen +CircleAdd : bool +GravityAdd : bool +Values : List<TextBox>
+CircleEditor() Initializes component +finalCheck() +EnterValidCheck(String : string) : bool -GravMagnitude_TextChanged(sender : object, e : EventArgs) -GravLocationX_TextChanged(sender : object, e : EventArgs) -GravLOcationY_TextChanged(sender : object, e : EventArgs) -CircleDiameterBox_TextChanged(sender : object, e : EventArgs) -CircleVelXBox_TextChanged(sender : object, e : EventArgs) -CircleVelYBox_TextChanged(sender : object, e : EventArgs) -ColourBox_SelectedIndex(sender : object, e : EventArgs) -MassBox_TextChanged(sender : object, e : EventArgs)

Miscellaneous UML classes:

Border : Polygon
+Border(Px : float, Py : float, vertexVectors : float[,]) : base(0, 0, 0, 0, Px, Py, new Pen, vertexVectors, 0, 0, 0, float.PositiveInfinity) Runs setPosition() and assigns values

- Inherits from polygon to make it easier to add borders to the form

Collision Allows for a list of collisions to be made
+A : Polygon +B : Polygon +simplex : Simplex
+Collision(A : Polygon, B : Polygon, simplex : Simplex)

Assigns values

Force
+Mass : float +Position : float[] +Radial : bool
+Force(Magnitude : float, Position : float[], Radial : bool) Assigns values

Point Makes it easier to add a point image to the form
+position : float[] +width : float
+Point(coordinates : float[], width : float) Assigns values

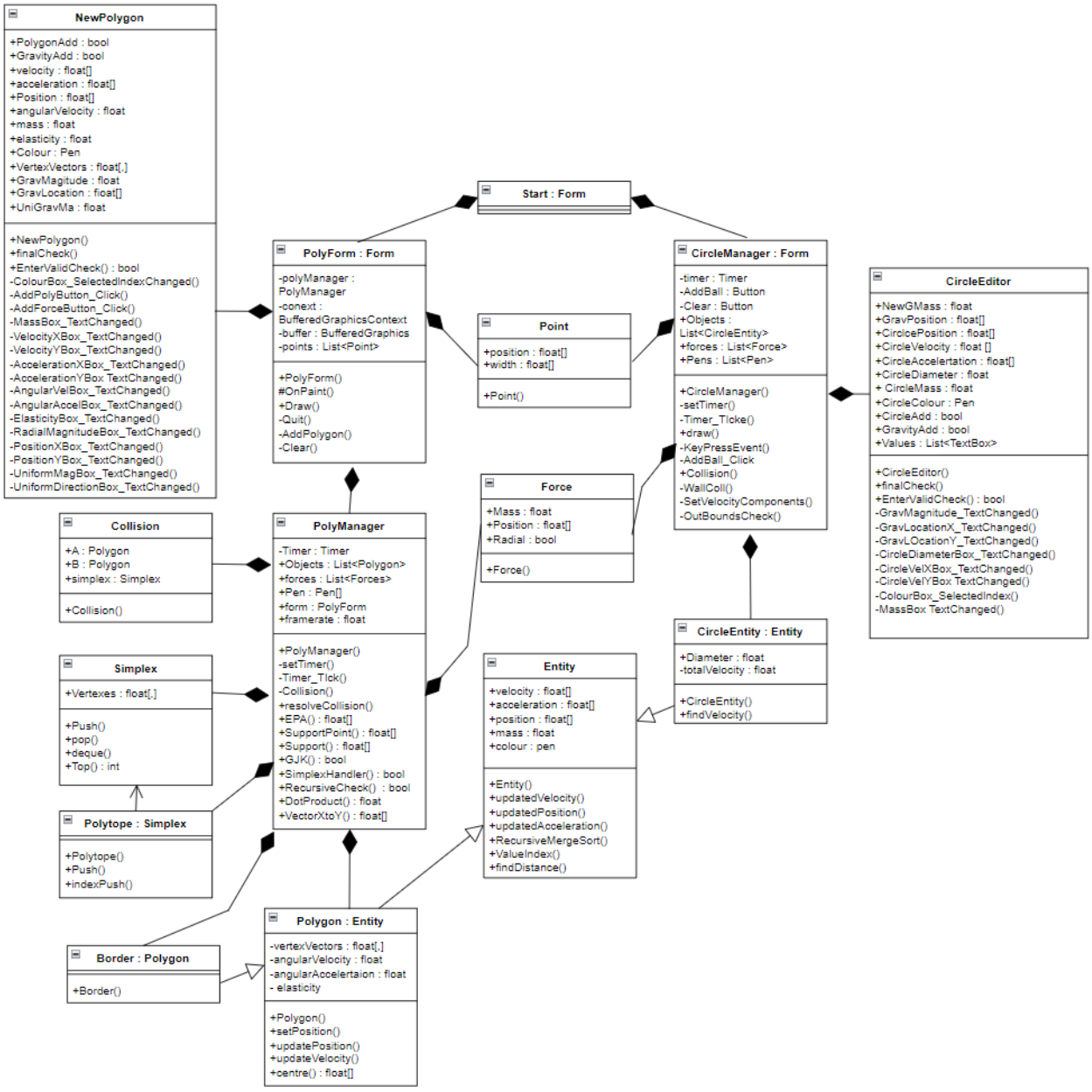
Polytope : Simplex A queue type
+Polytope() : base() Makes top = -1 +Push(Point : float[]) Adds a vector to the top of the polytope, removes back points if it needs to +indexPush(Point : float[], k : int) Adds a point to a specific place in polytope by moving other values around it

- Inherits most form simplex but changes one method for polyyope's larger size

Simplex A queue that removes the first item when another is added
+Vertexes : float[,] #top : int
+Push(Point float[]) Adds a point to the top and removes the first vector if it is a triangle +pop() Removes the top value +deque(k : int) Removes the value at the inputted index point +Top() : int Returns the index of the top value

Simplified UML diagram using classes above:

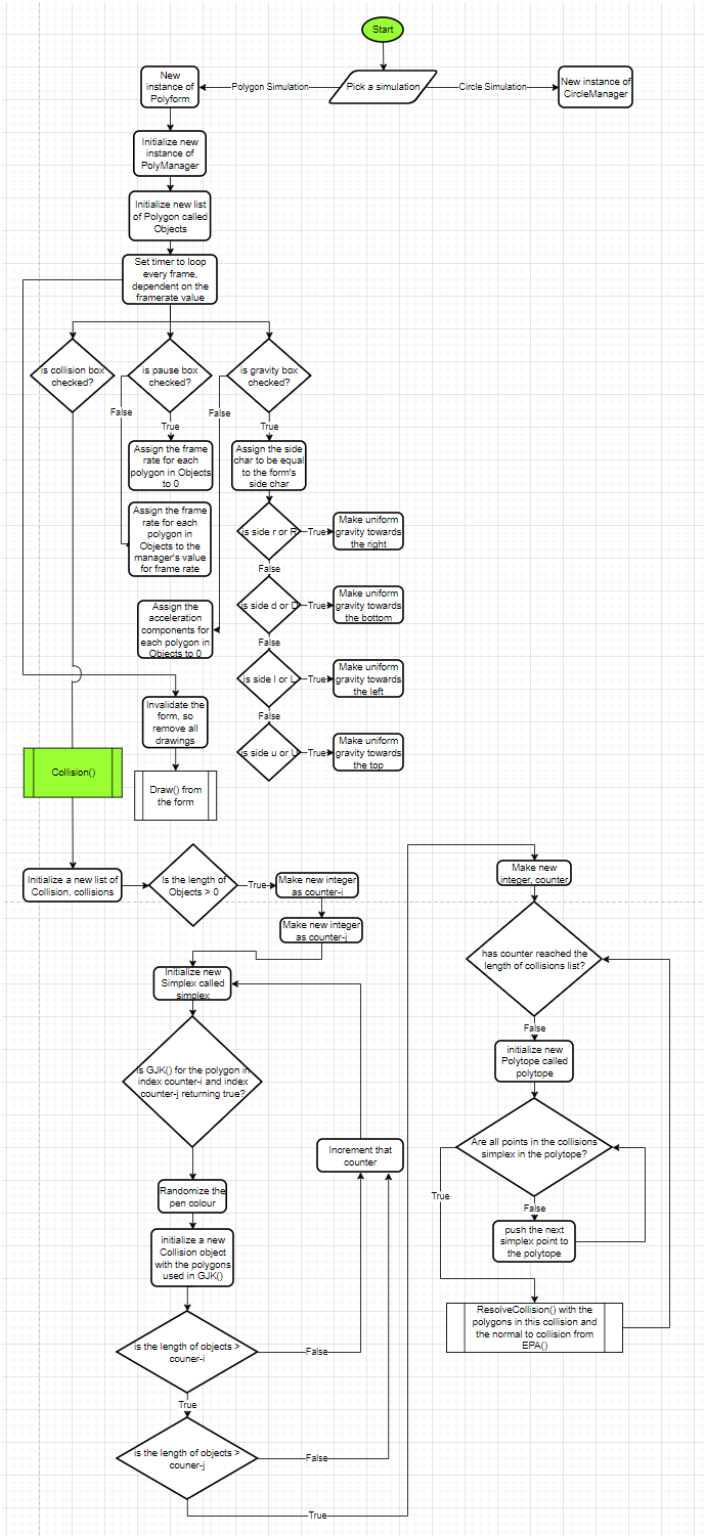
This is simplified as it does not include the inputted variables for each method

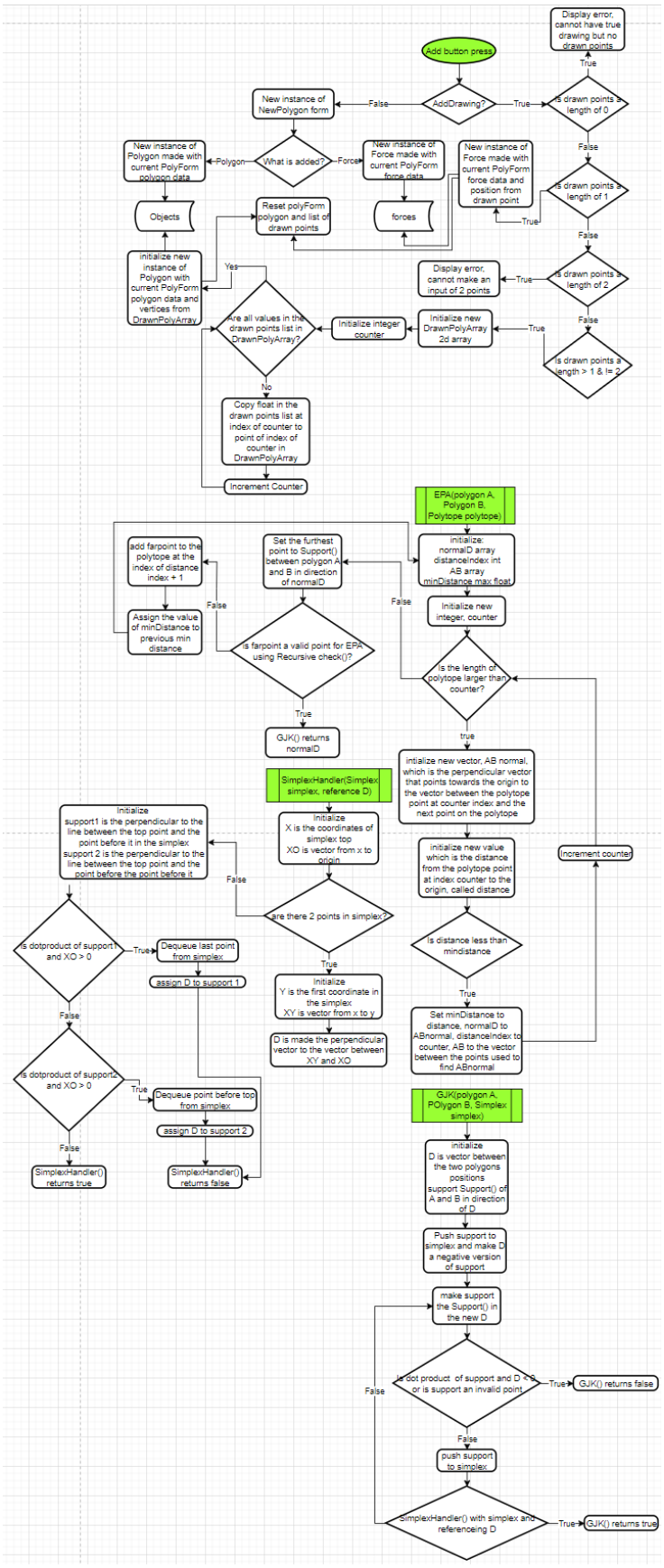


Flow chart for majority of program

When the validity of a point or vector is mentioned, it is in reference to the recursive check method. I have only included the logic heavy algorithms here as some methods in my program are virtually just calculations. The logic after the circle manager class is initialized is also not included since it is practically

just a much simpler version of the same algorithm.




Add button press
AddDrawing?
False
True
Display error, cannot have true drawing but no drawn points
Is drawn points a length of 0
True
False
New instance of NewPolygon form
What is added?
Polygon
Force
New instance of Polygon made with current PolyForm polygon data
New instance of Force made with current PolyForm force data
New instance of Force made with current PolyForm force data and position from drawn point
Is drawn points a length of 1
True
False
Objects
Reset polyForm polygon and list of drawn points
forces
initialize new instance of Polygon with current PolyForm polygon data and vertices from DrawnPolyArray
Yes
Are all values in the drawn points list in DrawnPolyArray?
Initialize integer counter
Initialize new DrawnPolyArray 2d array
Display error, cannot make an input of 2 points
Is drawn points a length of 2
True
False
Is drawn points a length > 1 & != 2
No
Copy float in the drawn points list at index of counter to point of index of counter in DrawnPolyArray
Increment Counter
EPA(polygon A, Polygon B, Polytope polytope)
initialize: normalD array distanceIndex int AB array minDistance max float
Initialize new integer, counter
Is the length of polytope larger than counter?
true
False
Set the furthest point to Support() between polygon A and B in direction of normalD
Is farpoint a valid point for EPA using Recursive check()?
False
True
add farpoint to the polytope at the index of distance index + 1
Assign the value of minDistance to previous min distance
GJK() returns normalD
intialize new vector, AB normal, which is the perpendicular vector that points towards the origin to the vector between the polytope point at counter index and the next point on the polytope
initialize new value which is the distance from the polytope point at index counter to the origin, called distance
Is distance less than mindistance
True
Set minDistance to distance, normalD to ABnormal, distanceIndex to counter, AB to the vector between the points used to find ABnormal
Increment counter
SimplexHandler(Simplex simplex, reference D)
Initialize X is the coordinates of simplex top XO is vector from x to origin
are there 2 points in simplex?
False
True
Initialize support1 is the perpendicular to the line between the top point and the point before it in the simplex support 2 is the perpendicular to the line between the top point and the point before the point before it
Initialize Y is the first coordinate in the simplex XY is vector from x to y
D is made the perpendicular vector to the vector between XY and XO
is dotproduct of support1 and XO > 0
True
Dequeue last point from simplex
assign D to support 1
False
is dotproduct of support2 and XO > 0
True
Dequeue point before top from simplex
assign D to support 2
False
SimplexHandler() returns true
SimplexHandler() returns false
GJK(polygon A, POlygon B, Simplex simplex)
initialize D is vector between the two polygons positions support Support() of A and B in direction of D
Push support to simplex and make D a negative version of support
make support the Support() in the new D
is dot product of support and D < 0 or is support an invalid point
True
GJK() returns false
False
push support to simplex
SimplexHandler() with simplex and referenceing D
True
GJK() returns true

Windows forms and set up of the program

This explains the logic behind the non-simulation related aspects such as using windows forms, graphics objects, timers etc.

Graphics:

Both simulations use graphics objects related to windows forms to produce an image I have used two different methods where I used basic PaintEventArgs for the prototype version (circle simulation) and a buffered graphics for the polygon one as that allows for the rendering to be done before it appears on screen to improve performance, the code for which was found from this Microsoft C# page.

Timer:

This is also not particularly complex, a timer is set up, so it elapses every 1000/60 milliseconds (though since the simulation can handle it, it is best to increase this as much as possible to prevent error). When it elapses it then starts the collision method and initializes the component to redraw the form.

A problem from using only one timer is that we need the timer to be in the manager class (the class where the main calculations take place) while the graphics object must be made in the form that it is drawn in. To solve this, I made the manager class its own form in my circle simulation, however this made it much more difficult to make buttons and caused a lot of errors due to any change to manager often causing the form itself to break. So in the polygon simulation I instead separated the manager and form it is drawn in by simply making the timer object in the manager, and after each tick of the timer it invalidates the form (this is done by relating the poly manager backwards to the poly form so that it can use form methods), since I now understand that this just removes drawings and re-runs the draw method in the form class, the draw() method in polyform then just takes the objects list from the poly manager and draws them onto the form. Doing this simplifies the layout of my code so much more, making it easier to add to and understand.

How movement occurs

For circles and polygons we use the same simple technique of splitting velocity and acceleration into X and Y components, when the timer ticks the velocity is changed by multiplying it by the acceleration which is multiplied by the time passed which is 1 divided by the frame rate (usually 60) and adding that to the current velocity. The position changes in the same way but just with velocity instead of acceleration.

Polygon vertex changes:

Polygon position values work differently to the circles as the position is a coordinate with vectors from the position to represent the polygon's vertices. But we want the position to be in the point of the polygons centre of mass at the polygon's initialization so we change the position to the new calculated centre of mass depending on the polygon's points, then find the new vectors to the vertices. There are different ways of finding this centre but I used this method - https://lexrent.eu/wp-content/uploads/torza/artikel_groep_sub_2_docs/BYZ_3_Polygon-Area-and-Centroid.pdf?utm_source.

```
FUNCTION Centre ARRAY
    Area <- 0
    Cx <- 0
    Cy <- 0
    NoVertices <- PointsList.Length
    FOR a <- 0 TO NoVertices-1
        Plus = a + 1
        IF Plus > NoVertices-1
            Plus = 0 // since we need a point and the next point for each calculation
        XVertex <- PointsList[a].X
        YVertex <- PointsList[a].Y
        XNextVertex <- PointsList[Plus].X
        YNextVertex <- PointsList{Plus].Y
        Equation <- XVertex * YNextVertex - XNextVertex * YVertex
        Area <- Area + Equation
        Cx <- Cx + (XVertex + XNextVertex) * Equation
        Cy <- Cx + (YVertex + YNextVertex) * Equation
    END FOR
    Area <- area/2
    Return Cx/(6*area), Cy/ (6*area)
```

This can work as I have two arrays, one for shapes vertices (vectors from the position) and one for the actual coordinates of each point which is re calculated each frame as the points move. This is done by just adding the vertex vectors to the position to find the vertex positions, the centre is found, and the position is readjusted and then the new vertex vectors from the new position is found by the reverse.

Angular velocity:

Angular velocity works by using these vectors from the position to each polygon's point to make it look like the polygons are rotating. In each frame the angular velocity is used to find a cos and sin velocity which is:

CosVelocity <- Cos(angularvelocity*framerate)

Where angular velocity is in radians per second. The vertex vectors themselves are then changed to use these angular changes, and after that we can change the actual point of each vertex depending on the polygon position and these vertex vectors.

```
FOR i <- 0 TO vertexVector.Length
        xVector <- vertexVectors[i].X
        vertexVector[i].X <- vertexVectors[i].X * CosVelocity - vertexVectors[i].Y * SinVelocity
        VertexVector[i].Y <- vertexVectors[i].X * SinVelocity + vertexVectors[i].Y * CosVelocity
END FOR
```

We can then also implement angular acceleration in the same way as our normal acceleration.

Force objects (gravity):

The simulation also includes force objects which are points on the form with a mass that causes a gravitational field. During each frame, the acceleration of every shape object is updated to compensate for the force from each force object using newtons force of gravity in a radial field equation, the resultant force is then found using f = ma. But to increase the range of the force (make it the same force but work for objects further away) we use a trick called softening which changes the radius that the force equation due to mass is divided by.

(Sqrt finds square root of number)

```
G <- 6.67e-6
finalForce <- 0, 0
softeningLength < - 100
FOR A <-0 TO forces.Count // this is to sum the force vectors from multiple force objects
        Distance <- force[A].position – polygon.position
        DistanceMagnitude <- Sqrt(Distance[0]^2 + Distance[1]^2)
        IF distanceMagnitude < 1
                distanceMganitude =  1 // used to stop division by 0
        END IF
        softenedDistance <- Sqrt(distanceMagnitude^2 + softeningLength^2)
        forceMagnitude <- (G * forces[A].Mass * Mass) / softenedDistance^3
```

forceDirection = Distance / distanceMagnitude

finalForce = finalForce + forceMagnitude * forceDirection //this is all to convert the force from the gravitational equation into a vector

END FOR

NewAcceleration = finalforce/polygon.mass

Acceleration = new acceleration

From that we can place as many force objects as we wish and the acceleration for every object will be updated depending on the distance, mass of the force, and mass of the object consistently allowing for orbits and attractive behaviours to occur. Similarly, there is a gravity algorithm in the polygon simulation for a uniform field rather than a radial one, this works in a similar way but simply iterates through each object to change their acceleration for the direction (up, down, left or right) of the uniform field.

Collision Detection

Source for maths and some of the logic: https://www.youtube.com/watch?v=MDusDn8oTSE

To detect collisions in the circle simulation is very simple as all we need to do is find the distance between the position coordinates of two arbitrary objects and sum their radii. We compare the distance and sum, if the distance is larger, no collision, if it is equal to or less than, then collision is true.

Doing this for two polygons is far more complex as each point on each polygon is not necessarily the same distance from its centre. Therefore, we are using a very different algorithm called the GJK (Gilbert-Johnson–Keerthi) algorithm. This works by finding the minkowski difference between two polygons (basically a set of points with a mass), making this difference into its own shape, if it includes/surrounds the origin (0, 0) then the two polygons must be overlapping/colliding. Just for clarification, the minkowski difference between two shapes is basically the shape which can be made by finding the difference between every point of one polygon and every point on another polygon

This following Image shows the basic concept, with A and B being the shapes colliding and A-B the difference between them, since it includes the origin, A and B must overlap and therefore are colliding.

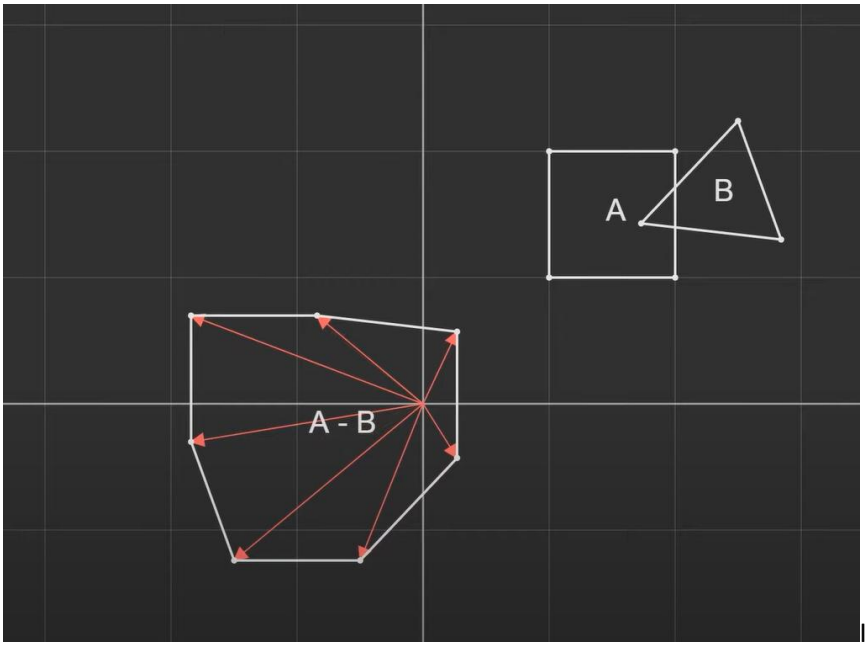


Image from: [the previously mentioned video]

Finding the minkowski difference

Since finding the difference between every point on one polygon with every point on the other polygon can become incredibly expensive for the computer (especially when using many objects) as this is $O(n^2)$ so we must use a different method that uses simplexes which are the simplest shapes possible in a dimension (line or triangle in 2D). How is this done? Well firstly, we do not need to use the difference between every point in one polygon and every point in the other polygon to find the minkowski difference, instead we just need to find the difference between the points furthest apart in the two polygons. But we don't even need to find all of the points on the difference, this is done by using a simplex that is inside of the minkowski difference, this means we just need just need the necessary number of points to find whether a simplex that is in the minkowski difference contains the origin, and therefore the difference, includes the origin.

To find the first simplex (a 2d array of coordinates) we pick an arbitrary direction vector (optimally the vector between the objects) and find the furthest point of one of the polygons in this direction, and the furthest point of the other polygon in the negative of the direction (this point is found by finding the dot product of the chosen direction and one polygon point, the polygon point with the highest dot product is the furthest in that direction), the difference between these two maximum points is found. We now have a point on the minkowski difference which we add to our simplex.

This pseudo code one of the furthest points for a polygon in a direction:
(Dotproduct() finds the dotproduct between the two vectors)

FurthestPoint(Direction, Polygon)

```
MaxDotProduct <- Negative infinity
MaxPoint <- 0, 0
FOR point IN PolygonPoints
        DotProduct <- Dotproduct(Point, Direction)
        IF Dotproduct > MaxDotProduct
                MaxPoint <- point
                MaxDotProduct <- DotProduct
        ENDIF
ENDFOR
RETURN MaxPoint
```

This therefore finds the furthest point in Polygon in Direction. This must then also be done for the other colliding polygon but with a reverse direction to then find the difference between the maximum points, giving us a new point we know to be on the minkowski difference and one we can use for the simplex. This would give us the same minkowski difference for much less calculations, and using the simplex allows us to use much less points to find the origin in reference to the minkowski difference. To increase the number of points on the simplex we change the direction by the perpendicular to the position vector of the latest point; or latest line if we already have 2 or more points.

Does the minkowski difference contain the origin?

Now we can do the proper GJK algorithm. We must now use the simplex (a section of the minkowski difference) to deduce whether the difference holds the origin by using recursion, check if the origin is in this current simplex, if not and no other possible simplex can then stop and return false collision, if it is still possible then continue until it is found to not be possible or it is found to be in the simplex.

When we get our first simplex point we can do the first test. If the point is on the same side as the direction vector we used to find it then we know that the minkowski difference cannot contain the origin due to the furthest point in the direction not even reaching the origin. This can easily be found via the dot product once again, a negative dot product with direction and our point means no collision:

(Support points are points found to be on the minkowski difference to be added to the simplex, RecursiveCheck() checks for the inputted point and whether it is inside the current simplex)

```
support <- Support(PolygonA, PolygonB, Direction)
IF DotProduct(support, Direction) <= 0 OR RecursiveCheck(support)
        Return false
```

So, if the dot product of the new point and direction is negative or the new point is a repeat of one of the previous points, we know there cannot be a collision so GJK returns false. We use this algorithm on every new point added to the simplex as it is the only ending condition if the minkowski difference does not contain the origin and it acts as a way to 'verify' each point.

After this we must run an algorithm dependent on the simplex length which is defined by its 'top' value as it acts similarly to a queue. First a negative version of the position vector of the new point is found. Then, if we added the second point to the simplex the GJK method enters the Line case when simplex.Top() == 1 (2 points). Though other than the previous point check, there isn't a way for us to get any more information from a line, so we just need to change the direction by finding the perpendicular to the vector between the negative top point and the vector between top and first point.

(Negative() finds the negative version of a vector, VectorXtoY() finds the vector between two points, PerpendicularVectorToOrigin() finds the perpendicular vector to between two points/position vectors that faces towards the origin)

X <- Top point

XO <- Negative(X)

IF simplex.Top IS 1

    Y <- Bottom vector

    XY <- VectorXtoY(X, Y)

    Direction <- PerpendicularVectorToOrigin(XY, XO)

The direction must face the origin because the next point on the minkowski difference furthest towards the origin has the highest chance of allowing the simplex to include the origin. However, if the simplex is made up of 3 points and is a triangle (the largest size the simplex can and should ever be) then we can actually make a more robust check for the origin's location in reference to the simplex and therefore the minkowski difference.

The triangle case must check different regions in relation to itself for any other minkowski difference points to add to itself, it does this by making two new vectors, one perpendicular to the top and the previous point, and then one with top and the other/starting point. By using the dot product of each vector and XO (the negative of top which would logically point towards the origin when thought of as a vector) we can see if the origin is outside of the triangle's sides or inside (negative dot product means origin is inside), so if it is not outside of them, we return true. If it is outside of one of them that just means we must search for more minkowski points in the direction of that vector since we know the origin is in that direction. Therefore, the direction is made into this vector and the point validation repeats, if no more points exist in that direction it means that it will repeat a point which was one of our ending conditions, so GJK returns false:

(PerpendicularVectorToOrigin() finds the perpendicular vector to between two points/position vectors that faces towards the origin, DotProduct() finds the dot product between two vectors, Deque() removes the point in the indexed position from the simplex)

```
PerpVector1 <- PerpendicularVectorToOrigin(top, top-1)

PerpVector2 <- PerpendicularVectorToOrigin(top, top-2)

IF DotProduct(PerpVector1, XO) > 0

        Simplex.Deque(top-2)

        D <- PerpVector1

ELSE IF DotProduct(PerpVector2, XO) > 0

        Simplex.Deque(top-1)

        D <- PerpVector2

ELSE

        Return true //therefore origin must be inside simplex
```

For if the triangle case causes another point to be found, the point that caused the vector which did not result in the direction of the origin is removed as it is in the wrong direction.

This together allows us to find whether the origin is found in the minkowski difference with a complexity of O(n) as we find the difference in points with the maximum values in a direction only and then only find the minimum amount of points to find the origin by making a simplex and using various techniques to find whether the origin is in that simplex to ultimately find whether the polygons collide. So the ultimate GJK algorithm looks a bit like this.

GJK pseudo code:

(push() adds a point to simplex)

```
FUNCTION GJK(PolygonA, PolygonB, Simplex)

Direction <- VectorXtoY(PositionA, PositionB)

support <- Support(PolygonA, PolygonB, Simplex)

Simplex.Push(support)

Direction <- Negative(support) //direction must be different to find the next point

While TRUE
```

```
    support <- Support(PolygonA, PolygonB, Direction)
    IF DotProduct(support, Direction) <= 0 OR RecursiveCheck(support)
        Return false
    END IF
    Simplex.Push(support) //add the recently validated point
    IF SimplexHandler(simplex, Direction)
        Return true //only return true once simplex handler returns true
    END IF
END FUNCTION
```

The reason that we can add a point at the start of GJK without running the validation if statement on it is because when the second point gets validated it would return false since it would be a repeat of the first point in the case that the origin is not in the minkowski difference.

```
FUNCTION SimplexHandler(simplex, Direction)
    X = simplex.TopVector
    XO = Negative(X)
IF simplex.Top == 1
    LineCase()
ELSE
    TriangleCase()
END IF
Return false
```

So in conclusion of GJK, the logic just loops through until something is returned (true or false), once the validity test returns true we return false, and then if the simplex handler returns true we only then know that the origin is in the simplex so GJK returns true. We now know if two polygons are colliding by just calling upon this function.

Resolving the collision

Now that we know two objects are colliding, we must change their velocities depending on the circumstances of collision. Once again, the circle implementation is far simpler so I will start with that.

Circles:

Since we already know the position coordinates of the circles, we can just subtract one from the other to find the vector between the two (the vector of collision). This is then used to reflect the vector of the circle's velocity around it to find the circles new velocity. This is achieved by using an equation I found on stack overflow: https://stackoverflow.com/questions/17395860/how-to-reflect-a-line-over-another-line.

VelocityGradient <- velocity/velocity

ReflectedVelocity <- Reflection(VelocityGradient, CollisionVector)

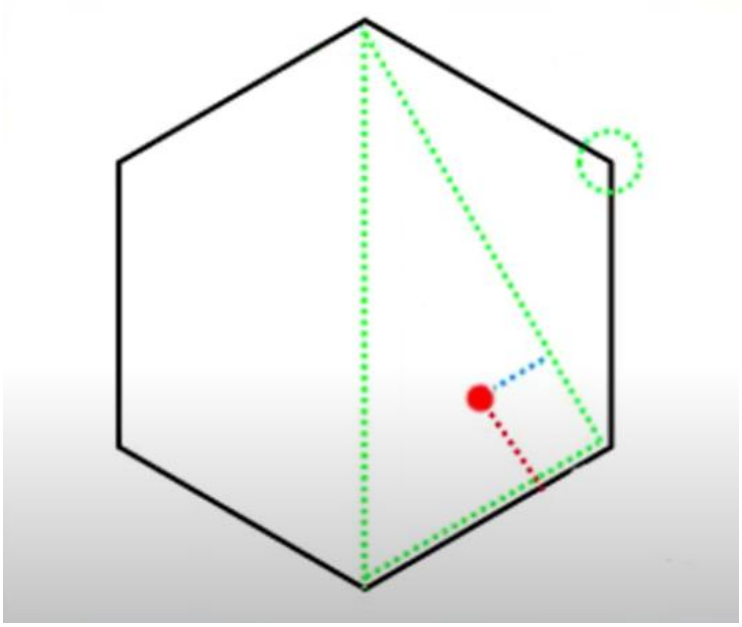
The Velocity magnitude is then used with the new gradient to find the new components. These components are then tested for being real values, if not then they are set to 0 to prevent error. Then finally the new components are set to the velocity values of the circle objects.

Polygons:

Here I have used another widely used algorithm to find the normal to the collision and then used the impulse found via this normal and the inertia of the polygons to resolve the collision. To find the normal I have used the EPA (expanding polytope algorithm) which is of course only ran on a collision if the collision is true (if GJK returns true). The algorithms logic I learnt from the brilliant explanation in this video: https://www.youtube.com/watch?v=6rgiPrzqt9w

EPA algorithm

EPA works by creating a similar datatype to the simplex called a polytope, this instead of acting like an array with a top, it acts more like a queue, it then expands to the minkowski difference to find the normal to collision. To do this we must find the closest edge of the difference to the origin. You may think that this means that we just use the simplex's closest edge to the origin, however the simplex did not search through every point and therefore edge of the difference so we must basically continue a slightly different version to our simplex finding algorithm to make sure we have the closest edge that is actually on the minkowski difference to the origin.



This image shows how the closest simplex (green triangle) edge to the origin (blue line points to it) is not actually the closest edge of the minkowski difference (black shape). In reality the edge in the blue line direction is quite far from the origin, and the true closest edge of the difference is pointed to by the red line. Therefore we must use EPA to find the true closest edge of the minkowski difference and not just the simplex's closest edge.

Image from https://www.youtube.com/watch?v=6rgiPrzqt9w

First we assign all the simplex values into our new data type of polytope, then we begin the function by iterating through the polytopes points (so up to its top index) while finding the dot product of each normal to these edges and the vector that points towards the origin, the one with the smallest dot product be the one that relates to the closest edge to the origin. Now we check for any further points on the minkowski difference in that direction (using the same support function from GJK), if there aren't anymore, then the previous normal is the normal to the closest edge so we use it as the collision normal. Though if we do find another point then we must iterate our algorithm again but this time with a new point added to the polytope. But adding this point is also slightly annoying as we must put it in a specific point in the polytope between the two points that made up the previously found edge so that the order of points is kept. Eventually our simplex, now polytope, will expand to reach the edge of the minkowski difference, and the normal to the edge closest to the origin is the one we use in our collision.

Making our polytope:

```
FOR A <- 0 TO simplex.Top()
        Polytope.push(simplex.point[A]
```

EPA pseudocode:

```
FUNCTION EPA(polytope, polygonA, polygonB)

normalDirection <- 0, 0

DistanceIndex <- 0
```

```
AB <- 0, 1
Iteration <- 0
previousMinDIstance <- Positive Infinity
MinDistance <- positive Inifity
WHILE true
FOR A <- 0 TO polytope.TopIndex-1
        ABnormal <- PerpendicularVectorToOrigin(Polytope.Points[A], Polytope.points[A+1]) // normal to current edge
        Distance <- DotProduct(polytope.Points[A], ABnormal) / Magnitude(ABnormal)
        IF Distance < minDistance
                minDistance <- Distance
                normalDirection <-Abnormal
                DustanceIndex <- A
                AB <- VectorXtoY(Polytope.Points[A], Polytope.points[A+1])
        END IF
END FOR
// we now have the closest edge of the polytope
FarPoint <- support(polygonA, polygonB, normalDirection)
IF NotValid(FarPoint)
        Return normalDirection
ELSE
        IF DistanceIndex+1 > polytope.TopIndex
                DistanceIndex <- 0
        END IF
        Polytope.Push(FarPoint, DistanceIndex+1) //puts the new point in where it should be
END IF
```

previousMInDistanc <- MinDistance

This iterates through points to find the closest polytope edge and then if it is on the minkowski difference, we then have our collision normal, otherwise we repeat the algorithm to find the actual closest edge.

Implementing EPA normal into impulse calculation

We now have the direction of our collision with EPA, so to resolve the collisions we must implement this equation from https://medium.com/@www.seymour/coding-a-2d-physics-engine-from-scratch-and-using-it-to-simulate-a-pendulum-clock-964b4ac2107a

$$j = \frac{-1(1+e)\,\bar{v}_{ab1} \cdot \bar{n}}{\frac{1}{m_a} + \frac{1}{m_b} + (\bar{r}_{ap} \times \bar{n})^2 / I_a + (\bar{r}_{bp} \times \bar{n})^2 / I_b}$$

This is pretty simple to implement though we do need to calculate the moment of inertia for both colliding objects (Ia and Ib) this is found in many ways but the one I used relates the mass and area of the object to find the inertia. I used the maths from this Khan Academy page combined with this stack exchange page to find make a method that finds it.

We find the moment of inertia by looping, incrementing the area each time by the stack exchange equation. Then the final area sum is divided by 2 and used with the mass value to find the inertia. To implement our impulse value, we must turn it into a vector by multiplying it by the normal. This impulse vector is then use as an x and y component for the two polygons while one of the polygons' velocity (and angular velocity) is changed by the negative of the vector and the other is changed via the positive of the vector (since impulse will be acting in opposite directions). This was done by using the following equations:

$$\bar{v}_{a2} = \bar{v}_{a1} + j\bar{n}/m_a$$

$$\bar{v}_{b2} = \bar{v}_{b1} - j\bar{n}/m_b$$

$$\omega_{a2} = \omega_{a1} + (\bar{r}_{ap} \times j\bar{n})/I_a$$

$$\omega_{b2} = \omega_{b1} - (\bar{r}_{bp} \times j\bar{n})/I_b$$

We now should have some shapes that will know if they are colliding, and then change their velocities accordingly. Now we can have a look at the overall logic in the collision() method which is only used in the polygon simulation for simplicity:

```
Rand <- new Random()

Collisions <- new List<Collision> // makes a list of collisions to resolve

IF Objects.Count NOT EQUAL TO 0
        // loops through every object pair
        FOR a <- 0 TO Objects.Count
                FOR b <- 0 TO Objects.Count
                        Simplex <- new Simplex()
                        IF GJK()
                                Objects[a].Colour <- rand.Pen// makes colour a new random one to show a collision has occurred
                                Collisions.Add(new Collision)
                        END IF
                END FOR
        END FOR
        // runs every collision in the list of collisions after they have all been found
        FOREACH collision in Collisions
                Polytope <- new Polytope()
                FOR a <- 0 TO collision.simplex.Top()
                        Polytope.Push(collision.simplex.vertexes[a]) // adds a each point in the simplex to the new polytope
                END FOR
        CollisionNormal <- EPA()
        ResolveCollision()
        END FOREACH
END IF
```

Adding objects

In the polygon simulation, when the add button is pressed in the polyform it does not just simply add a new random polygon (like what the circle simulation add button does), it first checks whether any drawn points have been added to the form.

Drawn points are added very easily by simply pausing the simulation and clicking the points on the form where the vertices will be. This is done by the mouse click event recording the location of the mouse and then adding those coordinates to a drawn points list; then on each timer tick it is checked whether the drawn points list > 0, if so then a bool called AddDrawing is set to true. This bool is what is used when the add button is pressed.

In the add button clicked event it is checked whether AddDrawing is true, if not then we open the new polygon form where we can change the default values for a polygon or force object or add a specific polygon or force object directly. Alternatively, if AddDrawing is true we find how many points have been added by the user. If only one then we make that point into the location of a new force object, the mass of which is the default or previously inputted value for force mass in the new polygon form. If there are two drawn points we show an error message. If more than two then we use those points as the vertices of a new polygon. This is done by iterating through the list of drawn points, moving them to an array of the same length and then simply initialize a new polygon object with a list of vertices that is that new array, and then adding that to poly manager's list of polygons (the mass and other values are those from the new polygon form). Finally, all the previously used lists and bools are reset for the next time the add button is pressed.

Inputting values

The new polygon form is a simple way for the user to input all the data they could want to (with the exception of shape and location of a polygon since they can just draw it), it works by having a variable for every force and polygon value needed which are all defined with default values (that are shown to the user in the textboxes for each variable). When one is changed it is checked if it is a valid float or char (depending on the variable) and if so then it changes the form's variable for that value. When Add polygon or Add force buttons are pressed a new instance of one of them is simply made with the values in the form, however to make these values save so that when we exit the form to draw a polygon with the inputted values we need them to be accessible constantly in the poly form not just the new polygon form. This is done by the initialization of the new polygon form being an attribute of the poly form. This means that whenever it is opened during the simulation it is the same new polygon form with the same stored values.

That may have been a little hard to follow but basically all the values are saved in the new polygon form, allowing for any kind of polygon to be drawn in the polyform form.

Technical solution

Links for sources to find code is included where that code is first used (or their respective explanation in documented design), so ones for graphics and timers are in circle manager since that was made first and is the first use of that learnt code.

Circle manager was initially my prototype but I have ended up including it with my final product (the polygon simulation) as it adds an extra option for the end client if they want to, at some point, simulate movements of curved objects rather than just one with straight sides. The reason some classes do not inherit from each other where you would expect them to (such as newPolygon and circle editor) is because the two simulations are separately built, the only exception to this being the entity class.

Some examples of coding skills:

Disclaimer: There are other examples of the skills shown here within the technical solution that have not been linked here. Some of the following linked algorithms are also found in the circle simulation, such as dynamically adding OOP objects and list operations.

Abstract data structures + stack/queue operations:

Pushing and dequeuing operations are found in these classes:
Simplex: Polygon simulation files -> Simplex.cs

Polytope: Polygon simulation files -> Polytope.cs

List operations:

Drawing objects: Polygon simulation files -> PolyForm.cs -> Draw()

Adding to polygons: Polygon simulation files -> PolyForm.cs -> AddPolygon_Click()

Finding resultant force: Main classes -> Entity.cs -> updateAcceleration()

Converting list to array: Polygon simulation files -> PolyForm.cs -> AddPolygon_Click()

Sort, search and recursion:

Merge sort: Polygon simulation files -> PolyManager.cs -> RecursiveMergeSort()

Binary search: Polygon simulation files -> PolyManager.cs -> ValueIndex()

Complex user defined algorithms:

Collision detection: Polygon simulation files -> PolyManager.cs -> GJK()

Dynamic generation of OOP objects:

Making new polygons and making new forces: Polygon simulation files -> PolyForm.cs -> AddPolygon_Click()

Code:

Using directives and the namespace have been removed from the code as namespace is the same for all files.

Main classes:

Classes that are not related to either simulation directly

Program.cs

```
1
1       internal static class Program
2       {
3           /// <summary>
4           /// The main entry point for the application.
5           /// </summary>
6           [STAThread]
7           static void Main()
8           {
9               Application.EnableVisualStyles();
10              Application.SetCompatibleTextRenderingDefault(false);
11              Application.Run(new Start());// start it :O
12          }
13      }
```

Start.cs

```
1
1   public partial class Start : Form
2       {
3           public Start()
4           {
5               InitializeComponent();//shows buttons and items on form
6           }
7
8           private void PolySim_Click(object sender, EventArgs e)
9           {
10              PolyForm form1 = new PolyForm();
11              form1.ShowDialog();//open an instance of the polygon simulation
12          }
13
14          private void CircleSim_Click(object sender, EventArgs e)
15          {
16              CircleManager form1 = new CircleManager();
17              form1.ShowDialog();//open an instance of the polygon simulation
18          }
19
20          private void Quit_Click(object sender, EventArgs e)
21          {
22              this.Close();//quits the simulation
23          }
```

```
    }
```

Entity.cs

```

internal class Entity// this is where the basic calculations for both circle
and polygon objects are kept and ran
    {
        public float[] velocity;
        public float[] acceleration;
        public float[] position; // position of top left corner (this is
changed in polygon to make it easier to calculate collisions and other things)
        public float mass;
        public Pen Colour;
        public float frameRate = 1f/60f;//this is not the framerate of the
simulation, just the one used in calculations which should not change (unless
pausing)
        public Entity(float vx, float vy, float ax, float ay, float px, float
py, Pen Colour, float mass)
        {
            velocity = new float[] { vx, vy };
            acceleration = new float[] { ax, ay };
            position = new float[] { px, py };
            this.Colour = Colour;
            this.mass = mass;// assign values
        }
        public virtual void updateVelocity()
        {
            velocity[0] = velocity[0] + acceleration[0] * frameRate;//Vx, the
time taken is 1 frame (1/fps)
            velocity[1] = velocity[1] + acceleration[1] * frameRate;//Vy
        }
        public virtual void updatePosition()
        {
            position[0] = (position[0] + velocity[0] * frameRate);// position
is incremented by the velocity in that direction
            position[1] = (position[1] + velocity[1] * frameRate); // position
must always be positive
        }
        public virtual void updateAcceleration(List<Force> forces)// used for
finding resultant acceleration from force objects
        {
            float G = 6.67f * (float)Math.Pow(10f, -6f); // gravitational
constant, to a lower power for ease of use in the simulation
            float[] finalForce = { 0, 0 };
            float softeningLength = 100;// this stretches the range of the
field, found this concept from:
https://astronomy.stackexchange.com/questions/50106/what-is-gravitational-
softening-length
            for (int i = 0; i < forces.Count; i++)//sums components of
different forces to find resultant force on this entity
```

```
            {
                // Calculate the distance vector between the particle and the
force object
                float[] Distance = { forces[i].Position[0] - position[0],
forces[i].Position[1] - position[1] };
                float distanceMagnitude = (float)Math.Sqrt(Distance[0] *
Distance[0] + Distance[1] * Distance[1]);

                if (distanceMagnitude < 1)
                    distanceMagnitude = 1;//prevents division by 0
                float softenedDistance = (float)Math.Sqrt(distanceMagnitude *
distanceMagnitude + softeningLength * softeningLength);//stretches the range

                float forceMagnitude = (G * forces[i].Mass * mass) /
((float)Math.Pow(softenedDistance, 3));
                float[] forceDirection = { Distance[0] / distanceMagnitude,
Distance[1] / distanceMagnitude };//puts gravitational force into vector,
https://www.khanacademy.org/computing/computer-programming/programming-natural-
simulations/programming-forces/a/gravitational-attraction
                finalForce[0] += forceMagnitude * forceDirection[0];
                finalForce[1] += forceMagnitude * forceDirection[1];
            }
            float[] NewAcceleration = { finalForce[0] / mass, finalForce[1] /
mass };// f = ma
            acceleration[0] = NewAcceleration[0];
            acceleration[1] = NewAcceleration[1];//assigns new accelerations
        }
    }
```

Force.cs

```
    internal class Force// force just stores data for updateAcceleration in
entity, and for it to be drawn in polyform
    {
        public float Mass;
        public float[] Position;
        public bool Radial;
        public Force(float Magnitude, float[] Position, bool Radial)
        {
            this.Mass = Magnitude;
            this.Position = Position;
            this.Radial = Radial;
        }
    }
```

Point.cs

```
    internal class Point//point is just used to store data that is then used to
draw the point in the polyform
    {
        public float[] position;
        public float width;
        public Point(float[] coordinates, float width)
        {
            position = coordinates;
            this.width = width;
        }
    }
```

Circle simulation files:

CircleManager.cs

```
    internal class CircleManager : Form//this is where the circle simulation is
displayed and ran
    {

        private static System.Windows.Forms.Timer timer;
        private Button AddBall;
        private Button Clear;
        public List<CircleEntity> Objects = new List<CircleEntity>(); // list
of entities
        public static List<Force> forces = new List<Force>();
        Pen black = new Pen(Color.Black, 4);// make a pen
        Pen blue = new Pen(Color.Blue, 4);
        Pen Red = new Pen(Color.Red, 4);
        Pen green = new Pen(Color.Green, 4);
        Pen yellow = new Pen(Color.Yellow, 4);
        Pen purple = new Pen(Color.Purple, 4);
        Pen turquoise = new Pen(Color.Turquoise, 4);
        Pen magenta = new Pen(Color.Magenta, 4);
        Pen pink = new Pen(Color.Pink, 4);
        Pen orange = new Pen(Color.Orange, 4);
        Pen brown = new Pen(Color.Brown, 4);
        Rectangle area = new Rectangle(0, 0, 1050, 703);
        private Button Quit;
        private Label label1;
        private CheckBox CollisionBox;
        private Button AddSpecifics;
        private CheckBox PausedBox;
        public List<Pen> Pens = new List<Pen>();


        public CircleManager()
```

```
        {
            InitializeComponent(); // initializes all windows forms elements
before velocities are calculated
            this.KeyPress += new KeyPressEventHandler(KeyPressEvent);// when a
key is pressed 'KeyPressEvent' is called
            this.DoubleBuffered = true;//makes it run timer tick before
rendering
            this.Size = new Size(1050, 850);//size of window
            this.Paint += new PaintEventHandler(draw);

            Pens.Add(black);
            Pens.Add(blue);
            Pens.Add(Red);
            Pens.Add(green);
            Pens.Add(yellow);
            Pens.Add(purple);
            Pens.Add(turquoise);
            Pens.Add(magenta);
            Pens.Add(pink);
            Pens.Add(orange);
            Pens.Add(brown);// all pen colours must be added to the list

            setTimer();

        }
        private void setTimer()// from
https://www.youtube.com/watch?v=GNyeojGBqmQ&t=322s and
https://learn.microsoft.com/en-us/dotnet/api/system.timers.timer?view=net-8.0
        {
            timer = new System.Windows.Forms.Timer();
            timer.Interval = (1000 / 120);// assigns the frame rate
            timer.Tick += Timer_Tick;//makes the timer tick method
            timer.Start();//starts
        }
        private void Timer_Tick(object sender, EventArgs e)
        {
            for (int i = 0; i < Objects.Count; i++)
            {
                WallColl(i);//checks for all collisions with walls seperately
            }
            if (CollisionBox.Checked)
            {
                Collision();//checks for all collisions between circles
            }
            if (PausedBox.Checked)// the framerates for objects is different to
the ones for the simulation as a whole
            {
                foreach (CircleEntity circle in Objects)
                {
```

```
                    circle.frameRate = 0;//makes all circles stop while
pausedBox is checked
                }
            }
            else if (!PausedBox.Checked)
            {
                foreach (CircleEntity circle in Objects)
                {
                    circle.frameRate = 1f / 60f;//when it isnt checked the
framerate gets reset
                }
            }
            this.Invalidate(area); // should redraw the form in the specified
area
        }
        public void draw(object sender, PaintEventArgs e)// code learnt from
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/advanced/how-to-
create-graphics-objects-for-drawing?view=netframeworkdesktop-4.8
        {//runs every time the form is invalidated in the timer
            Graphics g = e.Graphics;// obtaining graphics object
            Random rand = new Random();
            lock (Objects)//makes code run with thread error prevention
            {
                for (int i = 0; i < Objects.Count; i++)
                {
                    Objects[i].updateAcceleration(forces);
                    Objects[i].updateVelocity();
                    Objects[i].updatePosition();

                    if (Objects[i].position[0] + Objects[i].Diameter <=
this.Width && Objects[i].position[1] + Objects[i].Diameter <= this.Height)
                    {
                        e.Graphics.DrawEllipse(Objects[i].Colour,
Objects[i].position[0], Objects[i].position[1], Objects[i].Diameter,
Objects[i].Diameter);// pen, x of top left, y of top left, diameter across,
diameter up
                    }
                    // makes sure that the objects are within the form before
drawing them to prevent overflow error while drawing
                }

            }
            g.DrawLine(Red, 10, 10, 10, 700);
            g.DrawLine(black, 1025, 700, 10, 700);
            g.DrawLine(green, 1025, 700, 1025, 10);
            g.DrawLine(blue, 10, 10, 1025, 10);
        }
        private void KeyPressEvent(object sender, KeyPressEventArgs e)
        {
            if (e.KeyChar == (char)Keys.Space)
```

```
            {
                AddBall.PerformClick();// Trigger the button's Click event
            }
        }
        private void AddBall_Click(object sender, EventArgs e)// adds a ball
with random properties
        {
            Random rand = new Random();
            CircleEntity Ball = new CircleEntity(rand.Next(-300, 300),
rand.Next(-400, -100), 0, 400, 500, 500, 150, Pens[rand.Next(1, 10)], true, 4);
            lock (Objects)// lock prevents error by making this only occur in a
single thread https://learn.microsoft.com/en-us/dotnet/csharp/language-
reference/statements/lock
            {
                Objects.Add(Ball);
            }
        }
        public void Collision()
        {
            Random rand = new Random();
            lock (Objects)// previously sourced
            {
                for (int i = 0; i < Objects.Count; i++)// checks for every ball
                {
                    if (Objects.Count >= 2)
                    {
                        for (int n = 0; n < Objects.Count; n++)//detects
whether each ball collides with every other ball
                        {
                            // to find distance between Object i and Object n
                            float Distance =
(float)Math.Sqrt((Math.Pow(((Objects[i].position[0] + Objects[i].Diameter / 2)
- (Objects[n].position[0] + Objects[n].Diameter / 2)), 2) +
Math.Pow(((Objects[i].position[1] + Objects[i].Diameter / 2) -
(Objects[n].position[1] + Objects[n].Diameter / 2)), 2)));
                            if ((Distance <= (Objects[i].Diameter / 2 +
Objects[n].Diameter / 2)) && n != i)// compares distances and sum of radii to
determine whether a collision has occured
                            {
                                double M1 = -((Objects[i].position[1] -
Objects[n].position[1]) / (Objects[i].position[0] - Objects[n].position[0]));//
gradient of normal to ball's collision
                                Objects[i].Colour = Pens[rand.Next(1, 10)];//
randomizes colour to show that a collision has occured

                                SetVelocityComponents(i, n, M1);
                                SetVelocityComponents(n, i, M1);// finds the
new velocities for each ball and sets them
                            }
                        }
```

```
                    }
                }
            }
        }
        private void WallColl(int i)
        {
            float DX10 = Objects[i].position[0] + (Objects[i].Diameter / 2) -
10; // distance from left
            float DX1025 = 1025 - Objects[i].position[0] - (Objects[i].Diameter
/ 2); // distance right
            float DY10 = Objects[i].position[1] + (Objects[i].Diameter / 2) -
10; // distance from top
            float DY700 = 700 - Objects[i].position[1] - (Objects[i].Diameter /
2);// distance from bottom

//numbers used in finding distance for readability

            //double RD = Math.Sqrt(Math.Pow(Math.Sqrt(Math.Pow(DX10, 2) +
Math.Pow(DY10, 2)), 2) - Math.Pow(DY10, 2));// distance from Red line
            //double BKD = Math.Sqrt(Math.Pow(Math.Sqrt(Math.Pow(DX10, 2) +
Math.Pow(DY700, 2)), 2) - Math.Pow(DX10, 2));// distance from Black line
            //double GD = Math.Sqrt(Math.Pow(Math.Sqrt(Math.Pow(DX1025, 2) +
Math.Pow(DY700, 2)), 2) - Math.Pow(DY700, 2));// distance from green line
            //double BLD = Math.Sqrt(Math.Pow(Math.Sqrt(Math.Pow(DX10, 2) +
Math.Pow(DY10, 2)), 2) - Math.Pow(DX10, 2));// distance from blue line
            // the code ubove has been kept for further additions to the
simulation, however it is not needed for the simple simulation in circle
manager
            if (DX10 <= (Objects[i].Diameter / 2) || DX1025 <=
(Objects[i].Diameter / 2))// checks for the ball coming close to the lines on x
axis
            {
                Objects[i].velocity[0] = -Objects[i].velocity[0]; // flips
velocity in x direction

            }
            if (DY10 <= (Objects[i].Diameter / 2) || DY700 <=
(Objects[i].Diameter / 2))// checks for the ball coming close to the lines on y
axis
            {
                Objects[i].velocity[1] = -Objects[i].velocity[1]; // flips
velocity in y direction
            }
        }

        private void SetVelocityComponents(int i, int n, double M1)// finds and
sets new velocities after a collision
        {
            double M2 = Objects[i].velocity[1] / Objects[i].velocity[0];//
gradient of ball i's velocity
```

```
231
232             double M3 = ((2 * M1) + (M2 * Math.Pow(M1, 2)) - M2) / (2 * M1 * M2
233 - Math.Pow(M1, 2) + 1); // gradient of reflected M2 over M1
234             double Vo = Objects[i].findVelocity();
235             double newVx = Vo / (1 + M3);
236             double newVy = Math.Sqrt(Math.Pow(Vo, 2) - Math.Pow(newVx, 2));//
237 finds new velocity values, from
238 https://stackoverflow.com/questions/17395860/how-to-reflect-a-line-over-
239 another-line
240
241             Objects[i].velocity[0] = float.IsNaN((float)newVx) ? 0 :
242 (float)newVx;
243             Objects[i].velocity[1] = float.IsNaN((float)newVy) ? 0 :
244 (float)newVy;
245             //checks whether nexV is NaN, then makes it 0 or itself if it isnt,
246 from: https://how.dev/answers/how-to-check-if-a-value-is-not-a-number-nan-in-c-
247 sharp
248         }
249         private void OutBoundsCheck()
250         {
251             for (int i = 0; i < Objects.Count; i++)
252             {
253                 float center = Objects[i].position[0] - Objects[i].Diameter /
254 2;
255                 if (Objects[i].findDistance('T') < 0 ||
256 Objects[i].findDistance('B') < 0 || Objects[i].findDistance('L') < 0 ||
257 Objects[i].findDistance('R') < 0)
258                 {// if out of bounds:
259                     float M3 = 1;
260                     double Vo = Objects[i].findVelocity();
261                     double newVx = Vo / (1 + M3);
262                     double newVy = Math.Sqrt(Math.Pow(Vo, 2) - Math.Pow(newVx,
263 2));// flips velocities
264                 }
265             }
266
267
268         }
269
270
271
272         private void InitializeComponent()
273         {
274             this.AddBall = new System.Windows.Forms.Button();
275             this.Clear = new System.Windows.Forms.Button();
276             this.Quit = new System.Windows.Forms.Button();
277             this.label1 = new System.Windows.Forms.Label();
278             this.CollisionBox = new System.Windows.Forms.CheckBox();
279             this.AddSpecifics = new System.Windows.Forms.Button();
280             this.PausedBox = new System.Windows.Forms.CheckBox();
```

```
            this.SuspendLayout();
            //
            // AddBall
            //
            this.AddBall.Location = new System.Drawing.Point(469, 750);
            this.AddBall.Name = "AddBall";
            this.AddBall.Size = new System.Drawing.Size(75, 23);
            this.AddBall.TabIndex = 0;
            this.AddBall.Text = "Add Ball";
            this.AddBall.UseVisualStyleBackColor = true;
            this.AddBall.Click += new System.EventHandler(this.AddBall_Click);
            //
            // Clear
            //
            this.Clear.Location = new System.Drawing.Point(12, 747);
            this.Clear.Name = "Clear";
            this.Clear.Size = new System.Drawing.Size(75, 23);
            this.Clear.TabIndex = 2;
            this.Clear.Text = "Clear";
            this.Clear.UseVisualStyleBackColor = true;
            this.Clear.Click += new System.EventHandler(this.Clear_Click);
            //
            // Quit
            //
            this.Quit.BackColor =
System.Drawing.Color.FromArgb(((int)(((byte)(255)))), ((int)(((byte)(128)))),
((int)(((byte)(128)))));
            this.Quit.Location = new System.Drawing.Point(945, 750);
            this.Quit.Name = "Quit";
            this.Quit.Size = new System.Drawing.Size(75, 23);
            this.Quit.TabIndex = 3;
            this.Quit.Text = "Quit";
            this.Quit.UseVisualStyleBackColor = false;
            this.Quit.Click += new System.EventHandler(this.Quit_Click);
            //
            // label1
            //
            this.label1.AutoSize = true;
            this.label1.Location = new System.Drawing.Point(466, 731);
            this.label1.Name = "label1";
            this.label1.Size = new System.Drawing.Size(67, 13);
            this.label1.TabIndex = 4;
            this.label1.Text = "Press Space";
            //
            // CollisionBox
            //
            this.CollisionBox.AutoSize = true;
            this.CollisionBox.Checked = true;
            this.CollisionBox.CheckState =
System.Windows.Forms.CheckState.Checked;
```

```
            this.CollisionBox.Location = new System.Drawing.Point(637, 747);
            this.CollisionBox.Name = "CollisionBox";
            this.CollisionBox.Size = new System.Drawing.Size(64, 17);
            this.CollisionBox.TabIndex = 5;
            this.CollisionBox.Text = "Collision";
            this.CollisionBox.UseVisualStyleBackColor = true;
            //
            // AddSpecifics
            //
            this.AddSpecifics.BackColor =
System.Drawing.Color.FromArgb(((int)(((byte)(128)))), ((int)(((byte)(255)))),
((int)(((byte)(255)))));
            this.AddSpecifics.Location = new System.Drawing.Point(229, 745);
            this.AddSpecifics.Name = "AddSpecifics";
            this.AddSpecifics.Size = new System.Drawing.Size(128, 23);
            this.AddSpecifics.TabIndex = 6;
            this.AddSpecifics.Text = "Add specifics";
            this.AddSpecifics.UseVisualStyleBackColor = false;
            this.AddSpecifics.Click += new
System.EventHandler(this.AddSpecifics_Click);
            //
            // PausedBox
            //
            this.PausedBox.AutoSize = true;
            this.PausedBox.Location = new System.Drawing.Point(707, 747);
            this.PausedBox.Name = "PausedBox";
            this.PausedBox.Size = new System.Drawing.Size(62, 17);
            this.PausedBox.TabIndex = 7;
            this.PausedBox.Text = "Paused";
            this.PausedBox.UseVisualStyleBackColor = true;
            //
            // CircleManager
            //
            this.ClientSize = new System.Drawing.Size(1050, 850);
            this.Controls.Add(this.PausedBox);
            this.Controls.Add(this.AddSpecifics);
            this.Controls.Add(this.CollisionBox);
            this.Controls.Add(this.label1);
            this.Controls.Add(this.Quit);
            this.Controls.Add(this.Clear);
            this.Controls.Add(this.AddBall);
            this.Name = "CircleManager";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        private void Clear_Click(object sender, EventArgs e)
        {
            Objects.Clear();
```

```
            forces.Clear();// clears both lists
        }

        private void Quit_Click(object sender, EventArgs e)
        {
            this.Close();// quit button
        }

        private void AddSpecifics_Click(object sender, EventArgs e)
        {
            CircleEditor editor = new CircleEditor();
            editor.ShowDialog();
            if (editor.circleAdd)
            {
                Objects.Add(new CircleEntity(editor.CircleVelocity[0],
editor.CircleVelocity[1], editor.CircleAcceleration[0],
editor.CircleAcceleration[1], editor.CirclePosition[0],
editor.CirclePosition[1], editor.CircleDiameter, editor.CircleColour, true,
editor.CircleMass));
            }
            if (editor.GravityAdd)
            {
                forces.Add(new Force(editor.NewGMass, editor.GravPosition,
true));
            }
            //changes what is added depending on what button is pressed in
circle simulation
        }

    }
```

CircleEntity.cs

```
    internal class CircleEntity : Entity//where data for the circle objects is
kept
    {
        public float Diameter;
        private float totalVelocity;
        public bool Ball = false;
        public CircleEntity(float vx, float vy, float ax, float ay, float px,
float py, float Diameter, Pen Colour, bool ball, float mass) : base( vx, vy,
ax, ay, px, py, Colour, mass)
        {
            velocity = new float[] { vx, vy };
            acceleration = new float[] { ax, ay };
            position = new float[] { px, py };
            this.Diameter = Diameter;
            this.Colour = Colour;
            this.mass = mass;
```

```
        }
        public double findVelocity()
        {
            velocity[0] = velocity[0] + acceleration[0] * frameRate;//Vx, the
time taken is 1 frame (1/fps)
            velocity[1] = velocity[1] + acceleration[1] * frameRate;//Vy
            return totalVelocity = (float)Math.Sqrt(Math.Pow(velocity[0], 2) +
Math.Pow(velocity[1], 2));
        }
        public void RecursiveMergeSort(ref float[] Array)
        {
            int Len = Array.Length;
            int Mid = Len / 2;
            float[] rArray = new float[Mid];
            float[] lArray = new float[Len - Mid];
            Array.CopyTo(lArray, 0);
            Array.CopyTo(rArray, Mid);
            RecursiveMergeSort(ref lArray);//recurse
            RecursiveMergeSort(ref rArray);// ref is used to makesure the final
values are changed

        }
        public int ValueIndex(ref float[] Array, float Value)// recursive
bianry search
        {
            int Len = Array.Length;
            int Mid = Len / 2;//split array
            if (Array[Mid] == Value)
                return Mid;//end condition for finding value
            else if (Array[Mid] < Value)
            {
                float[] nextArray = new float[Mid];
                Array.CopyTo(nextArray, 0);
                return ValueIndex(ref nextArray, Value);//recurse
            }
            else if (Array[Mid] > Value)
            {
                float[] nextArray = new float[Len - Mid];
                Array.CopyTo(nextArray, Mid);
                return ValueIndex(ref nextArray, Value);//recurse
            }
            else
            {
                return 0;//not in list
            }
        }
        public float findDistance(char W)
        {
            if (W == 'L')
            {
```

```
                return position[0] + (Diameter / 2) - 10; // distance from left
            }
            else if (W == 'R')
            {
                return 1025 - position[0] - (Diameter / 2); // distance from
right
            }
            else if (W == 'T')
            {
                return position[1] + (Diameter / 2) - 10; // distance from top
            }
            else if (W == 'B')
            {
                return 700 - position[1] - (Diameter / 2);// distance from
bottom
            }
            else
            {
                return 0;
            }
        }
    }
```

CircleEditor.cs

```
    public partial class CircleEditor : Form//this is where the data for the
circle's add button is processed
    {
        public Pen black = new Pen(Color.Black, 4);
        public Pen blue = new Pen(Color.Blue, 4);
        public Pen Red = new Pen(Color.Red, 4);
        public Pen green = new Pen(Color.Green, 4);
        public Pen yellow = new Pen(Color.Yellow, 4);
        public Pen purple = new Pen(Color.Purple, 4);
        public Pen turquoise = new Pen(Color.Turquoise, 4);
        public Pen magenta = new Pen(Color.Magenta, 4);
        public Pen pink = new Pen(Color.Pink, 4);
        public Pen orange = new Pen(Color.Orange, 4);
        public Pen brown = new Pen(Color.Brown, 4);

        public float NewGMass = 50 * (float)Math.Pow(10, 10);// makes mass
large enough for force from force object to make an effect on the shapes
        public float[] GravPosition = { 500, 400 };
        public float[] CirclePosition = { 500, 400 };
        public float[] CircleVelocity = { -10, -100 };
        public float[] CircleAcceleration = { 0, 0};
        public float CircleDiameter = 100;
        public float CircleMass = 1;// ubove are the defaults
        public Pen CircleColour;
```

```
25
26          public bool circleAdd = false;
27          public bool GravityAdd = false;
28          public List<TextBox> Values = new List<TextBox>();// for error checking
29  on whether the values are appropriate
30
31          public CircleEditor()
32          {
33              CircleColour = black;
34              InitializeComponent();
35          }
36          private void finalCheck()// tests for text validity once values are
37  used and shows error message otherwise
38          {
39              bool x = false;
40              foreach (TextBox box in Values)
41              {
42                  if (!EnterValidCheck(box.Text))
43                  {
44                      x = true;
45                  }
46              }
47              if (x)
48              {
49                  MessageBox.Show("All values must be numbers");//error message
50              }
51              else if (!x)
52              {
53                  this.Close();
54              }
55          }
56          private void AddGravObj_Click(object sender, EventArgs e)
57          {
58              finalCheck();
59              GravityAdd = true;
60              //CircleManager.forces.Add(new Force(NewGMagnitude, GravPosition,
61  true));
62          }
63          private void AddCircle_Click(object sender, EventArgs e)
64          {
65              finalCheck();
66              circleAdd = true;
67          }
68          public bool EnterValidCheck(string String)// checks for validitiy
69          {
70              try
71              {
72                  Convert.ToInt32(String);
73              }
74              catch
```

```
            {
                return false;
            }
            return true;
        }
        //the following classes go through the same pattern of testing whether
the value is valid and then changing the circle editors value to that new
inputted value
        private void GravMagnitude_TextChanged(object sender, EventArgs e)
        {
            string GMagnitude = GravMagnitude.Text;
            if (EnterValidCheck(GMagnitude) == true)
            {
                NewGMass = Convert.ToInt32(GMagnitude) * (float)Math.Pow(10,
10);
            }
            Values.Add(GravMagnitude);
        }

        private void GravLocationX_TextChanged(object sender, EventArgs e)
        {
            string GLocationX = GravLocationX.Text;
            if (EnterValidCheck(GLocationX))
            {
                GravPosition[0] = Convert.ToInt32(GLocationX);
            }
            Values.Add(GravLocationX);
        }

        private void GravLocationY_TextChanged(object sender, EventArgs e)
        {
            string GLocationY = GravLocationY.Text;
            if (EnterValidCheck(GLocationY))
            {
                GravPosition[1] = Convert.ToInt32(GLocationY);
            }
            Values.Add(GravLocationY);
        }

        private void CircleDiameterBox_TextChanged(object sender, EventArgs e)
        {
            string String = CircleDiameterBox.Text;
            if (EnterValidCheck(String))
            {
                CircleDiameter = Convert.ToInt32(String);
            }
            Values.Add(CircleDiameterBox);
        }

        private void CircleVelXBox_TextChanged(object sender, EventArgs e)
```

```
125         {
126             string String = CircleVelXBox.Text;
127             if (EnterValidCheck(String))
128             {
129                 CircleVelocity[0] = Convert.ToInt32(String);
130             }
131             Values.Add(CircleVelXBox);
132         }
133 
134         private void CircleVelYBox_TextChanged(object sender, EventArgs e)
135         {
136             string String = CircleVelYBox.Text;
137             if (EnterValidCheck(String))
138             {
139                 CircleVelocity[1] = Convert.ToInt32(String);
140             }
141             Values.Add(CircleVelYBox);
142         }
143 
144         private void CirclePositionXBox_TextChanged(object sender, EventArgs e)
145         {
146             string String = CirclePositionXBox.Text;
147             if (EnterValidCheck(String))
148             {
149                 CirclePosition[0] = Convert.ToInt32(String);
150             }
151             Values.Add(CirclePositionXBox);
152         }
153 
154         private void CirclePositionYBox_TextChanged(object sender, EventArgs e)
155         {
156             string String = CirclePositionYBox.Text;
157             if (EnterValidCheck(String))
158             {
159                 CirclePosition[1] = Convert.ToInt32(String);
160             }
161             Values.Add(CirclePositionYBox);
162         }
163 
164         private void CircleAccelXBox_TextChanged(object sender, EventArgs e)
165         {
166             string String = CircleAccelXBox.Text;
167             if (EnterValidCheck(String))
168             {
169                 CircleAcceleration[1] = Convert.ToInt32(String);
170             }
171             Values.Add(CircleAccelXBox);
172         }
173 
174         private void CircleAccelYBox_TextChanged(object sender, EventArgs e)
```

```
175         {
176             string String = CircleAccelYBox.Text;
177             if (EnterValidCheck(String))
178             {
179                 CircleAcceleration[1] = Convert.ToInt32(String);
180             }
181             Values.Add(CircleAccelYBox);
182         }
183 
184         private void ColourBox_SelectedIndexChanged(object sender, EventArgs e)
185         {
186             string selected = ColourBox.SelectedItem.ToString();
187 
188             if (selected == "Black")
189             {
190                 CircleColour = black;
191             }
192             if (selected == "Blue")
193             {
194                 CircleColour = blue;
195             }
196             if (selected == "Red")
197             {
198                 CircleColour = Red;
199             }
200             if (selected == "Green")
201             {
202                 CircleColour = green;
203             }
204             if (selected == "Yellow")
205             {
206                 CircleColour = yellow;
207             }
208             if (selected == "Purple")
209             {
210                 CircleColour = purple;
211             }
212             if (selected == "Turquoise")
213             {
214                 CircleColour = turquoise;
215             }
216             if (selected == "Magenta")
217             {
218                 CircleColour = magenta;
219             }
220             if (selected == "Pink")
221             {
222                 CircleColour = pink;
223             }
224             if (selected == "Orange")
```

```
            {
                CircleColour = orange;
            }
            if (selected == "Brown")
            {
                CircleColour = brown;
            }
        }


        private void MassBox_TextChanged(object sender, EventArgs e)
        {
            string String = MassBox.Text;
            if (EnterValidCheck(String))
            {
                CircleMass = Convert.ToInt32(String);
            }
            Values.Add(MassBox);
        }
    }
```

Polygon simulation files:

PolyForm.cs

```
    public partial class PolyForm : Form//this is where the polygon simulation
is displayed
    {
        private PolyManager polyManager;
        private BufferedGraphicsContext context;// creates the graphics objects
https://learn.microsoft.com/en-
us/dotnet/api/system.drawing.bufferedgraphics?view=windowsdesktop-9.0
        private BufferedGraphics buffer;// stores the drawing
        private List<Point> points = new List<Point>();
        private List<float[]> DrawnPoly = new List<float[]>();
        private bool AddDrawing = false;
        private NewPolygon newPolygon;
        public float UniGravMag;
        public char Side = 'D';

        public PolyForm()
        {
            InitializeComponent();
            newPolygon = new NewPolygon(this);
            polyManager = new PolyManager(this);// both these ligns link the
poly manager and newpolygon form to this one
            context = BufferedGraphicsManager.Current;
            buffer = context.Allocate(this.CreateGraphics(),
this.ClientRectangle);// where and how the drawing is allocated
```

```
            this.DoubleBuffered = true;
            this.SetStyle(ControlStyles.AllPaintingInWmPaint |
ControlStyles.OptimizedDoubleBuffer, true);//this is for dubble buffering
(prevents flickering and improving simulation quality), found from
https://stackoverflow.com/questions/76993/how-to-double-buffer-net-controls-on-
a-form
        }
        protected override void OnPaint(PaintEventArgs e)// code from
https://learn.microsoft.com/en-
us/dotnet/api/system.drawing.bufferedgraphics?view=windowsdesktop-9.0
        {
            base.OnPaint(e);
            buffer.Graphics.Clear(Color.White);
            Draw(e);
            buffer.Render(e.Graphics);//double buffered graphics
        }
        public void Draw(PaintEventArgs e)
        {
            Graphics g = buffer.Graphics;
            for (int i = 0; i < polyManager.Objects.Count; i++)
            {
                if (polyManager.forces.Count > 0)
                {

polyManager.Objects[i].updateAcceleration(polyManager.forces);
                }
                polyManager.Objects[i].updateVelocity();
                polyManager.Objects[i].updatePosition();


                buffer.Graphics.DrawPolygon(polyManager.Objects[i].Colour,
polyManager.Objects[i].drawnPoints);
            }
            for (int i = 0; i < points.Count; i++)// points are used to show
where something is drawn
            {
                buffer.Graphics.DrawEllipse(polyManager.black,
points[i].position[0], points[i].position[1], points[i].width,
points[i].width);
            }
            for (int i = 0; i < polyManager.forces.Count; i++)//shows locatioin
of forces
            {
                buffer.Graphics.DrawEllipse(polyManager.blue,
polyManager.forces[i].Position[0], polyManager.forces[i].Position[1], 1, 1);
            }
            if (DrawnPoly.Count > 0)// if there is a point in it
            {
                AddDrawing = true;
            }
```

```
            else if (DrawnPoly.Count == 0)//this should not be needed but it
will prevent any possible errors relating to the drawnpoly list
                    AddDrawing = false;
            UniGravMag = newPolygon.UniGravMag;//sets to the inputted value
        }

        private void Quit_Click(object sender, EventArgs e)
        {
            this.Close();// quit
        }

        private void AddPolygon_Click(object sender, EventArgs e)
        {
            if (!AddDrawing)// runs normal code to add new object
            {
                newPolygon.ShowDialog();
                if (newPolygon.PolygonAdd)
                {
                    polyManager.Objects.Add(new Polygon(newPolygon.Velocity[0],
newPolygon.Velocity[1], newPolygon.Acceleration[0], newPolygon.Acceleration[1],
newPolygon.Position[0], newPolygon.Position[1], newPolygon.Colour,
newPolygon.VertexVectors, newPolygon.AngularVelocity,
newPolygon.AngularAcceleration, newPolygon.Elasticity, newPolygon.Mass));
                    // new force
                }
                if (newPolygon.GravityAdd)
                {
                    polyManager.forces.Add(new Force(newPolygon.GravMagnitude,
newPolygon.GravLocation, true));//new force
                }
                //buffer.Graphics.DrawEllipse(polyManager.black,
newPolygon.GravLocation[0], newPolygon.GravLocation[1], 30, 30);

            }
            else if (AddDrawing)// runs code for drawing an object
            {
                if (DrawnPoly.Count == 0)// this should not work as there
cannot be a drawing if there are no drawn points, so an error message is here
to show if it ever does happen
                {
                    MessageBox.Show("How did u do that");
                }
                else if (DrawnPoly.Count == 1)//if there is just one point, it
is made into a force
                {
                    polyManager.forces.Add(new Force(newPolygon.GravMagnitude,
new float[] { DrawnPoly[0][0], DrawnPoly[0][1] }, true));//puts force in first
point in points
                }
```

```
                else if (DrawnPoly.Count == 2)// cannot have a polygon with two
points with the current program
                {
                    MessageBox.Show("You have inputted 2 points at once, a new
object must have\n 1 point for a force\n 3 or more points for a polygon\n");
                    // tells the user what has to be inputted to draw
                }
                else if (DrawnPoly.Count > 1 && DrawnPoly.Count != 2)// drawn
poly has a reasonable amount of points
                {
                    float[,] DrawnPolyArray = new float[DrawnPoly.Count, 2];
                    for (int i = 0; i < DrawnPoly.Count; i++)// assigns the
list of drawn points to an array
                    {
                        DrawnPolyArray[i, 0] = DrawnPoly[i][0];
                        DrawnPolyArray[i, 1] = DrawnPoly[i][1];
                    }
                    polyManager.Objects.Add(new Polygon(newPolygon.Velocity[0],
newPolygon.Velocity[1], newPolygon.Acceleration[0], newPolygon.Acceleration[1],
CentreDepVertices()[0], CentreDepVertices()[1], newPolygon.Colour,
DrawnPolyArray, newPolygon.AngularVelocity, newPolygon.AngularAcceleration,
newPolygon.Elasticity, newPolygon.Mass));
                }
                DrawnPoly = new List<float[]>();// resets the drawn poly object
after use
                points = new List<Point>();// would remove all points
                AddDrawing = false;// reset bool
            }

        }

        private void Clear_Click(object sender, EventArgs e)
        {
            polyManager.forces.Clear();
            polyManager.Objects.Clear();
            points.Clear();
            DrawnPoly.Clear();// clears all lists
        }


        private void PolyForm_MouseClick(object sender, MouseEventArgs e)//
from https://stackoverflow.com/questions/6376186/c-sharp-detect-where-user-
clicked-on-a-image-bitmap
        // and from https://learn.microsoft.com/en-
us/dotnet/api/system.windows.forms.mouseeventargs?view=windowsdesktop-9.0
        {
            if (Paused.Checked)
            {
                float[] coords = new float[] { e.X, e.Y };
                DrawnPoly.Add(coords);
```

```
                points.Add(new Point(coords, 1));// records mouse point to add
a draw point
            }
        }
        public float[] CentreDepVertices()//finds centre based on the vertices
of the added polygon https://lexrent.eu/wp-
content/uploads/torza/artikel_groep_sub_2_docs/BYZ_3_Polygon-Area-and-
Centroid.pdf?utm_source
        {
            float area = 0;
            float Cx = 0;
            float Cy = 0;
            int NoVerticies = DrawnPoly.Count;//amount of vertices
            for (int i = 0; i < NoVerticies; i++)// sum equation
            {
                int plus = i + 1;
                if (i + 1 > NoVerticies - 1)
                    plus = 0;
                float XVertex = DrawnPoly[i][0];
                float YVertex = DrawnPoly[i][1];
                float NextXVertex = DrawnPoly[plus][0];
                float NextYVertex = DrawnPoly[plus][1];
                float thatPart = XVertex * NextYVertex - NextXVertex * YVertex;

                area = area + (thatPart);
                Cx = Cx + (XVertex + NextXVertex) * (thatPart);
                Cy = Cy + (YVertex + NextYVertex) * (thatPart);
            }

            area = 0.5f * area;
            return new float[] { Cx / (6 * area), Cy / (6 * area) };
        }

    }

```

PolyManager.cs

```
    internal class PolyManager//this is where the polygon simulation is ran
    {
        private static System.Windows.Forms.Timer Timer;
        public List<Polygon> Objects = new List<Polygon>();
        public List<Force> forces = new List<Force>();
        public Pen black = new Pen(Color.Black, 4);
        public Pen blue = new Pen(Color.Blue, 4);
        public Pen Red = new Pen(Color.Red, 4);
        public Pen green = new Pen(Color.Green, 4);
        public Pen yellow = new Pen(Color.Yellow, 4);
```

```
11          public Pen purple = new Pen(Color.Purple, 4);
12          public Pen turquoise = new Pen(Color.Turquoise, 4);
13          public Pen magenta = new Pen(Color.Magenta, 4);
14          public Pen pink = new Pen(Color.Pink, 4);
15          public Pen orange = new Pen(Color.Orange, 4);
16          public Pen brown = new Pen(Color.Brown, 4);
17          public Pen[] Pens;
18          public PolyForm form;
19          public float frameRate = 1f/60f;// frame rate for velocities, position
20  and acceleration changes
21          public char side;
22          public PolyManager(PolyForm form)// manager is only called once
23          {
24              this.form = form;// links polyform to this instance of poly manager
25              Pens = new Pen[] { black, blue, Red, green, yellow, purple,
26  turquoise, magenta, pink, orange, brown };//makes the pens list
27
28              //data for border object that are made to be on the edge of the
29  simulation
30              PointF Point1 = new PointF(10F, 10F);
31              PointF Point2 = new PointF(10F, 700F);
32              PointF Point3 = new PointF(1025F, 700F);
33              PointF Point4 = new PointF(1025F, 10F);
34              float[,] Border1 = new float[,] { { 200, 6 }, { 200, -6 }, { -544,
35  -6 }, { -544, 6 } };
36              float[,] Border2 = new float[,] { { 100, 10 }, { 100, -10 }, { -
37  100, -10 }, { -100, 10 } };
38              float[,] Border3 = new float[,] { { 100, 10 }, { 100, -10 }, { -
39  100, -10 }, { -100, 10 } };
40              float[,] Border4 = new float[,] { { 100, 10 }, { 100, -10 }, { -
41  100, -10 }, { -100, 10 } };
42              Border BorderA = new Border(556, 6, Border1);
43              Border BorderB = new Border(1106, 6, Border2);
44              Border BorderC = new Border(544, 344, Border3);
45              Border BorderD = new Border(544, 344, Border4);
46
47              //data for preset polygons
48              float[] Vertex1 = new float[] { 25, 25 };
49              float[] Vertex2 = new float[] { -25, 25};
50              float[] Vertex3 = new float[] { -25, -25};
51              float[] Vertex4 = new float[] { 25, -25};
52              float[,] verticies1 = new float[,] { { 25, 25 }, { -25, 25 }, { -
53  25, -25 }, { 25, -25 } };
54              float[,] verticies2 = new float[,] { { 25, 25 }, { -25, 25 }, { -
55  25, -25 } };
56              Polygon Shape1 = new Polygon(100, 100, 0, 0, 100, 100, black,
57  verticies1, 0, 0, 0.5f, 10);
58              Polygon Shape2 = new Polygon(100, -100, 0, 0, 100, 500, brown,
59  verticies2, 0, 0, 0.5f, 10);//these shapes are preset to the simulation
60              // the position is where the center of mass is positioned
```

```
61              Objects.Add(Shape1);
62              Objects.Add(Shape2);//adding the two preset polygons to Objects
63              setTimer();
64          }
65          private void setTimer()
66          {
67              Timer = new System.Windows.Forms.Timer();
68              Timer.Interval = 1000 / 1000;// frame rate for running the
69  simulation
70              Timer.Tick += Timer_Tick;//begins the tick method
71              Timer.Start();//starts the timer
72          }
73
74          private void Timer_Tick(object sender, EventArgs e)
75          {
76              if (form.CollisionCheck.Checked)
77              {
78                  Collision();//only run collisions when the collision box is
79  checked
80              }
81              if (form.Paused.Checked)
82              {
83                  foreach (Polygon shape in Objects)
84                  {
85                      shape.frameRate = 0;//set framerate for calculation updates
86  to 0 stops all motion
87                  }
88              }
89              else if (!form.Paused.Checked)
90              {
91                  foreach (Polygon shape in Objects)
92                  {
93                      shape.frameRate = frameRate;// makes objects move again
94                  }
95              }
96              if (form.Gravity.Checked)// for assigning direction to uniform
97  gravity
98              {
99                  side = form.Side;
100                 if (side == 'r' || side == 'R')
101                 {
102                     ChangeGravity(form.UniGravMag, 0);
103                 }
104                 if (side == 'd' || side == 'D')
105                 {
106                     ChangeGravity(0, form.UniGravMag);
107                 }
108                 if (side == 'l' || side == 'L')
109                 {
110                     ChangeGravity(-form.UniGravMag, 0);
```

```
111                 }
112                 if (side == 'u' || side == 'U')
113                 {
114                     ChangeGravity(0, -form.UniGravMag);
115                 }
116             }
117             else if (!form.Gravity.Checked)
118             {
119                 ChangeGravity(0, 0);// reset gravity to 0
120             }
121             form.Invalidate();
122         }
123         private void ChangeGravity(float X, float Y)// method to change
124 acceleration for all objects
125         {
126             for (int i = 0; i < Objects.Count; i++)
127             {
128                 Objects[i].acceleration[0] = X;
129                 Objects[i].acceleration[1] = Y;
130             }
131         }
132         private void Collision()//checks for collision and runs all neccessary
133 algorithms
134         {
135             Random rand = new Random();
136             List<Collision> collisions = new List<Collision>();//list of
137 collisions
138             if (Objects.Count > 0)//only runs if there are objects to calculate
139             {
140                 for (int i = 0; i < Objects.Count; i++)
141                 {
142                     for (int j = 0; j < Objects.Count; j++)
143                     {
144                         Simplex simplex = new Simplex();// simplex for this
145 particular check
146                         if (GJK(Objects[i], Objects[j], simplex))//there has
147 been a collision between the current objects if GJK returns true
148                         {
149                             Objects[i].Colour = Pens[rand.Next(1, 10)];//
150 randomizes colour to show collisions are occuring
151                             collisions.Add(new Collision(Objects[i],
152 Objects[j], simplex));//adds to list of collisions
153                         }
154                     }
155                 }
156                 foreach (Collision collision in collisions)
157                 {
158                     Polytope polytope = new Polytope();//stores more points
159                     for (int l = 0; l <= collision.simplex.Top();
160 l++)//iterates through each point in simplex
```

```
                    {
                        polytope.push(GetCoords(collision.simplex.vertexes,
l));//pushes all simplex points into polytope
                    }
                    float[] normalToCollision = EPA(collision.A, collision.B,
polytope);//collision normal
                    resolveCollision(collision.A, collision.B,
Normalize(normalToCollision));// finds new velocities and changes the
velocities to those depending on the results of the collisions

                }
            }
        }
        public void resolveCollision(Polygon A, Polygon B, float[] normal)//far
too long of an equation from, https://medium.com/@www.seymour/coding-a-2d-
physics-engine-from-scratch-and-using-it-to-simulate-a-pendulum-clock-
964b4ac2107a
        {
            if (DotProduct(normal, VectorXtoY(B.position, A.position)) <
0)//check that values are good for calculations
                normal = Negative(normal);

            //collision point is needed to find the relative velocity
            float[] AFarPoint = SupportPoint(normal, A);
            float[] BFarPoint = SupportPoint(Negative(normal), B);
            float[] collisionPoint = new float[] { (AFarPoint[0] +
BFarPoint[0]) / 2f, (AFarPoint[1] + BFarPoint[1]) / 2f };

            float[] CenterToCollisionA = VectorXtoY(A.position,
collisionPoint);
            float[] CenterToCollisionB = VectorXtoY(B.position,
collisionPoint);

            float[] AVelAngular = { A.velocity[0] + (-CenterToCollisionA[1] *
A.getAngularVelocity()), A.velocity[1] + (CenterToCollisionA[0] *
A.getAngularVelocity()) };
            float[] BVelAngular = { B.velocity[0] + (-CenterToCollisionB[1] *
B.getAngularVelocity()), B.velocity[1] + (CenterToCollisionB[0] *
B.getAngularVelocity()) };
            //finds angular velocities
            float[] relVel = { AVelAngular[0] - BVelAngular[0], AVelAngular[1]
- BVelAngular[1] };//needed for further calculations

            float velAlongNormal = DotProduct(relVel, normal);
            if (velAlongNormal > 0) return;// since there will be no impulse in
this case (though this should not happen)

            float elasticity = Math.Min(A.getElasticity(),
B.getElasticity());//elasticity must be minimum between two objects

```

```
            float ACTCMultNormal = CenterToCollisionA[0] * normal[1] -
CenterToCollisionA[1] * normal[0];
            float BCTCMultNormal = CenterToCollisionB[0] * normal[1] -
CenterToCollisionB[1] * normal[0];//values needed for impulse calculation

            float denominator = (1f / A.getMass()) + (1f / B.getMass()) +
(float)(Math.Pow(ACTCMultNormal, 2)) / A.getMomentOfInertia() +
(float)Math.Pow(BCTCMultNormal, 2) / B.getMomentOfInertia();

            float impulse = -(1 + elasticity) * velAlongNormal / denominator;
            float[] impulseVector = { impulse * normal[0], impulse * normal[1]
};//impulse as a vector

            //Update velocities and angular velocities
            A.changeVelocity(new float[] { A.velocity[0] + impulseVector[0] /
A.getMass(), A.velocity[1] + impulseVector[1] / A.getMass() });
            B.changeVelocity(new float[] { B.velocity[0] - impulseVector[0] /
B.getMass(), B.velocity[1] - impulseVector[1] / B.getMass() });
            A.changeAngular(A.getAngularVelocity() + (CenterToCollisionA[0] *
impulseVector[1] - CenterToCollisionA[1] * impulseVector[0]) /
A.getMomentOfInertia());
            B.changeAngular(B.getAngularVelocity() - (CenterToCollisionB[0] *
impulseVector[1] - CenterToCollisionB[1] * impulseVector[0]) /
B.getMomentOfInertia());

            //the following seperates objects if they have ended up inside
eachother, though this is not as rigorous as CCD (explained in evaluation)
            float penetration = 0.04f; //limit for penetration, 0,04 is chosen
because it seems to work best (rather than 0.03 or 0.05)
            float percent = 0.2f;// code is found from
https://code.tutsplus.com/how-to-create-a-custom-2d-physics-engine-the-basics-
and-impulse-resolution--gamedev-6331t
            float correction = Math.Max(penetration, 0f) / (1 / A.getMass() + 1
/ B.getMass()) * percent;
            float[] correctionVec = { correction * normal[0], correction *
normal[1] };// penetration correction algorithm

            A.position[0] += correctionVec[0] / A.getMass();
            A.position[1] += correctionVec[1] / A.getMass();
            B.position[0] -= correctionVec[0] / B.getMass();
            B.position[1] -= correctionVec[1] / B.getMass();//adjusts positions
of polygons depending on their mass (inertia) and the correction values
        }

        public float[] EPA(Polygon A, Polygon B, Polytope polytope)// returns
the normal to collision
        //https://www.youtube.com/watch?v=6rgiPrzqt9w,
https://www.youtube.com/watch?v=0XQ2FSz3EK8, again shows the algorithm but not
code
        {
```

```
;
            float[] normalD = new float[2];// direction of collision
            int distanceIndex = 0;//point in list of edge which is the furthest
from the origin
            float[] AB = new float[2];//vector between two points
            float previousMinDistance = float.MaxValue;// need to store the
previous smallest distance
            while (true)
            {
                float minDistance = float.MaxValue;//minimum distance set to
max so that we can find the smaller and smaller version of it,
https://stackoverflow.com/questions/11857797/maximum-value-for-type-float-in-c-
sharp
                for (int i = 0; i < polytope.Top(); i++)//finds the side of
polytope closest to the origin
                {
                    // to find the distance to a point using dot products:
https://brilliant.org/wiki/dot-product-distance-between-point-and-a-
line/#:~:text=The%20distance%20formula%20can%20be,%E2%88%A3%20a%202%20%2B%20b%2
02%20.
                    float[] ABNormal =
PerpendicularVectorToOrigin(GetCoords(polytope.vertexes, i),
GetCoords(polytope.vertexes, i + 1));// finds the normal to the line between
two points
                    float Distance = DotProduct(GetCoords(polytope.vertexes,
i), ABNormal) / (float)Math.Sqrt(Math.Pow(ABNormal[0], 2) +
Math.Pow(ABNormal[1], 2));// distance to origin
                    if (Distance < minDistance)//finds closest edge to origin
                    {
                        minDistance = Distance;// set smallest distance
                        normalD = ABNormal;// set new direction
                        distanceIndex = i;// save the point in vertexes the
point should be saved
                        AB = VectorXtoY(GetCoords(polytope.vertexes, i),
GetCoords(polytope.vertexes, i + 1));// saves the newest closest edge
                    }
                }
                float[] FarPoint = Support(A, B, normalD);// the minkowski
difference point furthest in the direction of the normal to the line closest to
the origin
                if (RecursiveCheck(polytope.vertexes, A, B, normalD, FarPoint,
polytope) == true || Math.Abs(previousMinDistance - minDistance) <
0.0001f)//point is new
                {
                    return normalD;// this is the end condition for finding the
final normal to the collision
                }
                else// then the closest edge is
                {
                    if (distanceIndex + 1 > polytope.Top())
```

```
311                     {
312                         distanceIndex = 0;
313                     }
314                     polytope.indexPush(FarPoint, distanceIndex + 1);
315                 }
316                 previousMinDistance = minDistance;
317             }
318         }
319 
320         public float[] SupportPoint(float[] D, Polygon A)//returns the maximum
321 point of a polygon in direction D
322         {
323             float[] coords;
324             float MaxDP = float.MinValue;//must be minimum since the largest
325 dot product could be negative
326             float[] MaxPoint = new float[2];
327             foreach (PointF point in A.drawnPoints)// searches through each
328 point of A
329             {
330                 coords = new float[] { point.X, point.Y };
331                 float DP = DotProduct(coords, D);
332                 if (DP > MaxDP)//finds the polygon's point with the maximum dot
333 product as that will be the point furthest in direction D
334                 {
335                     MaxDP = DP;
336                     MaxPoint = coords;
337                 }
338             }
339             return MaxPoint;
340         }
341         private float[] Support(Polygon A, Polygon B, float[] D)// one point on
342 minkowski difference for direction D (difference of two maximum points)
343         {
344             float[] NegativeD = new float[] { -D[0], -D[1] };
345             float[] AmaxPoint = SupportPoint(D, A);
346             float[] BmaxPoint = SupportPoint(NegativeD, B);
347             return VectorXtoY(BmaxPoint, AmaxPoint);// max point/vector of A in
348 D - max point/vector of B in -D
349 
350         }
351         public bool GJK(Polygon A, Polygon B, Simplex simplex)// returns
352 whether A and B colide
353         //https://www.youtube.com/watch?v=MDusDn8oTSE gives part of the general
354 algorithm, but not C# code
355         {
356             float[] D = VectorXtoY(A.position, B.position); ;// should be
357 vector between centers of the polygons
358             float[] support = Support(A, B, D);// current vector/point for
359 furthest point on minkowski difference
360 
```

```
            simplex.push(support);// add initial point to simplex stack
            D = Negative(support);// makes D the vector from support to the
origin
            while (true)// loops until origin is found to be in or oustide of
minkowski difference
            {
                support = Support(A, B, D);
                if (DotProduct(support, D) <= 0 ||
RecursiveCheck(simplex.vertexes, A, B, D, support, simplex))//check if point is
valid
                {
                    return false;// no collision since furthest point is before
the origin
                }
                simplex.push(support);// if point is valid it is added to
simplex

                if (SimplexHandler(simplex, ref D))//the algorithm is therefore
ran
                    // if simplexhandler returns false then the while loop just
repeats
                {
                    return true;// if origin is found to be in simplex
                }
            }
        }
        public bool SimplexHandler(Simplex simplex, ref float[] D)// runs if
collision is possible
        {
            float[] X = new float[] { simplex.vertexes[simplex.Top(), 0],
simplex.vertexes[simplex.Top(), 1] };// make Top its own point
            float[] XO = Negative(X); // X towards the origin
            if (simplex.Top() == 1)// when the simplex has two points, so is a
line
            {
                float[] Y = new float[] { simplex.vertexes[0, 0],
simplex.vertexes[0, 1] };// since it is a line, the only other point other than
top is at array[0]
                float[] XY = VectorXtoY(X, Y);
                D = PerpendicularVectorToOrigin(XY, XO);// perpendicular vector
to the simplex line of the two most recent points from
https://www.youtube.com/watch?v=ajv46BSqcK4&t=60s
                //we just have a line so cannot know if origin will be in the
simplex
            }
            else//triangle case
            {
                //perpendicular vectors to the lines between the current
support and the two other points on the triangle \/
```

```
                float[] support1 =
PerpendicularVectorToOrigin(GetCoords(simplex.vertexes, simplex.Top()),
GetCoords(simplex.vertexes, simplex.Top() - 1));//perp of support and the
previous point
                float[] support2 =
PerpendicularVectorToOrigin(GetCoords(simplex.vertexes, simplex.Top()),
GetCoords(simplex.vertexes, simplex.Top() - 2));//perp of support and two
points ago

                if (DotProduct(support1, XO) > 0)//if origin is perpendicular
to one side of simplex
                {
                    simplex.deque(0);
                    D = support1;//should find perpendicular vector between top
point and 2nd top point in simplex
                }
                else if (DotProduct(support2, XO) > 0)
                {
                    simplex.deque(1);
                    D = support2;
                }
                else
                    return true; //since at this point the origin cannot be
outside either side of the triangle
            }
            return false;
        }
        // below are the comonly used methods for vector manipulation
        public bool RecursiveCheck(float[,] vertexes, Polygon A, Polygon B,
float[] D, float[] support, Simplex simplex)//checks whether a new support
point is already on the stack
        {
            bool x = false;// it is assumed that the new support is not already
in the simplex
            for (int i = 0; i < simplex.Top(); i++)//getlength is used to find
the amount of points in vertexes https://www.programiz.com/csharp-
programming/multidimensional-
arrays#:~:text=GetLength(0)%20%2D%20gives%20the,of%20elements%20in%20the%20row
            {
                if (Math.Abs(support[0] - GetCoords(vertexes, i)[0]) < 0.0001f
&& Math.Abs(support[1] - GetCoords(vertexes, i)[1]) < 0.0001f)// compares
support to every simplex point to sheck for repeats
                //0.0001f is used to account for floating point error, i found
this technique on https://forums.ni.com/t5/LabVIEW/How-do-you-guys-deal-with-
floating-point-errors/td-p/4208871
                {
                    x = true;
                }
            }
            return x;
```

```
460         }
461         public float DotProduct(float[] x, float[] y)// finds dotproduct of two
462 vectors/points
463         {
464             return x[0] * y[0] + x[1] * y[1];
465         }
466         public float[] VectorXtoY(float[] x, float[] y)
467         {
468             return new float[] {y[0] - x[0], y[1] - x[1] };
469         }
470         public float[] MultVector(float[] x, float[] y)//wrong
471         {
472             return new float[] { x[0] * y[0], x[1] * y[1] };
473         }
474         public float[] PerpendicularVectorToOrigin(float[] x, float[]
475 y)//perpendicular vector pointing towards the origin to the vector between
476 points x and y
477         {
478             float dot = x[0] * y[1] - x[1] * y[0];
479             return new float[] { -dot * x[1], dot * x[0] };
480         }
481         public float[] Negative(float[]x)
482         {
483             return new float[] { -x[0], -x[1] };
484         }
485         public float[] GetCoords(float[,] vertexes, int rowIndex)
486         {
487             float[] Coords = new float[2];
488             Coords[0] = vertexes[rowIndex, 0];
489             Coords[1] = vertexes[rowIndex, 1];
490             return Coords;
491         }
492         public float[] Normalize(float[] vector)
493         {
494             float magnitude = (float)Math.Sqrt(Math.Pow(vector[0], 2) +
495 Math.Pow(vector[1], 2));
496             vector[0] /= magnitude;
497             vector[1] /= magnitude;
498             return vector;
499         }
500         public void RecursiveMergeSort(ref float[] array1d)// sorts a 1d array
501         {
502             int Len = array1d.Length;
503             int Mid = Len / 2;
504             float[] rArray = new float[Mid];
505             float[] lArray = new float[Len - Mid];
506             array1d.CopyTo(lArray, 0);
507             array1d.CopyTo(rArray, Mid);
508             RecursiveMergeSort(ref lArray);
```

```
509             RecursiveMergeSort(ref rArray);// ref is used to makesure the final
510 values are changed
511 
512         }
513         public float[] Array2dTO1d(float[,] array2d)//converts a 2d array of
514 coordinates to a 1d array where each value is a 2 digit number
515         {
516             float[] arrayCollumnOne = new float[array2d.GetLength(0)];//array
517 length of 2d array
518             float[] arrayCollumnTwo = new float[array2d.GetLength(0)];//array
519 width of 2d array
520             float[] array1d = new float[array2d.GetLength(0)];
521             for (int i = 0; i < array2d.GetLength(0); i++)
522             {
523                 arrayCollumnOne[i] = array2d[i, 0];
524                 arrayCollumnTwo[i] = array2d[i, 1];
525                 array1d[i] = arrayCollumnOne[i] * 10 + arrayCollumnTwo[i];
526             }
527             return array1d;
528         }
529         public int ValueIndex(ref float[] Array, float Value)// recursive
530 bianry search
531         {
532             int Len = Array.Length;
533             int Mid = Len / 2;
534             if (Array[Mid] == Value)
535                 return Mid;
536             else if (Array[Mid] < Value)
537             {
538                 float[] nextArray = new float[Mid];
539                 Array.CopyTo(nextArray, 0);
540                 return ValueIndex(ref nextArray, Value);
541             }
542             else if (Array[Mid] > Value)
543             {
544                 float[] nextArray = new float[Len - Mid];
545                 Array.CopyTo(nextArray, Mid);
546                 return ValueIndex(ref nextArray, Value);
547             }
548             else
549             {
550                 return 0;
551             }
552         }
553     }
```

Polygon.cs

```
    internal class Polygon : Entity//this is used as where the data for each
polygon object is stored
    {
        public PointF[] drawnPoints;// https://learn.microsoft.com/en-
us/dotnet/desktop/winforms/advanced/polygons-in-gdi?view=netframeworkdesktop-
4.8
        private float[,] vertexVectors;//stores vectors towards points
        private float angularVelocity;
        private float angularAcceleration;
        private float elasticity;
        public Polygon(float vx, float vy, float ax, float ay, float px, float
py, Pen Colour, float[,] vertexVectors, float angularVelocity, float
angularAcceleration, float elasticity, float mass) : base(vx, vy, ax, ay, px,
py, Colour, mass)
        {
            velocity = new float[] { vx, vy };
            acceleration = new float[] { ax, ay };
            position = new float[] { px, py };
            this.Colour = Colour;
            this.vertexVectors = vertexVectors;// shows where vertices are in
reference to the position/centre of mass
            this.angularVelocity = angularVelocity;
            this.angularAcceleration = angularAcceleration;
            this.elasticity = elasticity;
            this.mass = mass;
            setPosition();
            //due to center of mass, the vertex vectors must be adjusted
        }
        public void setPosition()// uses the centre to find the new position
that is seperate to the inputted position into polygon.
        {
            drawnPoints = new PointF[vertexVectors.GetLength(0)];
            for (int i = 0; i < vertexVectors.GetLength(0); i++)// makes an
array of each vertex in the polygon
            {
                drawnPoints[i].X = vertexVectors[i, 0] + position[0];
                drawnPoints[i].Y = vertexVectors[i, 1] + position[1];
            }
            float[] center = centre();
            for (int i = 0; i < vertexVectors.GetLength(0); i++)
            {
                vertexVectors[i, 0] = drawnPoints[i].X - center[0];
                vertexVectors[i, 1] = drawnPoints[i].Y - center[1];//finds new
vertex vectors depending on the new position and therefore the new points of
where the vertices are
            }
            //due to center of mass, the vertex vectors must be adjusted
        }
        public override void updatePosition()
        {
```

```
            base.updatePosition();//runs entity.updatePosition, found this code
from https://www.geeksforgeeks.org/c-sharp-method-overriding/
            float CosVelocity = (float)Math.Cos(angularVelocity * frameRate);
            float SinVelocity = (float)Math.Sin(angularVelocity * frameRate);
            for (int i = 0; i < vertexVectors.GetLength(0);
i++)//https://www.youtube.com/watch?v=fH5MJT3HI58&t=593s
            {
                float Xvector = vertexVectors[i, 0];
                float Yvector = vertexVectors[i, 1];// must be made variable to
prevent cross calculation error
                vertexVectors[i, 0] = Xvector * CosVelocity - Yvector *
SinVelocity;
                vertexVectors[i, 1] = Xvector * SinVelocity + Yvector *
CosVelocity;
            }
            // positive angular velocity causes clockwise motion
            for (int i = 0; i < drawnPoints.Length; i++)// makes each drawn
point which is a precise way of knowing where each vertex actually is
            {
                drawnPoints[i].X = vertexVectors[i, 0] + position[0];
                drawnPoints[i].Y = vertexVectors[i, 1] + position[1];
            }
        }
        public override void updateVelocity()
        {
            base.updateVelocity();
            angularVelocity = angularVelocity + angularAcceleration *
frameRate;// finds new angular velocity
        }
        public float[] centre()//this shows the maths behind finding the centre
of a polygon, https://lexrent.eu/wp-
content/uploads/torza/artikel_groep_sub_2_docs/BYZ_3_Polygon-Area-and-
Centroid.pdf?utm_source
        {
            float area = 0;
            float Cx = 0;
            float Cy = 0;//values used to find the resultant centre coordinates
            int NoVerticies = drawnPoints.GetLength(0);
            for (int i = 0; i < NoVerticies; i++)
            {
                int plus = i + 1;
                if (i + 1 > NoVerticies - 1)
                    plus = 0;
                float XVertex = drawnPoints[i].X;
                float YVertex = drawnPoints[i].Y;
                float NextXVertex = drawnPoints[plus].X;
                float NextYVertex = drawnPoints[plus].Y;
                float thatPart = XVertex * NextYVertex - NextXVertex * YVertex;

                area = area + (thatPart);
```

```
                Cx = Cx + (XVertex + NextXVertex) * (thatPart);
                Cy = Cy + (YVertex + NextYVertex) * (thatPart);
            }

            area = 0.5f * area;
            return new float[] { Cx / (6 * area), Cy / (6 * area) };
        }
        public float getMomentOfInertia()//some pretty annoying maths,
https://physics.stackexchange.com/questions/493736/moment-of-inertia-for-an-
arbitrary-polygon
        {
            //https://www.khanacademy.org/science/in-in-class11th-physics/in-
in-system-of-particles-and-rotational-motion/in-in-rotational-inertia-and-
angular-second-law/a/rotational-inertia
            float sumTerm = 0;
            float area = 0;
            int n = vertexVectors.GetLength(0);

            for (int i = 0; i < n; i++)
            {
                int plus = i + 1;
                if (plus >= n)//plus is set to 0 when it exceeds or is equal to
n to prevent out of range error
                    plus = 0;
                float xi = vertexVectors[i, 0];
                float yi = vertexVectors[i, 1];
                float xPLus = vertexVectors[plus, 0];
                float yPLus = vertexVectors[plus, 1];

                float cross = xi * xPLus - xPLus * yi;
                area += cross;
                sumTerm += cross * (xi * xi + xi * xPLus + xPLus * xPLus + yi *
yi + yi * xPLus + xPLus * xPLus);
            }

            area = Math.Abs(area) * 0.5f;
            if (area < 1e-7f) return float.PositiveInfinity; // to not divide
by 0

            return (mass / (12 * area)) * sumTerm;
        }
        // below methods to find data of the polygon
        public float getAngularVelocity()
        {
            return angularVelocity;
        }
        public float getElasticity()
        {
            return elasticity;
        }
```

```
        public float getMass()
        {
            return mass;
        }
        public void changeVelocity(float[] newVelocity)
        {
            velocity = newVelocity;
        }
        public void changeAngular(float newAngular)
        {
            angularVelocity = newAngular;
        }

    }
```

NewPolygon.cs

```
    public partial class NewPolygon : Form//this is where the data for the add
button is processed
    {
        public Pen black = new Pen(Color.Black, 4);
        public Pen blue = new Pen(Color.Blue, 4);
        public Pen Red = new Pen(Color.Red, 4);
        public Pen green = new Pen(Color.Green, 4);
        public Pen yellow = new Pen(Color.Yellow, 4);
        public Pen purple = new Pen(Color.Purple, 4);
        public Pen turquoise = new Pen(Color.Turquoise, 4);
        public Pen magenta = new Pen(Color.Magenta, 4);
        public Pen pink = new Pen(Color.Pink, 4);
        public Pen orange = new Pen(Color.Orange, 4);
        public Pen brown = new Pen(Color.Brown, 4);
        public PolyForm polyform;

        public bool PolygonAdd = false;
        public bool GravityAdd = false;//used in finding what is to be added in
poly form

        public float[] Velocity = new float[] { 10, -50 };
        public float[] Acceleration = new float[] { 0, 0 };
        public float[] Position = new float[] { 500, 500 };
        public float AngularVelocity = 0;
        public float AngularAcceleration = 0;
        public float Mass = 1;
        public float Elasticity = 1;
        public Pen Colour;
        public float[,] VertexVectors = { { 25, 25 }, { 25, -25 }, { -25, -25
}, { -25, 25 } };//these are all preset values

```

```
32          public float GravMagnitude = 10000 * (float)Math.Pow(10, 10);// same
33  reason as in circle editor, the inputted mass is increased to make the
34  gravitational force appropiately large
35          public float[] GravLocation = { 500, 400 };
36          public float UniGravMag = 100;// close to natural gravity
37
38          public List<TextBox> Values = new List<TextBox>();// for error checking
39  on whether the values are appropriate
40          public NewPolygon(PolyForm polyForm)
41          {
42              this.polyform = polyForm;// links the classes
43              Colour = black;
44              InitializeComponent();
45          }
46
47          private void ColourBox_SelectedIndexChanged(object sender, EventArgs e)
48          {
49              string selected = ColourBox.SelectedItem.ToString();
50
51              if (selected == "Black")
52              {
53                  Colour = black;
54              }
55              if (selected == "Blue")
56              {
57                  Colour = blue;
58              }
59              if (selected == "Red")
60              {
61                  Colour = Red;
62              }
63              if (selected == "Green")
64              {
65                  Colour = green;
66              }
67              if (selected == "Yellow")
68              {
69                  Colour = yellow;
70              }
71              if (selected == "Purple")
72              {
73                  Colour = purple;
74              }
75              if (selected == "Turquoise")
76              {
77                  Colour = turquoise;
78              }
79              if (selected == "Magenta")
80              {
81                  Colour = magenta;
```

```
            }
            if (selected == "Pink")
            {
                Colour = pink;
            }
            if (selected == "Orange")
            {
                Colour = orange;
            }
            if (selected == "Brown")
            {
                Colour = brown;
            }
        }
        public void finalCheck(ref bool Add)// final test for when a new object
is added or the values are saved
        {
            bool x = false;
            foreach (TextBox box in Values)
            {
                if (!EnterValidCheck(box.Text, true))
                {
                    x = true;
                }
                else if (!EnterValidCheck(box.Text, false))//if the input is a
char
                {
                    x = true;
                }
            }
            if (x)
            {
                MessageBox.Show("All values must be numbers (direction must be
a single char)");// error message
            }
            else if (!x)
            {
                Add = true;
                this.Close();
            }
        }
        private void AddPolyButton_Click(object sender, EventArgs e)
        {
            finalCheck(ref PolygonAdd);
        }
        private void AddForceButton_Click(object sender, EventArgs e)
        {
            finalCheck(ref GravityAdd);
        }
```

```
131         public bool EnterValidCheck(string String, bool Int)//check for whether
132 the data should be changed or not without making an error
133         {
134             if (Int)
135             {
136                 try
137                 {
138                     Convert.ToInt32(String);
139                 }
140                 catch
141                 {
142                     return false;
143                 }
144                 return true;
145             }
146             else
147             {
148                 try
149                 {
150                     Convert.ToChar(String);
151                 }
152                 catch
153                 {
154                     return false;
155                 }
156                 return true;
157             }
158 
159         }
160         //the following classes go through the same pattern of testing whether
161 the value is valid and then changing the newPolygon's value to that new
162 inputted value
163         private void MassBox_TextChanged(object sender, EventArgs e)
164         {
165             string strMass = MassBox.Text;
166             if (EnterValidCheck(strMass, true))//checks validity
167             {
168                 Mass = Convert.ToInt32(strMass);
169             }
170             if (!Values.Contains(MassBox))//prevents duplicates
171             {
172                 Values.Add(MassBox);
173             }
174         }
175 
176         private void VelocityXBox_TextChanged(object sender, EventArgs e)
177         {
178             string String = VelocityXBox.Text;
179             if (EnterValidCheck(String, true))
180             {
```

```
                Velocity[0] = Convert.ToInt32(String);
            }
            if (!Values.Contains(VelocityXBox))
            {
                Values.Add(VelocityXBox);
            }
        }

        private void VelocityYBox_TextChanged(object sender, EventArgs e)
        {
            string String = VelocityYBox.Text;
            if (EnterValidCheck(String, true))
            {
                Velocity[1] = Convert.ToInt32(String);
            }
            if (!Values.Contains(VelocityYBox))
            {
                Values.Add(VelocityYBox);
            }
        }

        private void AccelerationXBox_TextChanged(object sender, EventArgs e)
        {
            string String = AccelerationXBox.Text;
            if (EnterValidCheck(String, true))
            {
                Acceleration[0] = Convert.ToInt32(String);
            }
            if (!Values.Contains(AccelerationXBox))
            {
                Values.Add(AccelerationXBox);
            }
            Values.Add(AccelerationXBox);
        }

        private void AccelerationYBox_TextChanged(object sender, EventArgs e)
        {
            string String = AccelerationYBox.Text;
            if (EnterValidCheck(String, true))
            {
                Acceleration[1] = Convert.ToInt32(String);
            }
            if (!Values.Contains(AccelerationYBox))
            {
                Values.Add(AccelerationYBox);
            }
        }

        private void AngularVelBox_TextChanged(object sender, EventArgs e)
        {
```

```
231             string String = AngularVelBox.Text;
232             if (EnterValidCheck(String, true))
233             {
234                 AngularVelocity = Convert.ToInt32(String);
235             }
236             if (!Values.Contains(AngularVelBox))
237             {
238                 Values.Add(AngularVelBox);
239             }
240         }
241 
242         private void AngularAccelBox_TextChanged(object sender, EventArgs e)
243         {
244             string String = AngularAccelBox.Text;
245             if (EnterValidCheck(String, true))
246             {
247                 AngularAcceleration = Convert.ToInt32(String);
248             }
249             if (!Values.Contains(AngularAccelBox))
250             {
251                 Values.Add(AngularAccelBox);
252             }
253         }
254 
255         private void ElasticityBox_TextChanged(object sender, EventArgs e)
256         {
257             string String = ElasticityBox.Text;
258             if (EnterValidCheck(String, true))
259             {
260                 Elasticity = Convert.ToInt32(String);
261             }
262             if (!Values.Contains(ElasticityBox))
263             {
264                 Values.Add(ElasticityBox);
265             }
266         }
267 
268         private void RadialMagnitudeBox_TextChanged(object sender, EventArgs e)
269         {
270             string String = RadialMagnitudeBox.Text;
271             if (EnterValidCheck(String, true))
272             {
273                 GravMagnitude = Convert.ToInt32(String) * (float)Math.Pow(10,
274 10);
275             }
276             if (!Values.Contains(RadialMagnitudeBox))
277             {
278                 Values.Add(RadialMagnitudeBox);
279             }
280         }
```

```
281
282         private void PositionXBox_TextChanged(object sender, EventArgs e)
283         {
284             string String = PositionXBox.Text;
285             if (EnterValidCheck(String, true))
286             {
287                 GravLocation[0] = Convert.ToInt32(String);
288             }
289             if (!Values.Contains(PositionXBox))
290             {
291                 Values.Add(PositionXBox);
292             }
293         }
294
295         private void PositionYBox_TextChanged(object sender, EventArgs e)
296         {
297             string String = PositionYBox.Text;
298             if (EnterValidCheck(String, true))
299             {
300                 GravLocation[1] = Convert.ToInt32(String);
301             }
302             if (!Values.Contains(PositionYBox))
303             {
304                 Values.Add(PositionYBox);
305             }
306         }
307
308         private void UniformMagBox_TextChanged(object sender, EventArgs e)
309         {
310             string String = UniformMagBox.Text;
311             if (EnterValidCheck(String, true))
312             {
313                 UniGravMag = Convert.ToInt32(String);
314             }
315             if(!Values.Contains(UniformMagBox))
316             {
317                 Values.Add(UniformMagBox);
318             }
319         }
320
321         private void UniformDirectionBox_TextChanged(object sender, EventArgs
322 e)
323         {
324             string String = UniformDirectionBox.Text;
325             if (EnterValidCheck(String, false))
326             {
327                 polyform.Side = Convert.ToChar(String);
328             }
329             if (!Values.Contains(UniformDirectionBox))
330             {
```

```
                Values.Add(UniformDirectionBox);
            }
        }

        private void Close_Click(object sender, EventArgs e)
        {
            this.Close();//quit button
        }
    }
```

Simplex.cs

```
    internal class Simplex//used in GJK as an abstract array,
    {
        public float[,] vertexes = new float[3,2];
        protected int top;
        public Simplex()
        {
            top = -1;// so that when we add a value top = 0
        }
        public virtual void push(float[] Point)// sets top of stack to Point
        {
            if (top == 2)//there are already 3 point in simplex
            {
                deque(2);
            }
            top++;
            vertexes[top, 0] = Point[0];
            vertexes[top, 1] = Point[1];
        }
        public void pop()// removes top value by reducing top
        {
            top--;
        }
        public void deque(int k)//k represents the position that is removed
        {

            for (int i = 0; i < 3; i++)// move each point back a point
            {
                if (i > k && i != 0)//so it moves every item in list after k,
down 1, while everything before k stays in place
                {
                    vertexes[i - 1, 0] = vertexes[i, 0];
                    vertexes[i - 1, 1] = vertexes[i, 1];//moves coordinate in
position i to the position below them
                }
            }
            top--;//array has reduced so top is also decremented
```

```
        }
        public int Top()
        {
            return top;
        }
    }
```

Polytope.cs

```
    internal class Polytope : Simplex//this is an abstract array that is larger
than simplex and allows vectors to be added to a specific point in its array
    {
        public Polytope() : base()
        {
            vertexes = new float[10, 2];//polytope needs to be much larger than
simplex while still being an array, but it shouldnt become larger than 10
points
            top = -1;
        }
        public override void push(float[] Point)
        {
            if (top == 10)// so that it does not overflow
            {
                deque(2);
            }
            top++;
            vertexes[top, 0] = Point[0];
            vertexes[top, 1] = Point[1];
        }
        public void indexPush(float[] Point, int k)// puts a coordinate at a
specific point in vertexes
        {
            for (int i = 0; i < vertexes.GetLength(0)-1; i++)
            {
                if (i > k)
                {
                    vertexes[i + 1, 0] = vertexes[i, 0];
                    vertexes[i + 1, 1] = vertexes[i, 1];//moves ceach
corrdinate up 1 after k
                }
                vertexes[k, 0] = Point[0];
                vertexes[k, 1] = Point[1];//set the value at k
            }
        }
    }
```

Border.cs

```
    internal class Border : Polygon// this is to be used as an easy way to make
immovable polygons
    {
        public Border(float Px, float Py, float[,] vertexVectors) : base(0, 0,
0, 0, Px, Py, new Pen(Color.Blue, 4), vertexVectors, 0, 0, 0,
float.PositiveInfinity)
        {
            this.position[0] = Px;
            this.position[1] = Py;
            setPosition();//find its centre of mass
        }
    }
```

Collision.cs

```
    internal class Collision//collision is just storage for the data about a
collision
    {
        public Polygon A;
        public Polygon B;
        public Simplex simplex = new Simplex();
        public Collision(Polygon A, Polygon B, Simplex simplex)
        {
            this.A = A;
            this.B = B;
            this.simplex = simplex;
        }
    }
```

Designer code for forms other than CircleManager:

Circle manager designer code is in itself. The following code is automatically generated by visual studio due to items added by me in windows forms editors.

Start.Designer.cs

```
    partial class Start
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
```

```
10          /// </summary>
11          /// <param name="disposing">true if managed resources should be
12  disposed; otherwise, false.</param>
13          protected override void Dispose(bool disposing)
14          {
15              if (disposing && (components != null))
16              {
17                  components.Dispose();
18              }
19              base.Dispose(disposing);
20          }
21
22          #region Windows Form Designer generated code
23
24          /// <summary>
25          /// Required method for Designer support - do not modify
26          /// the contents of this method with the code editor.
27          /// </summary>
28          private void InitializeComponent()
29          {
30              this.label1 = new System.Windows.Forms.Label();
31              this.PolySim = new System.Windows.Forms.Button();
32              this.CircleSim = new System.Windows.Forms.Button();
33              this.Quit = new System.Windows.Forms.Button();
34              this.SuspendLayout();
35              //
36              // label1
37              //
38              this.label1.AutoSize = true;
39              this.label1.Font = new System.Drawing.Font("Microsoft Sans Serif",
40  10F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point,
41  ((byte)(0)));
42              this.label1.Location = new System.Drawing.Point(90, 44);
43              this.label1.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
44              this.label1.Name = "label1";
45              this.label1.Size = new System.Drawing.Size(139, 17);
46              this.label1.TabIndex = 0;
47              this.label1.Text = "Choose a simulation:";
48              //
49              // PolySim
50              //
51              this.PolySim.Location = new System.Drawing.Point(62, 94);
52              this.PolySim.Margin = new System.Windows.Forms.Padding(2);
53              this.PolySim.Name = "PolySim";
54              this.PolySim.Size = new System.Drawing.Size(95, 56);
55              this.PolySim.TabIndex = 1;
56              this.PolySim.Text = "Polygon Sim";
57              this.PolySim.UseVisualStyleBackColor = true;
58              this.PolySim.Click += new System.EventHandler(this.PolySim_Click);
59              //
```

```
            // CircleSim
            //
            this.CircleSim.Location = new System.Drawing.Point(161, 94);
            this.CircleSim.Margin = new System.Windows.Forms.Padding(2);
            this.CircleSim.Name = "CircleSim";
            this.CircleSim.Size = new System.Drawing.Size(95, 56);
            this.CircleSim.TabIndex = 2;
            this.CircleSim.Text = "Circle only sim";
            this.CircleSim.UseVisualStyleBackColor = true;
            this.CircleSim.Click += new
System.EventHandler(this.CircleSim_Click);
            //
            // Quit
            //
            this.Quit.BackColor =
System.Drawing.Color.FromArgb(((int)(((byte)(255)))), ((int)(((byte)(192)))),
((int)(((byte)(128)))));
            this.Quit.Location = new System.Drawing.Point(121, 165);
            this.Quit.Name = "Quit";
            this.Quit.Size = new System.Drawing.Size(75, 23);
            this.Quit.TabIndex = 3;
            this.Quit.Text = "Quit";
            this.Quit.UseVisualStyleBackColor = false;
            this.Quit.Click += new System.EventHandler(this.Quit_Click);
            //
            // Start
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(327, 241);
            this.Controls.Add(this.Quit);
            this.Controls.Add(this.CircleSim);
            this.Controls.Add(this.PolySim);
            this.Controls.Add(this.label1);
            this.Margin = new System.Windows.Forms.Padding(2);
            this.Name = "Start";
            this.Text = "Start";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label label1;
        private System.Windows.Forms.Button PolySim;
        private System.Windows.Forms.Button CircleSim;
        private System.Windows.Forms.Button Quit;
    }
```

```
    partial class PolyForm
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be
disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.Quit = new System.Windows.Forms.Button();
            this.AddPolygon = new System.Windows.Forms.Button();
            this.CollisionCheck = new System.Windows.Forms.CheckBox();
            this.Paused = new System.Windows.Forms.CheckBox();
            this.Clear = new System.Windows.Forms.Button();
            this.label1 = new System.Windows.Forms.Label();
            this.Gravity = new System.Windows.Forms.CheckBox();
            this.SuspendLayout();
            //
            // Quit
            //
            this.Quit.BackColor =
System.Drawing.Color.FromArgb(((int)(((byte)(255)))), ((int)(((byte)(128)))),
((int)(((byte)(128)))));
            this.Quit.Location = new System.Drawing.Point(722, 662);
            this.Quit.Margin = new System.Windows.Forms.Padding(2);
            this.Quit.Name = "Quit";
            this.Quit.Size = new System.Drawing.Size(56, 19);
```

```
48              this.Quit.TabIndex = 0;
49              this.Quit.Text = "Quit";
50              this.Quit.UseVisualStyleBackColor = false;
51              this.Quit.Click += new System.EventHandler(this.Quit_Click);
52              //
53              // AddPolygon
54              //
55              this.AddPolygon.Location = new System.Drawing.Point(366, 658);
56              this.AddPolygon.Margin = new System.Windows.Forms.Padding(2);
57              this.AddPolygon.Name = "AddPolygon";
58              this.AddPolygon.Size = new System.Drawing.Size(43, 23);
59              this.AddPolygon.TabIndex = 1;
60              this.AddPolygon.Text = "Add";
61              this.AddPolygon.UseVisualStyleBackColor = true;
62              this.AddPolygon.Click += new
63  System.EventHandler(this.AddPolygon_Click);
64              //
65              // CollisionCheck
66              //
67              this.CollisionCheck.AutoSize = true;
68              this.CollisionCheck.Checked = true;
69              this.CollisionCheck.CheckState =
70  System.Windows.Forms.CheckState.Checked;
71              this.CollisionCheck.Location = new System.Drawing.Point(503, 662);
72              this.CollisionCheck.Margin = new System.Windows.Forms.Padding(2);
73              this.CollisionCheck.Name = "CollisionCheck";
74              this.CollisionCheck.Size = new System.Drawing.Size(64, 17);
75              this.CollisionCheck.TabIndex = 2;
76              this.CollisionCheck.Text = "Collision";
77              this.CollisionCheck.UseVisualStyleBackColor = true;
78              //
79              // Paused
80              //
81              this.Paused.AutoSize = true;
82              this.Paused.Location = new System.Drawing.Point(568, 662);
83              this.Paused.Margin = new System.Windows.Forms.Padding(2);
84              this.Paused.Name = "Paused";
85              this.Paused.Size = new System.Drawing.Size(62, 17);
86              this.Paused.TabIndex = 3;
87              this.Paused.Text = "Paused";
88              this.Paused.UseVisualStyleBackColor = true;
89              //
90              // Clear
91              //
92              this.Clear.Location = new System.Drawing.Point(9, 658);
93              this.Clear.Margin = new System.Windows.Forms.Padding(2);
94              this.Clear.Name = "Clear";
95              this.Clear.Size = new System.Drawing.Size(56, 19);
96              this.Clear.TabIndex = 4;
97              this.Clear.Text = "Clear";
```

```
            this.Clear.UseVisualStyleBackColor = true;
            this.Clear.Click += new System.EventHandler(this.Clear_Click);
            //
            // label1
            //
            this.label1.AutoSize = true;
            this.label1.Location = new System.Drawing.Point(313, 643);
            this.label1.Name = "label1";
            this.label1.Size = new System.Drawing.Size(168, 13);
            this.label1.TabIndex = 6;
            this.label1.Text = "Click for details on adding objects:";
            //
            // Gravity
            //
            this.Gravity.AutoSize = true;
            this.Gravity.Location = new System.Drawing.Point(635, 662);
            this.Gravity.Name = "Gravity";
            this.Gravity.Size = new System.Drawing.Size(59, 17);
            this.Gravity.TabIndex = 7;
            this.Gravity.Text = "Gravity";
            this.Gravity.UseVisualStyleBackColor = true;
            //
            // PolyForm
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(788, 691);
            this.Controls.Add(this.Gravity);
            this.Controls.Add(this.label1);
            this.Controls.Add(this.Clear);
            this.Controls.Add(this.Paused);
            this.Controls.Add(this.CollisionCheck);
            this.Controls.Add(this.AddPolygon);
            this.Controls.Add(this.Quit);
            this.Margin = new System.Windows.Forms.Padding(2);
            this.Name = "PolyForm";
            this.Text = "Form1";
            this.MouseClick += new
System.Windows.Forms.MouseEventHandler(this.PolyForm_MouseClick);
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private Button Quit;
        private Button AddPolygon;
        public CheckBox CollisionCheck;
        public CheckBox Paused;
```

```
148         private Button Clear;
149         private Label label1;
150         public CheckBox Gravity;
151     }
```

NewPolygon.Designer.cs

```
1       partial class NewPolygon
2       {
3           /// <summary>
4           /// Required designer variable.
5           /// </summary>
6           private System.ComponentModel.IContainer components = null;
7
8           /// <summary>
9           /// Clean up any resources being used.
10          /// </summary>
11          /// <param name="disposing">true if managed resources should be
12  disposed; otherwise, false.</param>
13          protected override void Dispose(bool disposing)
14          {
15              if (disposing && (components != null))
16              {
17                  components.Dispose();
18              }
19              base.Dispose(disposing);
20          }
21
22          #region Windows Form Designer generated code
23
24          /// <summary>
25          /// Required method for Designer support - do not modify
26          /// the contents of this method with the code editor.
27          /// </summary>
28          private void InitializeComponent()
29          {
30              System.ComponentModel.ComponentResourceManager resources = new
31  System.ComponentModel.ComponentResourceManager(typeof(NewPolygon));
32              this.label1 = new System.Windows.Forms.Label();
33              this.MassBox = new System.Windows.Forms.TextBox();
34              this.label2 = new System.Windows.Forms.Label();
35              this.label3 = new System.Windows.Forms.Label();
36              this.VelocityXBox = new System.Windows.Forms.TextBox();
37              this.label4 = new System.Windows.Forms.Label();
38              this.label5 = new System.Windows.Forms.Label();
39              this.VelocityYBox = new System.Windows.Forms.TextBox();
40              this.label6 = new System.Windows.Forms.Label();
41              this.ElasticityBox = new System.Windows.Forms.TextBox();
```

```
42              this.label7 = new System.Windows.Forms.Label();
43              this.AccelerationYBox = new System.Windows.Forms.TextBox();
44              this.label8 = new System.Windows.Forms.Label();
45              this.label9 = new System.Windows.Forms.Label();
46              this.AccelerationXBox = new System.Windows.Forms.TextBox();
47              this.AngularVelBox = new System.Windows.Forms.TextBox();
48              this.label10 = new System.Windows.Forms.Label();
49              this.AngularAccelBox = new System.Windows.Forms.TextBox();
50              this.label11 = new System.Windows.Forms.Label();
51              this.ColourBox = new System.Windows.Forms.ComboBox();
52              this.label12 = new System.Windows.Forms.Label();
53              this.AddPolyButton = new System.Windows.Forms.Button();
54              this.AddForceButton = new System.Windows.Forms.Button();
55              this.PositionYBox = new System.Windows.Forms.TextBox();
56              this.label20 = new System.Windows.Forms.Label();
57              this.label21 = new System.Windows.Forms.Label();
58              this.PositionXBox = new System.Windows.Forms.TextBox();
59              this.label22 = new System.Windows.Forms.Label();
60              this.label23 = new System.Windows.Forms.Label();
61              this.RadialMagnitudeBox = new System.Windows.Forms.TextBox();
62              this.label13 = new System.Windows.Forms.Label();
63              this.label14 = new System.Windows.Forms.Label();
64              this.label15 = new System.Windows.Forms.Label();
65              this.UniformMagBox = new System.Windows.Forms.TextBox();
66              this.label16 = new System.Windows.Forms.Label();
67              this.label17 = new System.Windows.Forms.Label();
68              this.label18 = new System.Windows.Forms.Label();
69              this.UniformDirectionBox = new System.Windows.Forms.TextBox();
70              this.Close = new System.Windows.Forms.Button();
71              this.SuspendLayout();
72              //
73              // label1
74              //
75              this.label1.AutoSize = true;
76              this.label1.Font = new System.Drawing.Font("Microsoft Sans Serif",
77  9F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point,
78  ((byte)(0)));
79              this.label1.Location = new System.Drawing.Point(9, 6);
80              this.label1.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
81              this.label1.Name = "label1";
82              this.label1.Size = new System.Drawing.Size(206, 15);
83              this.label1.TabIndex = 0;
84              this.label1.Text = "Enter the values for the new polygon:\r\n";
85              //
86              // MassBox
87              //
88              this.MassBox.Location = new System.Drawing.Point(115, 26);
89              this.MassBox.Margin = new System.Windows.Forms.Padding(2);
90              this.MassBox.Name = "MassBox";
91              this.MassBox.Size = new System.Drawing.Size(76, 20);
```

```
            this.MassBox.TabIndex = 1;
            this.MassBox.Text = "1";
            this.MassBox.TextChanged += new
System.EventHandler(this.MassBox_TextChanged);
            //
            // label2
            //
            this.label2.AutoSize = true;
            this.label2.Location = new System.Drawing.Point(75, 28);
            this.label2.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label2.Name = "label2";
            this.label2.Size = new System.Drawing.Size(35, 13);
            this.label2.TabIndex = 2;
            this.label2.Text = "Mass:";
            //
            // label3
            //
            this.label3.AutoSize = true;
            this.label3.Location = new System.Drawing.Point(56, 51);
            this.label3.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label3.Name = "label3";
            this.label3.Size = new System.Drawing.Size(55, 13);
            this.label3.TabIndex = 3;
            this.label3.Text = "Velocities:";
            //
            // VelocityXBox
            //
            this.VelocityXBox.Location = new System.Drawing.Point(128, 49);
            this.VelocityXBox.Margin = new System.Windows.Forms.Padding(2);
            this.VelocityXBox.Name = "VelocityXBox";
            this.VelocityXBox.Size = new System.Drawing.Size(24, 20);
            this.VelocityXBox.TabIndex = 4;
            this.VelocityXBox.Text = "10";
            this.VelocityXBox.TextChanged += new
System.EventHandler(this.VelocityXBox_TextChanged);
            //
            // label4
            //
            this.label4.AutoSize = true;
            this.label4.Location = new System.Drawing.Point(117, 52);
            this.label4.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label4.Name = "label4";
            this.label4.Size = new System.Drawing.Size(14, 13);
            this.label4.TabIndex = 5;
            this.label4.Text = "X";
            //
            // label5
            //
            this.label5.AutoSize = true;
            this.label5.Location = new System.Drawing.Point(155, 51);
```

```
            this.label5.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label5.Name = "label5";
            this.label5.Size = new System.Drawing.Size(14, 13);
            this.label5.TabIndex = 6;
            this.label5.Text = "Y";
            //
            // VelocityYBox
            //
            this.VelocityYBox.Location = new System.Drawing.Point(166, 49);
            this.VelocityYBox.Margin = new System.Windows.Forms.Padding(2);
            this.VelocityYBox.Name = "VelocityYBox";
            this.VelocityYBox.Size = new System.Drawing.Size(25, 20);
            this.VelocityYBox.TabIndex = 7;
            this.VelocityYBox.Text = "-50";
            this.VelocityYBox.TextChanged += new
System.EventHandler(this.VelocityYBox_TextChanged);
            //
            // label6
            //
            this.label6.AutoSize = true;
            this.label6.Location = new System.Drawing.Point(60, 144);
            this.label6.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label6.Name = "label6";
            this.label6.Size = new System.Drawing.Size(51, 13);
            this.label6.TabIndex = 8;
            this.label6.Text = "Elasticity:";
            //
            // ElasticityBox
            //
            this.ElasticityBox.Location = new System.Drawing.Point(115, 141);
            this.ElasticityBox.Margin = new System.Windows.Forms.Padding(2);
            this.ElasticityBox.Name = "ElasticityBox";
            this.ElasticityBox.Size = new System.Drawing.Size(76, 20);
            this.ElasticityBox.TabIndex = 9;
            this.ElasticityBox.Text = "1";
            this.ElasticityBox.TextChanged += new
System.EventHandler(this.ElasticityBox_TextChanged);
            //
            // label7
            //
            this.label7.AutoSize = true;
            this.label7.Location = new System.Drawing.Point(44, 76);
            this.label7.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label7.Name = "label7";
            this.label7.Size = new System.Drawing.Size(69, 13);
            this.label7.TabIndex = 10;
            this.label7.Text = "Acceleration:";
            //
            // AccelerationYBox
            //
```

```
192             this.AccelerationYBox.Location = new System.Drawing.Point(166, 74);
193             this.AccelerationYBox.Margin = new System.Windows.Forms.Padding(2);
194             this.AccelerationYBox.Name = "AccelerationYBox";
195             this.AccelerationYBox.Size = new System.Drawing.Size(25, 20);
196             this.AccelerationYBox.TabIndex = 15;
197             this.AccelerationYBox.Text = "0";
198             this.AccelerationYBox.TextChanged += new
199 System.EventHandler(this.AccelerationYBox_TextChanged);
200             //
201             // label8
202             //
203             this.label8.AutoSize = true;
204             this.label8.Location = new System.Drawing.Point(155, 76);
205             this.label8.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
206             this.label8.Name = "label8";
207             this.label8.Size = new System.Drawing.Size(14, 13);
208             this.label8.TabIndex = 14;
209             this.label8.Text = "Y";
210             //
211             // label9
212             //
213             this.label9.AutoSize = true;
214             this.label9.Location = new System.Drawing.Point(117, 77);
215             this.label9.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
216             this.label9.Name = "label9";
217             this.label9.Size = new System.Drawing.Size(14, 13);
218             this.label9.TabIndex = 13;
219             this.label9.Text = "X";
220             //
221             // AccelerationXBox
222             //
223             this.AccelerationXBox.Location = new System.Drawing.Point(128, 74);
224             this.AccelerationXBox.Margin = new System.Windows.Forms.Padding(2);
225             this.AccelerationXBox.Name = "AccelerationXBox";
226             this.AccelerationXBox.Size = new System.Drawing.Size(24, 20);
227             this.AccelerationXBox.TabIndex = 12;
228             this.AccelerationXBox.Text = "0";
229             this.AccelerationXBox.TextChanged += new
230 System.EventHandler(this.AccelerationXBox_TextChanged);
231             //
232             // AngularVelBox
233             //
234             this.AngularVelBox.Location = new System.Drawing.Point(115, 96);
235             this.AngularVelBox.Margin = new System.Windows.Forms.Padding(2);
236             this.AngularVelBox.Name = "AngularVelBox";
237             this.AngularVelBox.Size = new System.Drawing.Size(76, 20);
238             this.AngularVelBox.TabIndex = 17;
239             this.AngularVelBox.Text = "0";
240             this.AngularVelBox.TextChanged += new
241 System.EventHandler(this.AngularVelBox_TextChanged);
```

```
242             //
243             // label10
244             //
245             this.label10.AutoSize = true;
246             this.label10.Location = new System.Drawing.Point(28, 98);
247             this.label10.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
248             this.label10.Name = "label10";
249             this.label10.Size = new System.Drawing.Size(85, 13);
250             this.label10.TabIndex = 16;
251             this.label10.Text = "Angular velocity:";
252             //
253             // AngularAccelBox
254             //
255             this.AngularAccelBox.Location = new System.Drawing.Point(115, 119);
256             this.AngularAccelBox.Margin = new System.Windows.Forms.Padding(2);
257             this.AngularAccelBox.Name = "AngularAccelBox";
258             this.AngularAccelBox.Size = new System.Drawing.Size(76, 20);
259             this.AngularAccelBox.TabIndex = 19;
260             this.AngularAccelBox.Text = "0";
261             this.AngularAccelBox.TextChanged += new
262 System.EventHandler(this.AngularAccelBox_TextChanged);
263             //
264             // label11
265             //
266             this.label11.AutoSize = true;
267             this.label11.Location = new System.Drawing.Point(8, 121);
268             this.label11.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
269             this.label11.Name = "label11";
270             this.label11.Size = new System.Drawing.Size(107, 13);
271             this.label11.TabIndex = 18;
272             this.label11.Text = "Angular acceleration:";
273             //
274             // ColourBox
275             //
276             this.ColourBox.FormattingEnabled = true;
277             this.ColourBox.Items.AddRange(new object[] {
278             "Black",
279             "Blue",
280             "Red",
281             "Green",
282             "Yellow",
283             "Purple",
284             "Turquoise",
285             "Magenta",
286             "Pink",
287             "Orange",
288             "Brown"});
289             this.ColourBox.Location = new System.Drawing.Point(115, 164);
290             this.ColourBox.Margin = new System.Windows.Forms.Padding(2);
291             this.ColourBox.Name = "ColourBox";
```

```
292             this.ColourBox.Size = new System.Drawing.Size(92, 21);
293             this.ColourBox.TabIndex = 20;
294             this.ColourBox.SelectedIndexChanged += new
295 System.EventHandler(this.ColourBox_SelectedIndexChanged);
296             //
297             // label12
298             //
299             this.label12.AutoSize = true;
300             this.label12.Location = new System.Drawing.Point(33, 167);
301             this.label12.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
302             this.label12.Name = "label12";
303             this.label12.Size = new System.Drawing.Size(80, 13);
304             this.label12.TabIndex = 21;
305             this.label12.Text = "Change Colour:";
306             //
307             // AddPolyButton
308             //
309             this.AddPolyButton.Location = new System.Drawing.Point(54, 198);
310             this.AddPolyButton.Margin = new System.Windows.Forms.Padding(2);
311             this.AddPolyButton.Name = "AddPolyButton";
312             this.AddPolyButton.Size = new System.Drawing.Size(129, 30);
313             this.AddPolyButton.TabIndex = 22;
314             this.AddPolyButton.Text = "Add the polygon";
315             this.AddPolyButton.UseVisualStyleBackColor = true;
316             this.AddPolyButton.Click += new
317 System.EventHandler(this.AddPolyButton_Click);
318             //
319             // AddForceButton
320             //
321             this.AddForceButton.Location = new System.Drawing.Point(269, 198);
322             this.AddForceButton.Margin = new System.Windows.Forms.Padding(2);
323             this.AddForceButton.Name = "AddForceButton";
324             this.AddForceButton.Size = new System.Drawing.Size(107, 30);
325             this.AddForceButton.TabIndex = 43;
326             this.AddForceButton.Text = "Add Force object";
327             this.AddForceButton.UseVisualStyleBackColor = true;
328             this.AddForceButton.Click += new
329 System.EventHandler(this.AddForceButton_Click);
330             //
331             // PositionYBox
332             //
333             this.PositionYBox.Location = new System.Drawing.Point(368, 49);
334             this.PositionYBox.Margin = new System.Windows.Forms.Padding(2);
335             this.PositionYBox.Name = "PositionYBox";
336             this.PositionYBox.Size = new System.Drawing.Size(25, 20);
337             this.PositionYBox.TabIndex = 29;
338             this.PositionYBox.Text = "400";
339             this.PositionYBox.TextChanged += new
340 System.EventHandler(this.PositionYBox_TextChanged);
341             //
```

```
            // label20
            //
            this.label20.AutoSize = true;
            this.label20.Location = new System.Drawing.Point(355, 52);
            this.label20.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label20.Name = "label20";
            this.label20.Size = new System.Drawing.Size(14, 13);
            this.label20.TabIndex = 28;
            this.label20.Text = "Y";
            //
            // label21
            //
            this.label21.AutoSize = true;
            this.label21.Location = new System.Drawing.Point(317, 52);
            this.label21.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label21.Name = "label21";
            this.label21.Size = new System.Drawing.Size(14, 13);
            this.label21.TabIndex = 27;
            this.label21.Text = "X";
            //
            // PositionXBox
            //
            this.PositionXBox.Location = new System.Drawing.Point(331, 49);
            this.PositionXBox.Margin = new System.Windows.Forms.Padding(2);
            this.PositionXBox.Name = "PositionXBox";
            this.PositionXBox.Size = new System.Drawing.Size(38, 20);
            this.PositionXBox.TabIndex = 26;
            this.PositionXBox.Text = "500";
            this.PositionXBox.TextChanged += new
System.EventHandler(this.PositionXBox_TextChanged);
            //
            // label22
            //
            this.label22.AutoSize = true;
            this.label22.Location = new System.Drawing.Point(266, 51);
            this.label22.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label22.Name = "label22";
            this.label22.Size = new System.Drawing.Size(47, 13);
            this.label22.TabIndex = 25;
            this.label22.Text = "Position:";
            //
            // label23
            //
            this.label23.AutoSize = true;
            this.label23.Location = new System.Drawing.Point(255, 28);
            this.label23.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label23.Name = "label23";
            this.label23.Size = new System.Drawing.Size(60, 13);
            this.label23.TabIndex = 24;
            this.label23.Text = "Magnitude:";
```

```
            //
            // RadialMagnitudeBox
            //
            this.RadialMagnitudeBox.Location = new System.Drawing.Point(317,
26);
            this.RadialMagnitudeBox.Margin = new
System.Windows.Forms.Padding(2);
            this.RadialMagnitudeBox.Name = "RadialMagnitudeBox";
            this.RadialMagnitudeBox.Size = new System.Drawing.Size(76, 20);
            this.RadialMagnitudeBox.TabIndex = 23;
            this.RadialMagnitudeBox.Text = "10000";
            this.RadialMagnitudeBox.TextChanged += new
System.EventHandler(this.RadialMagnitudeBox_TextChanged);
            //
            // label13
            //
            this.label13.AutoSize = true;
            this.label13.Font = new System.Drawing.Font("Microsoft Sans Serif",
9F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point,
((byte)(0)));
            this.label13.Location = new System.Drawing.Point(228, 7);
            this.label13.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label13.Name = "label13";
            this.label13.Size = new System.Drawing.Size(220, 15);
            this.label13.TabIndex = 44;
            this.label13.Text = "Enter values for a new radial force
field:\r\n";
            //
            // label14
            //
            this.label14.AutoSize = true;
            this.label14.Font = new System.Drawing.Font("Microsoft Sans Serif",
9F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point,
((byte)(0)));
            this.label14.Location = new System.Drawing.Point(260, 99);
            this.label14.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label14.Name = "label14";
            this.label14.Size = new System.Drawing.Size(124, 15);
            this.label14.TabIndex = 45;
            this.label14.Text = "Change uniform field:";
            //
            // label15
            //
            this.label15.AutoSize = true;
            this.label15.Location = new System.Drawing.Point(255, 119);
            this.label15.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label15.Name = "label15";
            this.label15.Size = new System.Drawing.Size(60, 13);
            this.label15.TabIndex = 47;
            this.label15.Text = "Magnitude:";
```

```
            //
            // UniformMagBox
            //
            this.UniformMagBox.Location = new System.Drawing.Point(317, 116);
            this.UniformMagBox.Margin = new System.Windows.Forms.Padding(2);
            this.UniformMagBox.Name = "UniformMagBox";
            this.UniformMagBox.Size = new System.Drawing.Size(76, 20);
            this.UniformMagBox.TabIndex = 46;
            this.UniformMagBox.Text = "98";
            this.UniformMagBox.TextChanged += new
System.EventHandler(this.UniformMagBox_TextChanged);
            //
            // label16
            //
            this.label16.AutoSize = true;
            this.label16.Font = new System.Drawing.Font("Microsoft Sans Serif",
10F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point,
((byte)(0)));
            this.label16.Location = new System.Drawing.Point(9, 285);
            this.label16.Name = "label16";
            this.label16.Size = new System.Drawing.Size(82, 17);
            this.label16.TabIndex = 49;
            this.label16.Text = "Information:";
            //
            // label17
            //
            this.label17.AutoSize = true;
            this.label17.Location = new System.Drawing.Point(12, 306);
            this.label17.Name = "label17";
            this.label17.Size = new System.Drawing.Size(284, 91);
            this.label17.TabIndex = 50;
            this.label17.Text = resources.GetString("label17.Text");
            //
            // label18
            //
            this.label18.AutoSize = true;
            this.label18.Location = new System.Drawing.Point(252, 144);
            this.label18.Name = "label18";
            this.label18.Size = new System.Drawing.Size(61, 26);
            this.label18.TabIndex = 51;
            this.label18.Text = "Direction:\r\n(U, D, L, R)";
            //
            // UniformDirectionBox
            //
            this.UniformDirectionBox.Location = new System.Drawing.Point(317,
144);
            this.UniformDirectionBox.Margin = new
System.Windows.Forms.Padding(2);
            this.UniformDirectionBox.Name = "UniformDirectionBox";
            this.UniformDirectionBox.Size = new System.Drawing.Size(76, 20);
```

```
            this.UniformDirectionBox.TabIndex = 52;
            this.UniformDirectionBox.Text = "D";
            this.UniformDirectionBox.TextChanged += new
System.EventHandler(this.UniformDirectionBox_TextChanged);
            //
            // Close
            //
            this.Close.BackColor =
System.Drawing.Color.FromArgb(((int)(((byte)(255)))), ((int)(((byte)(192)))),
((int)(((byte)(128)))));
            this.Close.Location = new System.Drawing.Point(180, 250);
            this.Close.Name = "Close";
            this.Close.Size = new System.Drawing.Size(75, 23);
            this.Close.TabIndex = 53;
            this.Close.Text = "Close/Save";
            this.Close.UseVisualStyleBackColor = false;
            this.Close.Click += new System.EventHandler(this.Close_Click);
            //
            // NewPolygon
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(435, 419);
            this.Controls.Add(this.Close);
            this.Controls.Add(this.UniformDirectionBox);
            this.Controls.Add(this.label18);
            this.Controls.Add(this.label17);
            this.Controls.Add(this.label16);
            this.Controls.Add(this.label15);
            this.Controls.Add(this.UniformMagBox);
            this.Controls.Add(this.label14);
            this.Controls.Add(this.label13);
            this.Controls.Add(this.AddForceButton);
            this.Controls.Add(this.PositionYBox);
            this.Controls.Add(this.label20);
            this.Controls.Add(this.label21);
            this.Controls.Add(this.PositionXBox);
            this.Controls.Add(this.label22);
            this.Controls.Add(this.label23);
            this.Controls.Add(this.RadialMagnitudeBox);
            this.Controls.Add(this.AddPolyButton);
            this.Controls.Add(this.label12);
            this.Controls.Add(this.ColourBox);
            this.Controls.Add(this.AngularAccelBox);
            this.Controls.Add(this.label11);
            this.Controls.Add(this.AngularVelBox);
            this.Controls.Add(this.label10);
            this.Controls.Add(this.AccelerationYBox);
            this.Controls.Add(this.label8);
            this.Controls.Add(this.label9);
```

```
542             this.Controls.Add(this.AccelerationXBox);
543             this.Controls.Add(this.label7);
544             this.Controls.Add(this.ElasticityBox);
545             this.Controls.Add(this.label6);
546             this.Controls.Add(this.VelocityYBox);
547             this.Controls.Add(this.label5);
548             this.Controls.Add(this.label4);
549             this.Controls.Add(this.VelocityXBox);
550             this.Controls.Add(this.label3);
551             this.Controls.Add(this.label2);
552             this.Controls.Add(this.MassBox);
553             this.Controls.Add(this.label1);
554             this.Margin = new System.Windows.Forms.Padding(2);
555             this.Name = "NewPolygon";
556             this.Text = "NewPolygon";
557             this.ResumeLayout(false);
558             this.PerformLayout();
559 
560         }
561 
562         #endregion
563 
564         private System.Windows.Forms.Label label1;
565         private System.Windows.Forms.TextBox MassBox;
566         private System.Windows.Forms.Label label2;
567         private System.Windows.Forms.Label label3;
568         private System.Windows.Forms.TextBox VelocityXBox;
569         private System.Windows.Forms.Label label4;
570         private System.Windows.Forms.Label label5;
571         private System.Windows.Forms.TextBox VelocityYBox;
572         private System.Windows.Forms.Label label6;
573         private System.Windows.Forms.TextBox ElasticityBox;
574         private System.Windows.Forms.Label label7;
575         private System.Windows.Forms.TextBox AccelerationYBox;
576         private System.Windows.Forms.Label label8;
577         private System.Windows.Forms.Label label9;
578         private System.Windows.Forms.TextBox AccelerationXBox;
579         private System.Windows.Forms.TextBox AngularVelBox;
580         private System.Windows.Forms.Label label10;
581         private System.Windows.Forms.TextBox AngularAccelBox;
582         private System.Windows.Forms.Label label11;
583         private System.Windows.Forms.ComboBox ColourBox;
584         private System.Windows.Forms.Label label12;
585         private System.Windows.Forms.Button AddPolyButton;
586         private System.Windows.Forms.Button AddForceButton;
587         private System.Windows.Forms.TextBox PositionYBox;
588         private System.Windows.Forms.Label label20;
589         private System.Windows.Forms.Label label21;
590         private System.Windows.Forms.TextBox PositionXBox;
591         private System.Windows.Forms.Label label22;
```

```
592             private System.Windows.Forms.Label label23;
593             private System.Windows.Forms.TextBox RadialMagnitudeBox;
594             private System.Windows.Forms.Label label13;
595             private System.Windows.Forms.Label label14;
596             private System.Windows.Forms.Label label15;
597             private System.Windows.Forms.TextBox UniformMagBox;
598             private System.Windows.Forms.Label label16;
599             private System.Windows.Forms.Label label17;
600             private System.Windows.Forms.Label label18;
601             private System.Windows.Forms.TextBox UniformDirectionBox;
602             private System.Windows.Forms.Button Close;
603         }
```

CircleEditor.Designer.cs

```
1       partial class CircleEditor
2       {
3           /// <summary>
4           /// Required designer variable.
5           /// </summary>
6           private System.ComponentModel.IContainer components = null;
7
8           /// <summary>
9           /// Clean up any resources being used.
10          /// </summary>
11          /// <param name="disposing">true if managed resources should be
12  disposed; otherwise, false.</param>
13          protected override void Dispose(bool disposing)
14          {
15              if (disposing && (components != null))
16              {
17                  components.Dispose();
18              }
19              base.Dispose(disposing);
20          }
21
22          #region Windows Form Designer generated code
23
24          /// <summary>
25          /// Required method for Designer support - do not modify
26          /// the contents of this method with the code editor.
27          /// </summary>
28          private void InitializeComponent()
29          {
30              this.AddGravObj = new System.Windows.Forms.Button();
31              this.GravMagnitude = new System.Windows.Forms.TextBox();
32              this.GravLocationX = new System.Windows.Forms.TextBox();
33              this.label1 = new System.Windows.Forms.Label();
```

```
34              this.label2 = new System.Windows.Forms.Label();
35              this.GravLocationY = new System.Windows.Forms.TextBox();
36              this.AddCircle = new System.Windows.Forms.Button();
37              this.label3 = new System.Windows.Forms.Label();
38              this.CircleDiameterBox = new System.Windows.Forms.TextBox();
39              this.CircleVelXBox = new System.Windows.Forms.TextBox();
40              this.label4 = new System.Windows.Forms.Label();
41              this.label5 = new System.Windows.Forms.Label();
42              this.label6 = new System.Windows.Forms.Label();
43              this.CircleVelYBox = new System.Windows.Forms.TextBox();
44              this.label7 = new System.Windows.Forms.Label();
45              this.label8 = new System.Windows.Forms.Label();
46              this.label9 = new System.Windows.Forms.Label();
47              this.CirclePositionYBox = new System.Windows.Forms.TextBox();
48              this.label10 = new System.Windows.Forms.Label();
49              this.CirclePositionXBox = new System.Windows.Forms.TextBox();
50              this.label11 = new System.Windows.Forms.Label();
51              this.label12 = new System.Windows.Forms.Label();
52              this.CircleAccelYBox = new System.Windows.Forms.TextBox();
53              this.label13 = new System.Windows.Forms.Label();
54              this.CircleAccelXBox = new System.Windows.Forms.TextBox();
55              this.label14 = new System.Windows.Forms.Label();
56              this.ColourBox = new System.Windows.Forms.ComboBox();
57              this.label15 = new System.Windows.Forms.Label();
58              this.MassBox = new System.Windows.Forms.TextBox();
59              this.label16 = new System.Windows.Forms.Label();
60              this.SuspendLayout();
61              //
62              // AddGravObj
63              //
64              this.AddGravObj.Location = new System.Drawing.Point(27, 11);
65              this.AddGravObj.Margin = new System.Windows.Forms.Padding(2, 2, 2,
66  2);
67              this.AddGravObj.Name = "AddGravObj";
68              this.AddGravObj.Size = new System.Drawing.Size(95, 19);
69              this.AddGravObj.TabIndex = 0;
70              this.AddGravObj.Text = "Add Gravity object";
71              this.AddGravObj.UseVisualStyleBackColor = true;
72              this.AddGravObj.Click += new
73  System.EventHandler(this.AddGravObj_Click);
74              //
75              // GravMagnitude
76              //
77              this.GravMagnitude.Location = new System.Drawing.Point(128, 35);
78              this.GravMagnitude.Margin = new System.Windows.Forms.Padding(2, 2,
79  2, 2);
80              this.GravMagnitude.Name = "GravMagnitude";
81              this.GravMagnitude.Size = new System.Drawing.Size(76, 20);
82              this.GravMagnitude.TabIndex = 1;
83              this.GravMagnitude.Text = "50";
```

```
            this.GravMagnitude.TextChanged += new
System.EventHandler(this.GravMagnitude_TextChanged);
            //
            // GravLocationX
            //
            this.GravLocationX.Location = new System.Drawing.Point(134, 60);
            this.GravLocationX.Margin = new System.Windows.Forms.Padding(2, 2,
2, 2);
            this.GravLocationX.Name = "GravLocationX";
            this.GravLocationX.Size = new System.Drawing.Size(25, 20);
            this.GravLocationX.TabIndex = 2;
            this.GravLocationX.Text = "500";
            this.GravLocationX.TextChanged += new
System.EventHandler(this.GravLocationX_TextChanged);
            //
            // label1
            //
            this.label1.AutoSize = true;
            this.label1.Location = new System.Drawing.Point(50, 37);
            this.label1.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label1.Name = "label1";
            this.label1.Size = new System.Drawing.Size(71, 13);
            this.label1.TabIndex = 3;
            this.label1.Text = "Gravity Mass:";
            //
            // label2
            //
            this.label2.AutoSize = true;
            this.label2.Location = new System.Drawing.Point(37, 63);
            this.label2.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label2.Name = "label2";
            this.label2.Size = new System.Drawing.Size(87, 13);
            this.label2.TabIndex = 4;
            this.label2.Text = "Gravity Location:";
            //
            // GravLocationY
            //
            this.GravLocationY.Location = new System.Drawing.Point(178, 60);
            this.GravLocationY.Margin = new System.Windows.Forms.Padding(2, 2,
2, 2);
            this.GravLocationY.Name = "GravLocationY";
            this.GravLocationY.Size = new System.Drawing.Size(25, 20);
            this.GravLocationY.TabIndex = 5;
            this.GravLocationY.Text = "400";
            this.GravLocationY.TextChanged += new
System.EventHandler(this.GravLocationY_TextChanged);
            //
            // AddCircle
            //
            this.AddCircle.Location = new System.Drawing.Point(32, 131);
```

```
            this.AddCircle.Margin = new System.Windows.Forms.Padding(2, 2, 2,
2);
            this.AddCircle.Name = "AddCircle";
            this.AddCircle.Size = new System.Drawing.Size(60, 19);
            this.AddCircle.TabIndex = 6;
            this.AddCircle.Text = "Add Circle";
            this.AddCircle.UseVisualStyleBackColor = true;
            this.AddCircle.Click += new
System.EventHandler(this.AddCircle_Click);
            //
            // label3
            //
            this.label3.AutoSize = true;
            this.label3.Location = new System.Drawing.Point(27, 162);
            this.label3.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label3.Name = "label3";
            this.label3.Size = new System.Drawing.Size(81, 13);
            this.label3.TabIndex = 7;
            this.label3.Text = "Circle Diameter:";
            //
            // CircleDiameterBox
            //
            this.CircleDiameterBox.Location = new System.Drawing.Point(108,
160);
            this.CircleDiameterBox.Margin = new System.Windows.Forms.Padding(2,
2, 2, 2);
            this.CircleDiameterBox.Name = "CircleDiameterBox";
            this.CircleDiameterBox.Size = new System.Drawing.Size(76, 20);
            this.CircleDiameterBox.TabIndex = 8;
            this.CircleDiameterBox.Text = "100";
            this.CircleDiameterBox.TextChanged += new
System.EventHandler(this.CircleDiameterBox_TextChanged);
            //
            // CircleVelXBox
            //
            this.CircleVelXBox.Location = new System.Drawing.Point(115, 183);
            this.CircleVelXBox.Margin = new System.Windows.Forms.Padding(2, 2,
2, 2);
            this.CircleVelXBox.Name = "CircleVelXBox";
            this.CircleVelXBox.Size = new System.Drawing.Size(31, 20);
            this.CircleVelXBox.TabIndex = 10;
            this.CircleVelXBox.Text = "-10";
            this.CircleVelXBox.TextChanged += new
System.EventHandler(this.CircleVelXBox_TextChanged);
            //
            // label4
            //
            this.label4.AutoSize = true;
            this.label4.Location = new System.Drawing.Point(30, 183);
            this.label4.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
```

```
184             this.label4.Name = "label4";
185             this.label4.Size = new System.Drawing.Size(76, 13);
186             this.label4.TabIndex = 9;
187             this.label4.Text = "Circle Velocity:";
188             //
189             // label5
190             //
191             this.label5.AutoSize = true;
192             this.label5.Location = new System.Drawing.Point(99, 186);
193             this.label5.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
194             this.label5.Name = "label5";
195             this.label5.Size = new System.Drawing.Size(14, 13);
196             this.label5.TabIndex = 11;
197             this.label5.Text = "X";
198             //
199             // label6
200             //
201             this.label6.AutoSize = true;
202             this.label6.Location = new System.Drawing.Point(145, 186);
203             this.label6.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
204             this.label6.Name = "label6";
205             this.label6.Size = new System.Drawing.Size(14, 13);
206             this.label6.TabIndex = 13;
207             this.label6.Text = "Y";
208             //
209             // CircleVelYBox
210             //
211             this.CircleVelYBox.Location = new System.Drawing.Point(160, 183);
212             this.CircleVelYBox.Margin = new System.Windows.Forms.Padding(2, 2,
213 2, 2);
214             this.CircleVelYBox.Name = "CircleVelYBox";
215             this.CircleVelYBox.Size = new System.Drawing.Size(31, 20);
216             this.CircleVelYBox.TabIndex = 12;
217             this.CircleVelYBox.Text = "-100";
218             this.CircleVelYBox.TextChanged += new
219 System.EventHandler(this.CircleVelYBox_TextChanged);
220             //
221             // label7
222             //
223             this.label7.AutoSize = true;
224             this.label7.Location = new System.Drawing.Point(121, 63);
225             this.label7.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
226             this.label7.Name = "label7";
227             this.label7.Size = new System.Drawing.Size(14, 13);
228             this.label7.TabIndex = 14;
229             this.label7.Text = "X";
230             //
231             // label8
232             //
233             this.label8.AutoSize = true;
```

```
234             this.label8.Location = new System.Drawing.Point(165, 63);
235             this.label8.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
236             this.label8.Name = "label8";
237             this.label8.Size = new System.Drawing.Size(14, 13);
238             this.label8.TabIndex = 15;
239             this.label8.Text = "Y";
240             //
241             // label9
242             //
243             this.label9.AutoSize = true;
244             this.label9.Location = new System.Drawing.Point(145, 208);
245             this.label9.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
246             this.label9.Name = "label9";
247             this.label9.Size = new System.Drawing.Size(14, 13);
248             this.label9.TabIndex = 20;
249             this.label9.Text = "Y";
250             //
251             // CirclePositionYBox
252             //
253             this.CirclePositionYBox.Location = new System.Drawing.Point(160,
254 206);
255             this.CirclePositionYBox.Margin = new
256 System.Windows.Forms.Padding(2, 2, 2, 2);
257             this.CirclePositionYBox.Name = "CirclePositionYBox";
258             this.CirclePositionYBox.Size = new System.Drawing.Size(31, 20);
259             this.CirclePositionYBox.TabIndex = 19;
260             this.CirclePositionYBox.Text = "400";
261             this.CirclePositionYBox.TextChanged += new
262 System.EventHandler(this.CirclePositionYBox_TextChanged);
263             //
264             // label10
265             //
266             this.label10.AutoSize = true;
267             this.label10.Location = new System.Drawing.Point(99, 208);
268             this.label10.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
269             this.label10.Name = "label10";
270             this.label10.Size = new System.Drawing.Size(14, 13);
271             this.label10.TabIndex = 18;
272             this.label10.Text = "X";
273             //
274             // CirclePositionXBox
275             //
276             this.CirclePositionXBox.Location = new System.Drawing.Point(115,
277 206);
278             this.CirclePositionXBox.Margin = new
279 System.Windows.Forms.Padding(2, 2, 2, 2);
280             this.CirclePositionXBox.Name = "CirclePositionXBox";
281             this.CirclePositionXBox.Size = new System.Drawing.Size(31, 20);
282             this.CirclePositionXBox.TabIndex = 17;
283             this.CirclePositionXBox.Text = "500";
```

```
            this.CirclePositionXBox.TextChanged += new
System.EventHandler(this.CirclePositionXBox_TextChanged);
            //
            // label11
            //
            this.label11.AutoSize = true;
            this.label11.Location = new System.Drawing.Point(30, 208);
            this.label11.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label11.Name = "label11";
            this.label11.Size = new System.Drawing.Size(76, 13);
            this.label11.TabIndex = 16;
            this.label11.Text = "Circle Position:";
            //
            // label12
            //
            this.label12.AutoSize = true;
            this.label12.Location = new System.Drawing.Point(145, 229);
            this.label12.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label12.Name = "label12";
            this.label12.Size = new System.Drawing.Size(14, 13);
            this.label12.TabIndex = 25;
            this.label12.Text = "Y";
            //
            // CircleAccelYBox
            //
            this.CircleAccelYBox.Location = new System.Drawing.Point(160, 227);
            this.CircleAccelYBox.Margin = new System.Windows.Forms.Padding(2,
2, 2, 2);
            this.CircleAccelYBox.Name = "CircleAccelYBox";
            this.CircleAccelYBox.Size = new System.Drawing.Size(31, 20);
            this.CircleAccelYBox.TabIndex = 24;
            this.CircleAccelYBox.Text = "0";
            this.CircleAccelYBox.TextChanged += new
System.EventHandler(this.CircleAccelYBox_TextChanged);
            //
            // label13
            //
            this.label13.AutoSize = true;
            this.label13.Location = new System.Drawing.Point(99, 230);
            this.label13.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label13.Name = "label13";
            this.label13.Size = new System.Drawing.Size(14, 13);
            this.label13.TabIndex = 23;
            this.label13.Text = "X";
            //
            // CircleAccelXBox
            //
            this.CircleAccelXBox.Location = new System.Drawing.Point(115, 227);
            this.CircleAccelXBox.Margin = new System.Windows.Forms.Padding(2,
2, 2, 2);
```

```
            this.CircleAccelXBox.Name = "CircleAccelXBox";
            this.CircleAccelXBox.Size = new System.Drawing.Size(31, 20);
            this.CircleAccelXBox.TabIndex = 22;
            this.CircleAccelXBox.Text = "0";
            this.CircleAccelXBox.TextChanged += new
System.EventHandler(this.CircleAccelXBox_TextChanged);
            //
            // label14
            //
            this.label14.AutoSize = true;
            this.label14.Location = new System.Drawing.Point(10, 229);
            this.label14.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label14.Name = "label14";
            this.label14.Size = new System.Drawing.Size(97, 13);
            this.label14.TabIndex = 21;
            this.label14.Text = "Circle acceleration:";
            //
            // ColourBox
            //
            this.ColourBox.FormattingEnabled = true;
            this.ColourBox.Items.AddRange(new object[] {
            "Black",
            "Blue",
            "Red",
            "Green",
            "Yellow",
            "Purple",
            "Turquoise",
            "Magenta",
            "Pink",
            "Orange",
            "Brown"});
            this.ColourBox.Location = new System.Drawing.Point(102, 249);
            this.ColourBox.Margin = new System.Windows.Forms.Padding(2, 2, 2,
2);
            this.ColourBox.Name = "ColourBox";
            this.ColourBox.Size = new System.Drawing.Size(92, 21);
            this.ColourBox.TabIndex = 26;
            this.ColourBox.SelectedIndexChanged += new
System.EventHandler(this.ColourBox_SelectedIndexChanged);
            //
            // label15
            //
            this.label15.AutoSize = true;
            this.label15.Location = new System.Drawing.Point(64, 252);
            this.label15.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label15.Name = "label15";
            this.label15.Size = new System.Drawing.Size(40, 13);
            this.label15.TabIndex = 27;
            this.label15.Text = "Colour:";
```

```
            //
            // MassBox
            //
            this.MassBox.Location = new System.Drawing.Point(108, 272);
            this.MassBox.Margin = new System.Windows.Forms.Padding(2, 2, 2, 2);
            this.MassBox.Name = "MassBox";
            this.MassBox.Size = new System.Drawing.Size(76, 20);
            this.MassBox.TabIndex = 29;
            this.MassBox.Text = "1";
            this.MassBox.TextChanged += new
System.EventHandler(this.MassBox_TextChanged);
            //
            // label16
            //
            this.label16.AutoSize = true;
            this.label16.Location = new System.Drawing.Point(69, 275);
            this.label16.Margin = new System.Windows.Forms.Padding(2, 0, 2, 0);
            this.label16.Name = "label16";
            this.label16.Size = new System.Drawing.Size(35, 13);
            this.label16.TabIndex = 28;
            this.label16.Text = "Mass:";
            //
            // CircleEditor
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(226, 313);
            this.Controls.Add(this.MassBox);
            this.Controls.Add(this.label16);
            this.Controls.Add(this.label15);
            this.Controls.Add(this.ColourBox);
            this.Controls.Add(this.label12);
            this.Controls.Add(this.CircleAccelYBox);
            this.Controls.Add(this.label13);
            this.Controls.Add(this.CircleAccelXBox);
            this.Controls.Add(this.label14);
            this.Controls.Add(this.label9);
            this.Controls.Add(this.CirclePositionYBox);
            this.Controls.Add(this.label10);
            this.Controls.Add(this.CirclePositionXBox);
            this.Controls.Add(this.label11);
            this.Controls.Add(this.label8);
            this.Controls.Add(this.label7);
            this.Controls.Add(this.label6);
            this.Controls.Add(this.CircleVelYBox);
            this.Controls.Add(this.label5);
            this.Controls.Add(this.CircleVelXBox);
            this.Controls.Add(this.label4);
            this.Controls.Add(this.CircleDiameterBox);
            this.Controls.Add(this.label3);
```

```
434             this.Controls.Add(this.AddCircle);
435             this.Controls.Add(this.GravLocationY);
436             this.Controls.Add(this.label2);
437             this.Controls.Add(this.label1);
438             this.Controls.Add(this.GravLocationX);
439             this.Controls.Add(this.GravMagnitude);
440             this.Controls.Add(this.AddGravObj);
441             this.Margin = new System.Windows.Forms.Padding(2, 2, 2, 2);
442             this.Name = "CircleEditor";
443             this.Text = "CircleEditor";
444             this.ResumeLayout(false);
445             this.PerformLayout();
446 
447         }
448 
449         #endregion
450 
451         private System.Windows.Forms.Button AddGravObj;
452         private System.Windows.Forms.TextBox GravMagnitude;
453         private System.Windows.Forms.TextBox GravLocationX;
454         private System.Windows.Forms.Label label1;
455         private System.Windows.Forms.Label label2;
456         private System.Windows.Forms.TextBox GravLocationY;
457         private System.Windows.Forms.Button AddCircle;
458         private System.Windows.Forms.Label label3;
459         private System.Windows.Forms.TextBox CircleDiameterBox;
460         private System.Windows.Forms.TextBox CircleVelXBox;
461         private System.Windows.Forms.Label label4;
462         private System.Windows.Forms.Label label5;
463         private System.Windows.Forms.Label label6;
464         private System.Windows.Forms.TextBox CircleVelYBox;
465         private System.Windows.Forms.Label label7;
466         private System.Windows.Forms.Label label8;
467         private System.Windows.Forms.Label label9;
468         private System.Windows.Forms.TextBox CirclePositionYBox;
469         private System.Windows.Forms.Label label10;
470         private System.Windows.Forms.TextBox CirclePositionXBox;
471         private System.Windows.Forms.Label label11;
472         private System.Windows.Forms.Label label12;
473         private System.Windows.Forms.TextBox CircleAccelYBox;
474         private System.Windows.Forms.Label label13;
475         private System.Windows.Forms.TextBox CircleAccelXBox;
476         private System.Windows.Forms.Label label14;
477         private System.Windows.Forms.ComboBox ColourBox;
478         private System.Windows.Forms.Label label15;
479         private System.Windows.Forms.TextBox MassBox;
480         private System.Windows.Forms.Label label16;
481     }
```

Testing

I have included some testing for the circle simulation, though was initially my prototype so there are many issues which are fixed in the polygon simulation which should be seen as my final product. The video and screenshots will show the objects changing colour at some points, this is intentional to show when the program thinks that a shape is colliding, as when it collides its colour will be changed to a new random colour.

All tests are for the polygon simulation, unless mentioned otherwise.

All tests use 60 fps, (form is set to repeat calculations every 1000/60 mS)

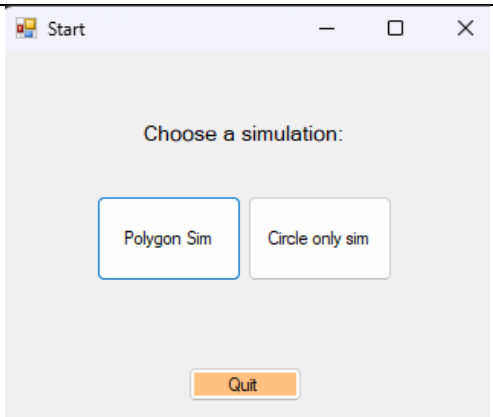
Test overview:

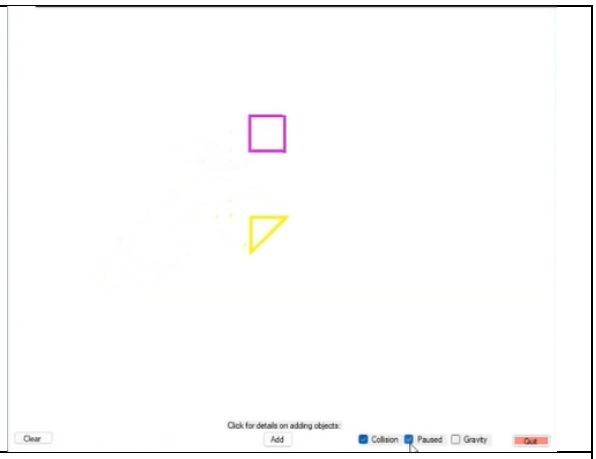
The time stamp column relates to the time in the attached video where this test is seen

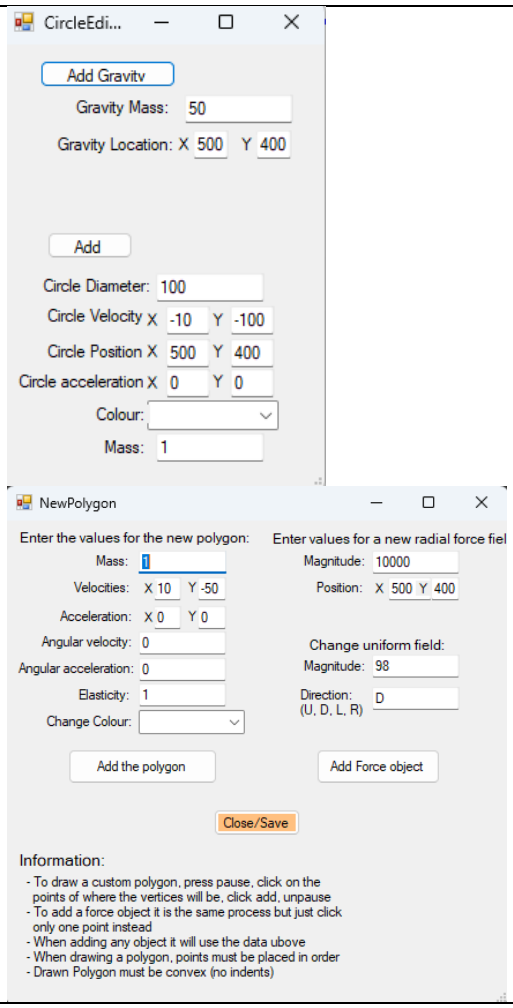
Section	Time stamp	Goal	Title	Pass or fail
GUI	0:01	1.1.1	Can all menus open well	
	0:01	1.1.2	Does pausing the simulation work	
	0:01	1.1.3	Quit button	
	0:25	1.1.4	Clear button, and then adding to the simulation	
	0:41	1.2	Ability to change values	
Customisation of forces and objects	0:52	2.0.1	Circle's movement	
	1:01	2.0.2	Polygon's Movement	
	1:05	2.0.3	Polygons spinning	
	1:10	2.1.1	Does polygon data change successfully	
	1:39	2.1.2	Circles data changing successfully	
	2:09	2.1.3	Error message on invalid inputs	
	2:21	2.1.4	Scalability of uniform gravity	
Variety of shapes ~ usability of inputting a new shape	2:39	3.1.1	Can different shapes be made	
	3:03	3.1.2	Is adding secure	
	N/A	3.2	Are Polygons receiving a correct centre of mass	
	2:39	3.4.1	Drawing vertices for a new polygon	
	3:12	3.4.1.1	Can drawn shapes be edited	
	3:49	3.4.2	Is drawing secure	
	N/A	3.4.3	Clearing and drawing	
	1:39	3.5	Scalability of new circles	
Collisions at appropriate frame rate	3:58	4.0.1	Circles to wall collision	
	4:06	4.0.1.1	Circles to wall collision at high velocity	
	4:29	4.0.2	Collisions between circles at low velocity	
	4:42	4.0.2.1	Collisions between circles at a normal or high velocity	
	4:47	4.0.3	Polygon to polygon collision	

	4:52	4.0.4	Polygon collision with angular velocity	
	4:56	4.0.5	Tunnelling due to high speed	
	4:58	4.0.6	Collision with large difference in size	
	5:04	4.0.7	Collisions at small angles	
	5:12	4.0.8	Collisions at small angles at high speeds	
	5:17	4.0.9	Polygons with many edges colliding	
	5:21	4.0.10	Polygons with many edges colliding at high speeds	
	5:24	4.0.11	Collisions between polygons with very small angles	
	5:30	4.0.12.1	Very different masses with elasticity of 1	
	5:34	4.0.12.2	Very different masses with elasticity of 0.5	
	5:38	4.0.13	Corner collisions	
	5:42	4.2	Can framerate change well	
Gravity object	5:52	5.0.1	Attraction to force	
	6:03	5.0.2	Orbits around a force	
	6:16	5.0.3	Multiple forces	
	6:29	5.0.4	Scalability of the force	
	6:51	5.0.5.1	Multiple forces and objects with collision	
	7:03	5.0.5.2	Multiple forces and objects no collision	

Tests with reference to original goals:

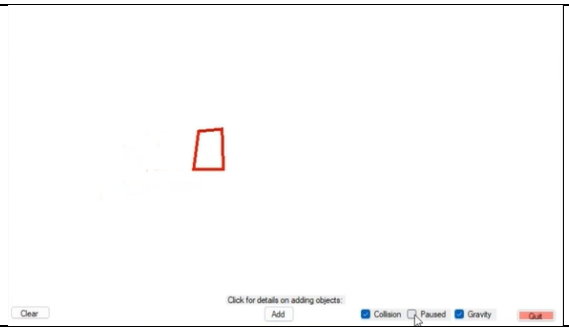
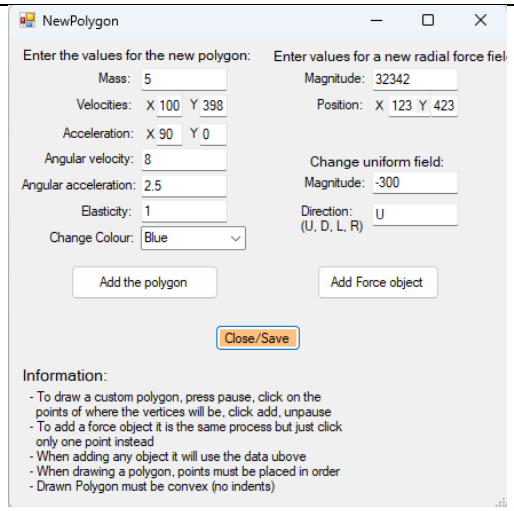
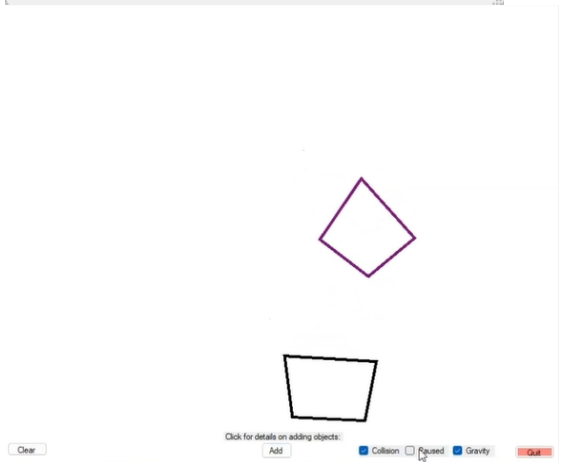
Tests	Evidence
GUI	
1.1.1 Can all menus open well Testing whether the start menu works as expected and other opening menus: Pressing each button leads to their respective simulations, this continues for every button that is supposed to lead to a new window.	

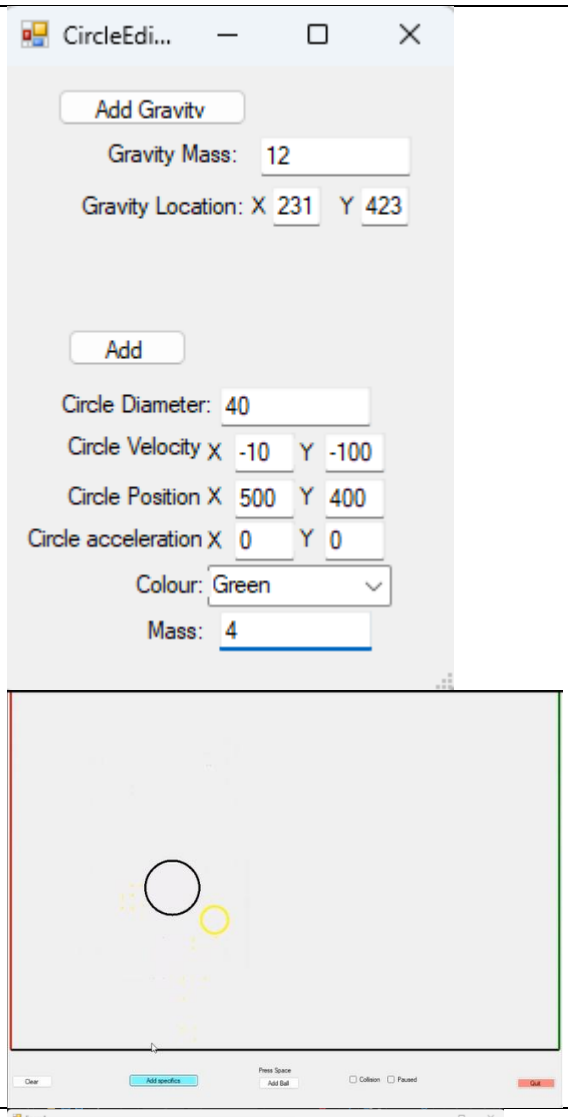
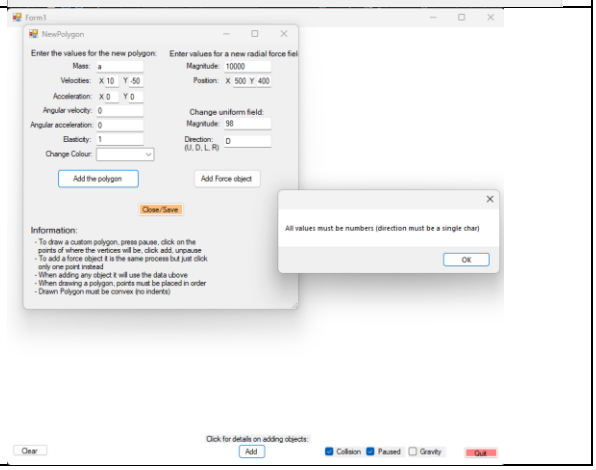
1.1.2 Does pausing the simulation work	
When the pause checkbox is ticked all objects in the form should be completely still. This works for both simulations as all velocities and angular velocities have stopped. While paused, the form and opening any other forms still works perfectly.	
1.1.3 Quit button	
A simple button but works in all the forms by closing the form being currently used	
1.1.4 clear button, and then adding to the simulation	
This button clears the form of any force or shape objects, after which any other forces or objects can be added. Disclaimer: video only shows clearing of shapes, though in other tests you can see forces also being cleared	

1.2 Changing values	
Simulation details can be changed by clicking the add buttons, we can then see all of the editable pieces of data for the simulation in text boxes which can all be changed in real time. This image shows the details that can be changed for new objects to be added. The circle simulation has a similar thing. In The polygon version we can add a new shape or save the values to then add a drawn shape.	

Customisation of objects data and gravity

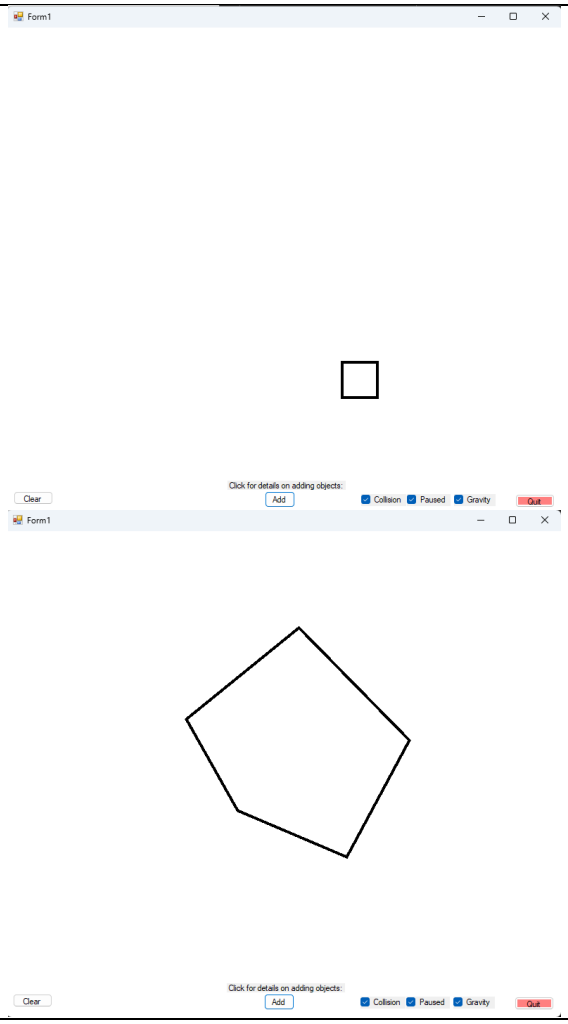
2.0.1 Circle's movement	
Circle simulation has movement that works since velocities, acceleration and positions are updated appropriately.	

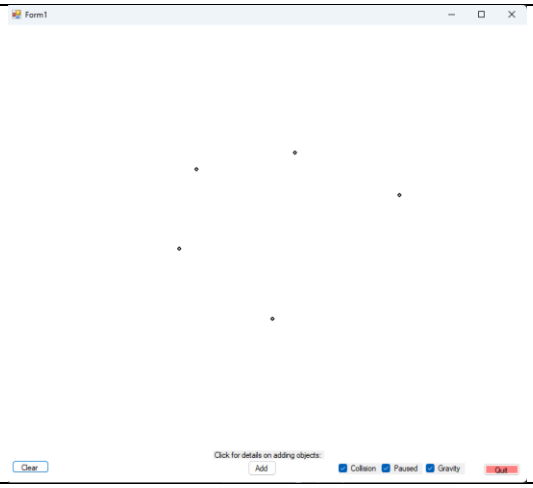
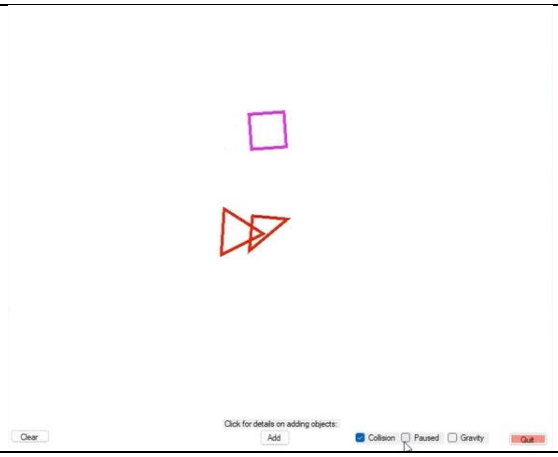
2.0.2 Polygon's Movement Polygon simulation shapes also move around perfectly with respect to force objects (gravitational fields), their own initial acceleration and any Uniform field. Once again this is because velocities, accelerations and positions all update.	
2.0.3 Polygons spinning Polygon's angular momentum and acceleration also works as expected since they spin and can get faster if an acceleration is added. Or change angular velocity after a collision	
2.1.1 Does polygon data change successfully Polygons can be made with a range of possible values successfully, including colour change, and all the other shown values.	 

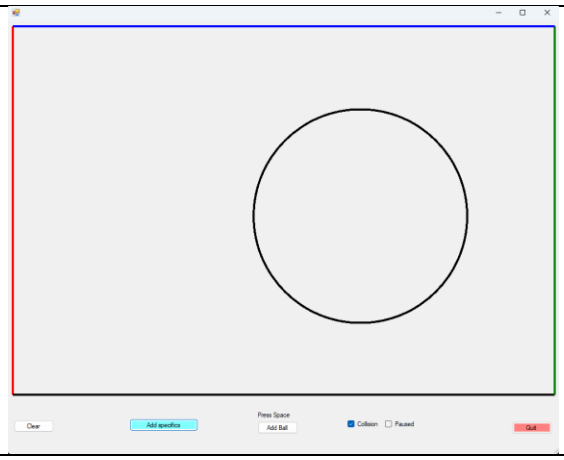
2.1.2 Circles data changing success	
The properties of the circles and circle's forces can also be changed in much the same way. **Disclaimer: in video there is a visual bug where the yellow circle leaves behind a trail, though this does not appear in the actual program.**	
2.1.3 Error message on invalid inputs	
Error message correctly appears after incorrect float or char inputs where applicable.	

2.1.4 scalability of uniform gravity	
Changing the direction and scale of uniform fields cause objects to move in that new direction perfectly.	

Veriety of shapes ~ usability of inputting a new shape

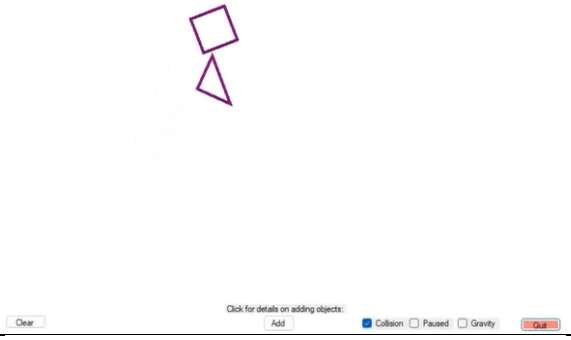
3.1.1 Can different shapes be made	
Default shapes are squares not circles as I was unable to integrate circles and polygons into the same simulation, though this is not a major issue as complex polygons can be inputted which is the main goal Default: Complex polygon example:	
3.1.2 Is adding secure	
When the Add values are saved no shapes are generated or should be generated, though when one is intentionally generated, every time the values are saved after that cause a new default shape to be generated.	
3.2 are polygons receiving a correct centre of mass	
Polygons are given a centre of mass at their creation; this allows rotations to occur naturally	

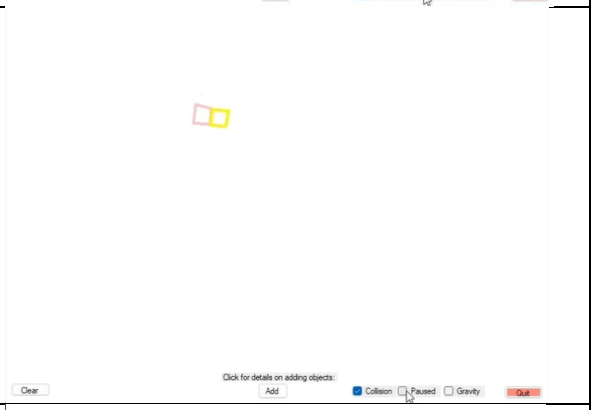
around that point which is also treated as the polygons position.	
3.4.1 Drawing vertices for a new polygon	
The user can input a polygon by simply pausing and clicking on where the vertices of the polygon will be, these can be modified just as easily. The resultant polygon works just as well as another.	
3.4.1.1 Can drawn shapes be edited	
Yes, when the add details are changed and a new shape is drawn, the new shape will have the previously saved values.	
3.4.2 is drawing secure	
When a polygon is drawn through or around another, the two objects successfully get pushed apart.	
3.4.3 Clearing and drawing	
If a point or several points are placed, but not added, and then the clear button is pressed it causes any new polygon drawn to include the points before clear was pressed. To fix this I added an else if statement below where AddDrawing is set to true when the drawn poly list is 0. This means that a new polygon will only be added when a new drawn point series is made.	

3.5 scalability of new circles Inputted diameter works for the circle simulation, and polygon size is determined by the drawing. This is seen in the video for changing circle values	

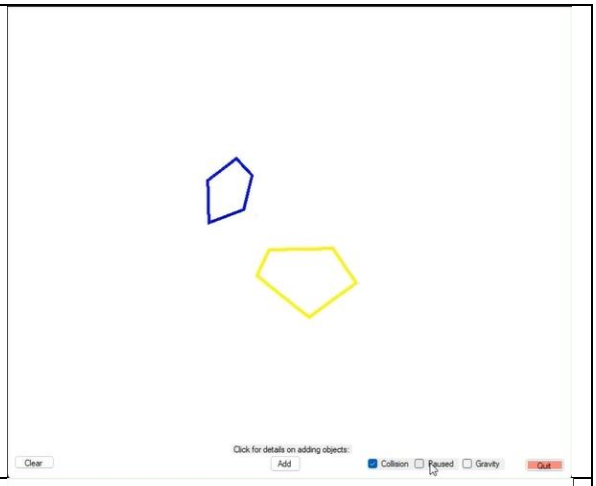
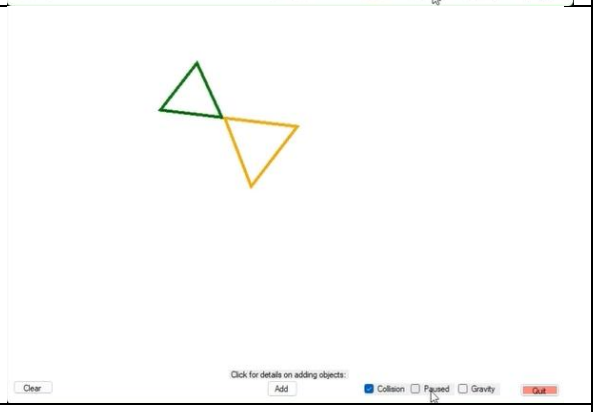
Collisions at appropriate frame rate

4.0.1 Circles to wall collision Works well, circle's velocities change as expected with each collision with a wall	
4.0.1.1 Circles to wall collision at high velocity This works for velocities of 1000, at 10,000 it works as expected but the ball eventually tunnels out of the walls, making ~10,000 the upper limit the simulation can handle at a framerate of 60.	
4.0.2 collisions between circles Collision between circles works though with admittedly a little jitter as they collide, though this is acceptable.	

4.0.2.1 collisions between circles at normal or high velocity This completely breaks the collisions, the algorithm doesn't seem to know what to do with itself and the circles go through each other and go at ridiculous speeds.	
4.0.3 polygon to polygon collision The polygons look to collide pretty much perfectly as they bounce off each other as expected.	
4.0.4 polygon collision with angular velocity The shapes do collide somewhat well, but their resultant velocities and angular velocities are far too great to be acceptable	

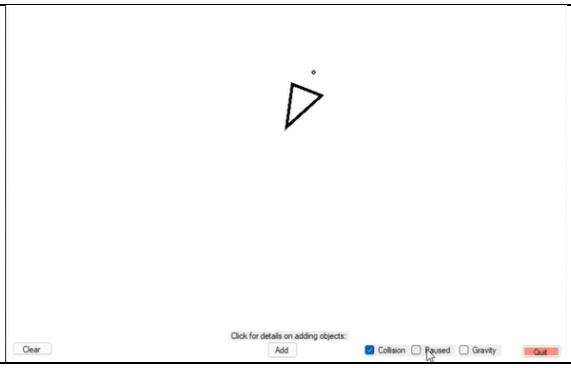
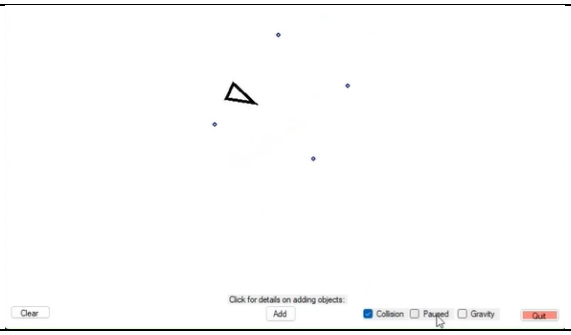
4.0.5 Tunnelling due to high speed	
There is a velocity that causes tunnelling to occur (objects to phase through each other) in the polygon simulation ~ 1000 pixels/s.	
4.0.6 Collision with large difference in size	
Small objects colliding with large objects cause the collision to look a bit odd with slight merging of the objects but nothing too drastic, so it is acceptable.	
4.0.7 Collisions with small angles	
Shallow angle collisions is also something that commonly causes tunnelling or an unstable collision, though in this case the collision does work, however I must say that it does seem to be bounce slightly more than it should.	
4.0.8 Collisions with small angles at high speeds	
Shallow angles at high speeds however does not work as the slight imperfections in the previous test are exemplified due to the high speed. This causes the collision to be very over exaggerated.	

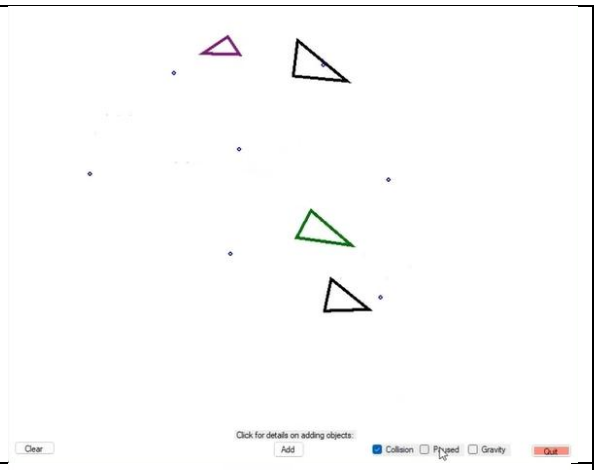
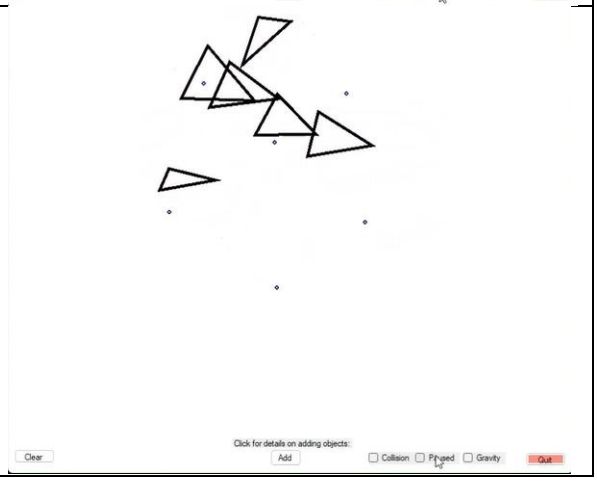
4.0.9 Polygons with many edges colliding This causes the polygons to collide slightly before colliding which is a clear bug.	
4.0.10 Polygons with many edges colliding at high speed Many edges and high velocities works well since the collisions stay realistic, more so than the previous test.	
4.0.11 collisions between polygons with very small angles Testing for other kinds of extreme shapes with extreme angles (a triangle with a very small angle) and pencil like objects causes collisions to go very wrong as the two objects are just constantly colliding for no apparent reason.	

4.0.12.1 Very different masses with elasticity of 1 Collisions between very light and very heavy objects (one has 1 kg, and the other is 1000 kg) cause an expected result of the light object being sent flying while the other hardly moves. This would make sense since they have very different momentums and the elasticity is 1.	
4.0.12.2 very different masses with elasticity 0.5 This gives practically the same result which I couldn't really say for sure is what I should expect, it is acceptable though so I will pass it.	
4.0.13 corner collisions When two corners directly collide the result is expected as the two objects bounce off without any drastic movements	
4.2 can framerate change well A change in framerate can make the simulation look smoother and reduce tunnelling due to high speeds in each object. As seen in the video the program can do 120fps and 1000fps, and therefore has a very wide range of frame rates	

Disclaimer: the video itself is not a high framerate, but the program will be outputting the appropriate framerate which is what we need.	

Force of gravity ~ ability to change

5.0.1 attraction to force This works just as expected, object is constantly accelerating towards the force point. The acceleration itself also increases as the distance decreases, as expected.	
5.0.2 orbits around a force The object goes around the force point as expected, slingshotting as expected with orbits.	
5.0.3 multiple forces The object moves between each force point with no issues	
5.0.4 scalability of forces An increase in the mass of the force leads to an increase in acceleration for the object around it in an expected fashion	

5.0.5.1 multiple forces and objects with collision Forces work well but collisions are far too over exaggerated due to the issues seen in the collision section	
5.0.5.2 multiple forces and objects no collision This, as you would guess, works much better as all of the objects orbit as you would expect.	

Evaluation

Here I have explained my final thoughts on the project, the changes I might make in another version and how well the final product aligns with what my client wanted and my own personal goals.

Individual goal success:

1. The program has a successful GUI that can be used to change data
    1.1. The menu and all buttons to separate forms work perfectly
    1.2. Settings are not an option to change from the start menu, but they can be accessed in real time during the simulation so the lack of it in the start menu is not an issue since settings can be changed and objects can then be added.
2. All data values can be successfully changed during run time
    2.1. Gravity, when on, pulls all objects linearly in the chosen direction and magnitude successfully
    2.2. One class is used for force objects, but the uniform gravity has no need for its own class, so it's better and more efficient to do it the way I have done where it just changes the accelerations
    2.3. This does not occur for uniform fields, as you would expect, but for the forces objects the distance directly corresponds to the amount of force and therefore acceleration felt by each object
3. Any kind of complex shape can be made
    3.1. Each shape does have the same default values, but more complex shapes can successfully be added to the simulation
    3.2. The centre of mass algorithm works perfectly and is only ran on the initialization of a new polygon, after this it is treated as the objects position
    3.3. The user can simply click on where they would like the vertices of the polygon to be, which I would say is the most intuitive way for a complex polygon to be implemented.
    3.4. The drawing editor works just as described in the goal
    3.5. Shape size ties to the drawing of the shape perfectly
4. Collisions and the simulation can perfectly run at 60fps and any reasonable frame rate beyond that
    4.1. This was not achieved as I did not implement a multi-phase collision algorithm as I described in the goal, however performance was not an issue so for the small simulation I have made I would say that it is okay.
        4.1.1. The collision algorithm itself was successful for the most part but has many issues which would need to be fixed in a future version
    4.2. Fps is perfectly maintained
    4.3. Frame rate works exactly as I described due to implementation of double buffered graphics
5. Force of gravity works perfectly but could do with the extra features I mentioned in my goals
    5.1. This was not achieved due to time constraints
        5.1.1. However the gravity itself in force objects works perfectly.
    5.2. As mentioned, this was not completed

5.3. Performance is not negatively affected due to any number of force objects, making this a success

6. Code is in a state that it can be easily implemented upon
    6.1. This was achieved as there are many different classes for different concepts where necessary
    6.2. This is achieved since different objects can inherit from the class entity
    6.3. A class for force was achieved, though, due to time constraints, I did not manage to implement friction or air resistance, however the force class is made so that these other forces can use it, allowing for further expansion of the program easily
7. I did not end up implementing any kind of database.

Did I achieve my final project?

Considering my project, I would say that I have certainly reached my goals in some areas, though come far too short in others. I was successful in implementing a program that could simulate the movement of objects of any kind of rigid-body shape with straight sides dependent on their own velocities and acceleration. The shapes are then able to react to any kind of or combination of gravitational fields in a realistic manner while being able to detect collisions between each other and then resolve those collisions somewhat realistically in most scenarios to find their new linear and angular velocities. This is all condensed into an easy-to-use GUI. Therefore, I have reached goal 5, 3, 2 and 1 from my analysis. However, it is undeniable that I have failed in some aspects of goal 4 since the resolving method of these collisions is not always accurate due to many times when it causes the shapes to bounce off in odd directions, and the absence of a CCD (continuous collision detection) algorithm, due to time constraints, is likely the cause for shapes to sometimes tunnel through each other at high speeds. Instead of implementing a CCD I decided to allocate more time towards the ease of use for the client in adding new items to the simulation, and extra features that I initially wanted to include such as the force objects.

Collision detection can also sometimes falter when working with shapes that have extremely large and small angles, meaning that the accuracy of the GJK algorithm could be improved. I was also not able to reach my 7th goal of implementing a database, which does remove functionality for saving instances of a simulation or customizing objects to have different materials, though this is not essential for a real time physics simulation, or a feature requested by the client, so it is not much of a loss. I did however mostly achieve my 6th goal since my code is easy to expand upon due to the existence of many algorithms and classes to aid any kind of expansion. For example, there are several methods in the poly manager class for simple vector calculations that could be used for the addition of any further algorithms. The different classes themselves can also be used for any new additions as polytope and simplex objects can be used for any kind of abstract data type; moreover, the entity class can be inherited from to make more classes like polygon. Therefore, there are multiple parts to the program which can aid in adding to the simulation's performance and accuracy if the client wishes to. The robustness of my simulation, which was a part of my 6th goal, was achieved by tests for varying masses, elasticities, angles of collisions and number of shapes; however, this goal does fall short due to the varying accuracy during high angular velocities, and acute angles of collisions.

What would I change next time?

If I were to repeat this project, there are several changes I would make. Obviously, I would need to improve the collision system first. This could be achieved by implementing a CCD and doing much more working out to find how to resolve collisions as well as more online research to make the simulation more robust to as many different scenarios as possible.

Otherwise, I would want to make the simulation be able to simulate orbits more realistically by making every object have its own force object at its centre of mass so that the shapes could all attract each other at sufficiently high masses. Furthermore, I would want to make gravitational forces affect the orientation of shapes depending on the dispersion of mass around their centre of mass since this would make the movements of shapes around force objects look more realistic. The simulation could also be improved to allow circles and polygons to work together. Moreover, the program could include support for resizing depending on the size of the window, I would also like to allow for more customisation with features such as allowing the user to implement 1000s of objects easily to stress test the simulation; for something like this it would be best if I also implemented a broad and narrow phase algorithm for collisions to improve performance. I would also like to reach my 7th goal of implementing a database for storing different kinds of scenarios, whether those are preset by me or saved by the user.

Final thoughts

In conclusion I am happy with what I was able to produce as my final project even though I have a lot that I could improve about it. However, I would be happy to re-do it since I could use what I have learned to make a much better final product that could simulate physics in a more realistic and efficient way. The project itself has not only improved my fluency in C# but has also taught me a lot about how to think about solutions to problems in ways that I can put into code; this is seen in how I have learnt to make and use abstract data types such as queues and stacks and have become far more comfortable with using object-oriented programming. The topic of a physics simulation has allowed me to learn about new vector and graph concepts which are necessary for making these kinds of programs in industries such as game development and real time simulations. Therefore, making this simulation has helped me improve a lot in multiple aspects related to computer science and project development, which will help me for my future in any kind of related industry. My client wanted a 2d physics engine/simulation that could resolve rigid-body collisions between polygons, allow for force customisation and further expansion, with these initial goals in mind I can say that my project reaches these objectives.